

A Unified Theory of Closed-Loop Active Learning for Retinal Ganglion Cells:

Gaussian-Process Inference, Information Utility, and Guided Diffusion

Consolidated theory reference

July 11, 2026

Abstract

This is a single, self-contained reference for the theory and mathematics of a closed-loop, actively-learning electrophysiology experiment on retinal ganglion cells. A multi-electrode array records spikes while natural grayscale images are displayed; a Gaussian-process (GP) model with an arc-cosine kernel over a structured spatial prior learns each neuron’s receptive field and tuning from Poisson spike counts by sparse variational inference; and *active learning* chooses the next image to maximize expected information gain about that tuning. The document develops, in one uniform notation: the generative model and the arc-cosine kernel with its structured prior $C(\beta, \rho, \xi_0)$ (Part I); sparse variational inference, the eigenspace representation and the two current implementations, and the EM-style training algorithm with its E-, F-, and M-steps and analytical/VJP gradients (Part II); the information utility for active learning, together with the predictive-conditioning, subspace, divergence, and numerical results beneath it (Part III); and guided diffusion, which extends the loop from *selecting* stimuli out of a pool to *synthesizing* them by steering a denoising diffusion model with the GP utility gradient (Part IV). Throughout, the current implementation is the ground truth; reconciled discrepancies and open caveats are collected in Appendix B. The treatment is restricted to the spatial GP (one image per input) and to the current implementations; a deprecated legacy engine is out of scope.

Contents

Scope, conventions, and reading guide	5
I Foundations: the generative model and its prior	5
1 Introduction: the closed-loop active-learning problem	6
1.1 The experiment: RGCs, the array, natural images, the closed loop	6
1.2 The active-learning goal	6
1.3 Roadmap of the document	6
2 The generative model	7
2.1 Latent Gaussian process, exponential link, Poisson counts	7
2.2 Why the GP models the latent, not the rate	7
3 The arc-cosine kernel and the structured spatial prior	8
3.1 The order-1 arc-cosine kernel	8
3.2 Non-stationarity and the self-kernel identity	9

3.3	The structured spatial prior covariance C	9
3.4	Raw parameterization and constraints	10
3.5	Kernel gradient with respect to the input, $\nabla_x K$	11
3.6	Kernel gradients with respect to the hyperparameters, $\partial C / \partial \theta$	11
3.7	Numerical stability	12
II	Inference: variational learning of the tuning	13
4	Sparse variational Gaussian-process inference	13
4.1	Poisson breaks conjugacy	13
4.2	Inducing points and the variational posterior	13
4.3	Projected posterior moments	14
4.4	The ELBO	14
4.5	Expected firing rate via the Gaussian MGF	15
4.6	Prediction: expected spike count	15
4.7	The posterior cross-covariance pitfall	15
4.8	Whitening	16
5	The eigenspace representation and the two implementations	16
5.1	Eigendecomposition of the inducing gram	17
5.2	The eigenspace state	17
5.3	Posterior moments in the eigenbasis	18
5.4	ELBO and KL in the eigenbasis	18
5.5	The recompute seam after the M-step	18
5.6	The <code>default_gpy</code> path	19
5.7	Why <code>vargp_direct</code> manages parameters directly	19
5.8	Shared math, divergent representation	20
5.9	Online growth and the checkpoint seam	20
6	The training algorithm: EM-style coordinate ascent	21
6.1	Overview: three coordinate blocks, one ELBO, one loop	22
6.2	The E-step: a Newton update of the variational posterior	22
6.2.1	Why the update is cheap: the eigenspace realisation	24
6.2.2	Iteration, damping, and the divergence guard	24
6.3	The F-step: fitting the firing-rate function	24
6.4	The M-step: updating the kernel hyperparameters	25
6.4.1	The analytical gradient chain	25
6.4.2	Two gradient systems — and why the VJP is not the M-step	27
6.4.3	Numerical guardrails (M-step closure)	27
III	Active learning: the information utility	28
7	The acquisition objective	28
7.1	Utility as expected entropy reduction and mutual information	29
7.2	The marginal spike-count entropy H_{marg}	30
7.3	The standard (production) utility	30
7.3.1	The aleatoric noise entropy $\mathbb{E}[H_{\text{noise}}]$ in closed form	31

7.4	The distribution-aware (research) utility	31
7.4.1	Conditional GP moments via Gaussian conditioning	31
7.4.2	The deterministic special case	32
7.5	Why we sample the latent instead of using its mean	33
7.6	The entropy landscape over image space	33
7.7	Code realization and conventions	34
8	The deep layer: conditioning, subspace, divergence, and numerics	35
8.1	GP moments and Gaussian conditioning (Proof III)	36
8.1.1	The self-kernel identity	36
8.1.2	Variational posterior moments	36
8.1.3	Conditioning on an observed latent	37
8.1.4	Log-firing transform and the DA utility in three scalars	37
8.2	The predictive distribution conditioned on a new observation	37
8.2.1	The rank-1 posterior update	38
8.2.2	Sparse vs. augmented predictive	38
8.2.3	Equivalence of predictive means	38
8.3	Subspace theory	39
8.3.1	The kernel touches x only through the C -eigenspace	40
8.3.2	Three bases and the Szegő/Fourier equivalence	40
8.3.3	Subspace $\neq$ naturalness constraint	41
8.4	Divergence theorems (Proof I): norm scaling and the utility blow-up	41
8.4.1	Growth rate of the diverging utility	42
8.4.2	The $\sigma_0^2 > 0$ damping and the alignment peak at $\kappa = 1$	42
8.5	Kernel solutions (Proof II)	43
8.5.1	Normalized arc-cosine kernel (project onto the unit sphere)	43
8.5.2	Biologically plausible saturation kernel (arc-sin)	44
8.6	Numerical analysis of the standard utility	44
8.6.1	The closed-form decomposition	44
8.6.2	Laplace approximation and the log-space Lambert-W fix	45
8.6.3	Four independent residual failure modes	45
8.6.4	“Utility diverges under gradient ascent,” re-examined	46
IV	From selection to synthesis: guided diffusion	47
9	Guided diffusion for stimulus synthesis	47
9.1	Motivation: synthesize rather than select	47
9.2	Diffusion fundamentals	48
9.2.1	Forward (noising) process	48
9.2.2	Score $\leftrightarrow$ noise, and the training objective	49
9.2.3	Reverse (denoising) process and the Tweedie estimate	49
9.2.4	The U-Net noise predictor and DDIM acceleration	49
9.3	Guidance: steering the reverse process with the GP utility gradient	50
9.3.1	Classifier-style guidance	50
9.3.2	The guided reverse update (two equivalent forms)	50
9.3.3	The GP link: what the guidance signal is	50
9.3.4	Full versus approximate guidance gradient	51

9.3.5	The four approaches (A–D) and the adopted method	51
9.3.6	The firing-rate guard	52
9.4	Implementation status and open issues	52
A	Notation reference	54
B	Reconciled discrepancies and open caveats	57
B.1	Reconciled (code is authoritative)	57
B.2	Open caveats (carried honestly)	57
C	Glossary of experimental terms	58

Scope, conventions, and reading guide

This document is a single self-contained reference for the theory and mathematics behind a closed-loop, actively-learning electrophysiology experiment on retinal ganglion cells (RGCs). A multi-electrode array records spikes while natural grayscale images are displayed; a Gaussian-process (GP) model learns each neuron’s receptive field (RF) and tuning from spike counts, and *active learning* chooses the next image so as to maximize information gain about that tuning. Part 9 extends the loop from *selecting* images out of a fixed pool to *synthesizing* them with a guided diffusion model.

Scope. The treatment is restricted to the **spatial** GP: the input is a single image and the latent is a scalar function over one image. All spatiotemporal machinery (a temporal covariance factor, the Kronecker product $C = C_{\text{spatial}} \otimes C_{\text{temporal}}$, and temporal warping) is deliberately out of scope. Two **current** implementations are described and treated as ground truth: the eigenspace variational GP (`vargp_direct`: the `eigenspace_*.py` modules, class `DirectVGPMModel`) and the standard GPyTorch variational path (`default_gpy`: `gpy_*.py`, class `VariationalGPMModel`). A deprecated legacy engine (`vargp_old`) is not described; where the current code reuses a piece of its mathematics, that mathematics is presented as the current code runs it.

Conventions.

- **Ground truth is the current code.** Where an internal note or a LaTeX write-up disagreed with the shipped implementation, the code wins; the discrepancy is flagged in place with a **Caveat** box and collected in Appendix B.
- **Notation is uniform.** One symbol denotes one concept throughout; the full table is Appendix A. A handful of symbols that collide across the literature are disambiguated there (e.g. the kernel smoothness ρ vs. the posterior correlation ρ_λ ; the GP prior width β vs. the diffusion noise schedule β_t ; the three entropies $H_{\text{marg}}, H_{\text{noise}}, H_{\text{cond}}$).
- **Spike counts.** The spike count is written y in the model and inference parts; the utility part follows the entropy-literature convention and writes the count realization r and the count random variable R (with r_{max} its truncation). These denote the same observed quantity, $y \equiv r$.

How to read this document. The four parts are logically ordered. Part 2–3 sets up the generative model and the arc-cosine kernel with its structured spatial prior. Part II (4–6) develops sparse variational inference, the eigenspace representation and the two implementations, and the EM-style training algorithm (E-, F-, and M-steps). Part III (7–8) is the active-learning core: the information utility and the proofs, subspace theory, and numerics beneath it. Part IV (9) is guided diffusion for stimulus synthesis. Forward and backward cross-references connect the utility gradient in Part III and Part IV back to the kernel input-gradient of Part I, and the training gradients back to the ELBO of Part II.

Part I

Foundations: the generative model and its prior

1 Introduction: the closed-loop active-learning problem

1.1 The experiment: RGCs, the array, natural images, the closed loop

A multi-electrode array records the spiking activity of retinal ganglion cells (RGCs) while natural grayscale images are displayed as the visual stimulus. For a single neuron and a single image x the recorded observation is a *spike count* y — the number of spikes emitted in a fixed post-stimulus window. Each image is a vectorized 108×108 grayscale frame, so $x \in \mathbb{R}^{n_{\text{px}}}$ with $n_{\text{px}} = 108^2 = 11664$; none of the formulas below assume this particular dimension.

The neuron’s *receptive field* (RF) — the region of the image and the spatial pattern within it that drives the cell — is unknown a priori and must be inferred from the recorded counts. We place a Gaussian-process (GP) model on each neuron’s latent tuning function and update it online as counts arrive. The loop is *closed*: after each response the model is refit and used to choose the next image, so the stimuli shown depend on the responses already collected.

1.2 The active-learning goal

The images are not shown in a fixed order. At each step the next stimulus is chosen to be maximally informative about the neuron’s tuning — this is *active learning*. Concretely, a candidate image x^* is scored by an information *utility* $U_{\text{std}}(x^*)$ (Part III) that measures the expected reduction in uncertainty about the neuron’s latent firing behaviour if x^* were shown next; the image maximizing the utility over the available pool is displayed. A second, research variant U_{DA} scores information about firing rates over the natural-image distribution $p(x)$. Because scoring and selecting from a fixed pool is limited by the pool, a final stage (Part IV) instead *synthesizes* a natural-image-like stimulus that maximizes the utility, using a diffusion prior steered by the utility gradient. All three stages consume two derivatives of the GP prior kernel that are established in this part: the input gradient $\nabla_x K$ (10) (utility and diffusion guidance) and the hyperparameter gradients $\partial C / \partial \theta$ (12) (model fitting).

1.3 Roadmap of the document

The document proceeds in the order the pipeline runs.

- **Part I (this part) — the model and its prior.** The generative model (§2): a latent GP over image space, an exponential link, and Poisson spike counts. The prior kernel (§3): the order-1 arc-cosine (ReLU) kernel built on a structured spatial prior covariance $C(\text{Amp}, \beta, \rho, \xi_0)$ that encodes a local, smooth, RF-centred weighting, together with the two kernel gradients the rest of the pipeline needs.
- **Part II — inference.** The Poisson likelihood breaks conjugacy, so the posterior over the latent is approximated by a sparse variational GP on M inducing points with Gaussian posterior $q(\tilde{\lambda}) = \mathcal{N}(m, V)$, optimized against the evidence lower bound (ELBO). Two numerically distinct realizations of the *same* model (an eigenspace projection and a Cholesky-whitened GPyTorch path) and the EM-style coordinate-ascent training loop (E-, F-, M-steps).

- **Part III — active learning.** The information utility: the marginal spike-count entropy H_{marg} , the production utility $U_{\text{std}} = H_{\text{marg}} - \mathbb{E}[H_{\text{noise}}]$, and the distribution-aware research variant $U_{\text{DA}} = H_{\text{marg}} - \mathbb{E}_{x \sim p(x)}[H_{\text{cond}}]$; a deep layer of conditioning, subspace, and divergence results grounded in the kernel of this part.
- **Part IV — synthesis.** Guided diffusion: a denoising diffusion prior whose reverse process is steered by $\nabla_x U_{\text{std}}$ (which flows through $\nabla_x K$) to synthesize high-utility natural-image-like stimuli.

A single thread runs from the prior of this part to the proofs of Part III: the arc-cosine kernel is positively homogeneous in the input norm ($K(x, x) = x^\top C x + \sigma_0^2$, §3.2), which makes the utility’s variance grow with input scale — the object of the divergence proofs, the reason for a subspace constraint, and the reason diffusion guidance needs a naturalness prior.

2 The generative model

2.1 Latent Gaussian process, exponential link, Poisson counts

The observed spike count y_i (also written r_i , or n for a novel test image) of a neuron in response to image x_i is a Poisson draw whose rate is an exponential link applied to a latent Gaussian-process function λ .

Definition 2.1 (Spatial generative model). With a zero-mean GP prior over image space,

$$\lambda(\cdot) \sim \mathcal{GP}(0, K), \quad f(x) = \exp(A \lambda(x) + \lambda_0), \quad y_i \sim \text{Poisson}(f(x_i)). \quad (1)$$

Here $\lambda(x) \in \mathbb{R}$ is the *latent tuning function* (what the GP learns; *not* the firing rate); $f(x) > 0$ is the *firing rate* (the Poisson rate parameter); $A > 0$ is the *gain*, scaling how strongly the latent modulates the rate; and $\lambda_0 \in \mathbb{R}$ is the *bias* / log-baseline firing rate, so that $f = e^{\lambda_0}$ when $\lambda = 0$. The log-firing rate is $g(x) = A \lambda(x) + \lambda_0$, giving $f = e^g$.

The GP prior is defined by the property that any finite set of latent values $\{\lambda(x_1), \dots, \lambda(x_n)\}$ is jointly Gaussian, $\lambda(X) \sim \mathcal{N}(0, K)$ with $[K]_{ij} = K(x_i, x_j) = \text{Cov}[\lambda(x_i), \lambda(x_j)]$. The mean is zero (the eigenspace path carries no mean module; the GPyTorch path uses `ZeroMean`). The kernel K is fixed in *form* before seeing data but its entries depend on the images through evaluation; its full definition is §3.

Code. `PoissonLikelihood` (`likelihoods.py`) realizes the link and sampling in **forward** (returns `Poisson(rate = exp(Aλ + λ0))`), with $A = \exp(\text{raw_A})$ under a positivity constraint and λ_0 unconstrained; bound guards keep $A \in (0, 10]$ and $\lambda_0 \in [-50, 50]$.

2.2 Why the GP models the latent, not the rate

Modelling λ as Gaussian and pushing it through $f = \exp(A\lambda + \lambda_0)$ means the firing rate is *log-normally* distributed: we learn a scaled, shifted log of the Poisson rate rather than the rate itself. Two properties make this the right choice for spike-count data.

- *Positivity for free.* The exponential link guarantees $f > 0$ for any real latent value. A Gaussian prior placed *directly* on the rate would assign mass to negative (invalid) rates.

- *Smoothness where it belongs.* The GP prior imposes spatial structure (locality and smoothness, §3) on the unconstrained latent λ , not on the constrained rate. The kernel’s RF structure is a statement about the log-rate’s spatial regularity, which is where the biological prior (a spatially local, smooth receptive field) is naturally expressed.

Remark 2.2 (Poisson breaks conjugacy). For a Gaussian likelihood the posterior and predictive would be closed-form. With the Poisson likelihood the marginal $p(y \mid X, \theta) = \int \text{Poisson}(y \mid f(x)) \mathcal{N}(\lambda \mid 0, K_\theta) d\lambda$ has no closed form, so neither the exact posterior $p(\lambda \mid y, X, \theta)$ nor the exact predictive can be computed directly. This is what forces the variational treatment of Part II; here we fix only the model and its prior.

3 The arc-cosine kernel and the structured spatial prior

The prior covariance K is a first-order arc-cosine (ReLU) kernel built on a structured input-covariance matrix C . This section owns the definitions of both K and C , their parameterization and constraints, and the two gradients ($\nabla_x K$, $\partial C / \partial \theta$) that the utility, the diffusion guidance, and the M-step consume. Ground truth is `kernels.py` (`ArcCosineKernel`); where a summary disagrees with the code, the code wins and the discrepancy is flagged.

3.1 The order-1 arc-cosine kernel

Definition 3.1 (Order-1 arc-cosine kernel). For two stimulus vectors $x, x' \in \mathbb{R}^{n_{\text{px}}}$,

$$K(x, x') = \frac{1}{\pi} \mathcal{M} J(\theta), \quad \mathcal{M} = \sqrt{v_x v_{x'}}, \quad \cos \theta = \frac{x^\top C x' + \sigma_0^2}{\mathcal{M}}, \quad J(\theta) = \sin \theta + (\pi - \theta) \cos \theta, \quad (2)$$

with self-magnitudes (prior variances) $v_x = x^\top C x + \sigma_0^2$ and $v_{x'} = x'^\top C x' + \sigma_0^2$, cross-term $x^\top C x' + \sigma_0^2$, magnitude $\mathcal{M}$, angle θ , and order-1 angular term J . Equivalently, expanding the angle,

$$\theta = \arccos \left(\frac{x^\top C x' + \sigma_0^2}{\sqrt{x^\top C x + \sigma_0^2} \sqrt{x'^\top C x' + \sigma_0^2}} \right). \quad (3)$$

Here σ_0^2 is a constant *bias variance* and $C \in \mathbb{R}^{n_{\text{px}} \times n_{\text{px}}}$ is the structured spatial prior covariance (§3.3).

Proportionality constant. The LaTeX summaries write $K \propto \mathcal{M} J(\theta)$; the code fixes the constant explicitly at $1/\pi$ (`kernels.py`: `K = M * J / torch.pi`). Throughout this document the constant is $1/\pi$: every $\propto$ in a summary becomes $= (1/\pi) \cdot$ in code.

Remark 3.2 (Order and ReLU-network interpretation). This is the order- $n = 1$ member of the Cho–Saul (2009) arc-cosine family ($n = 0$ is Heaviside/step, $n = 1$ is ReLU); the angular term $J(\theta) = \sin \theta + (\pi - \theta) \cos \theta$ is exactly the ReLU case. The kernel is the covariance of the output of an infinite-width, single-hidden-layer neural network with ReLU hidden units whose input-to-hidden weights are drawn $w \sim \mathcal{N}(0, C)$, with the bias contributing σ_0^2 :

$$K(x, x') = \mathbb{E}_{w \sim \mathcal{N}(0, C)} [\text{ReLU}(w^\top x) \text{ReLU}(w^\top x')] \quad (\text{up to the } 1/\pi \text{ constant, with bias } \sigma_0^2). \quad (4)$$

Thus C is literally the prior covariance of the network weights across pixel locations. Shaping C (§3.3) injects the prior belief that an RGC receptive field is spatially *local* and *smooth*.

3.2 Non-stationarity and the self-kernel identity

Proposition 3.3 (Self-kernel identity). *The prior variance at any image is the C -quadratic form plus the bias variance:*

$$K(x, x) = v_x = x^\top C x + \sigma_0^2. \quad (5)$$

Proof. At $x' = x$ the cross-term equals $v_x = \mathcal{M}$, so $\cos \theta = 1$, $\theta = 0$, and $J(0) = \sin 0 + (\pi - 0) \cos 0 = \pi$. Hence $K(x, x) = \mathcal{M} J(0) / \pi = \mathcal{M} = \sqrt{v_x v_x} = v_x$. The code’s `diag=True` branch returns $v_x = x^\top C x + \sigma_0^2$ directly (`kernels.py:614-618`). $\square$

Remark 3.4 (Non-stationarity and its consequence for active learning). Equation (5) shows K depends on the actual input values through $x^\top C x$, not only on $x - x'$: the kernel is *non-stationary* and the prior variance v_x grows with the input norm (in the C -metric). This has a direct downstream effect: the active-learning utility is driven by posterior variance, so utility maximization is biased toward high-norm images unless the kernel is normalized. For exactly this reason the code provides normalized and saturating siblings of the base kernel — `ArcCosineKernelNormalized` ($\bar{K} = J(\theta) / \pi$, constant $\bar{K}(x, x) = 1$) and `ArcSineKernel` (bounded diagonal) — which are analyzed in Part III as fixes to the norm-scaling divergence. This part keeps the focus on the base `ArcCosineKernel`.

3.3 The structured spatial prior covariance C

C encodes that RGC receptive fields are spatially local and smooth. It is *constant with respect to the stimulus x* — it depends only on the hyperparameters ($\text{Amp}, \beta, \rho, \xi_0$). Pixel i sits at grid coordinate ξ_i on a normalized $[-1, 1] \times [-1, 1]$ grid, and $\xi_0 = (\text{eps_0x}, \text{eps_0y})$ is the RF centre.

Definition 3.5 (Structured spatial prior). With an amplitude Amp , a per-pixel locality weight α_i , and a pairwise smoothing factor C_{smooth} ,

$$C_{ij} = \text{Amp} \alpha_i [C_{\text{smooth}}]_{ij} \alpha_j = \text{Amp} \exp\left(-\frac{\|\xi_i - \xi_0\|^2}{4\beta^2}\right) \exp\left(-\frac{\|\xi_i - \xi_j\|^2}{2\rho^2}\right) \exp\left(-\frac{\|\xi_j - \xi_0\|^2}{4\beta^2}\right), \quad (6)$$

where

$$\begin{aligned} \alpha_i &= \exp\left(-\frac{\|\xi_i - \xi_0\|^2}{4\beta^2}\right) \quad (\text{locality weight}), \\ [C_{\text{smooth}}]_{ij} &= \exp\left(-\frac{\|\xi_i - \xi_j\|^2}{2\rho^2}\right) \quad (\text{smoothness kernel}), \end{aligned} \quad (7)$$

followed by a symmetrization $C \leftarrow (C + C^\top) / 2$.

Physical meaning of the natural parameters (defaults $\text{Amp} = 1.0$, $\beta = \rho = 0.1$ on the $[-1, 1]$ grid):

- **Amp** — overall *amplitude*: a positive gain (default 1.0) on the structured covariance C , scaling the prior variance of the latent. It multiplies the C -quadratic part of the self-magnitude $v_x = x^\top C x + \sigma_0^2$ (not the bias σ_0^2); because it enters the kernel inside $\mathcal{M} = \sqrt{v_x v_{x'}}$ and $\cos \theta$, it rescales K non-linearly rather than as a plain output scale. Unlike β, ρ, ξ_0 it sets the magnitude, not the spatial shape, of the RF weighting.
- **β** — RF *half-width*: the locality weight α is a Gaussian of standard deviation $\sqrt{2}\beta$ in pixel-distance, reaching e^{-1} at $\text{dist} = 2\beta$; for $\beta = 0.1$ the RF spans roughly ± 0.2 ($\approx 20\%$ of the axis).

- ρ — smoothness *standard deviation* (correlation length) of the RBF-like C_{smooth} (variance ρ^2): pixels closer than ρ are strongly correlated, pixels beyond 3ρ nearly uncorrelated.
- ξ_0 — the 2D RF centre on $[-1, 1]^2$.

Together, α localizes weight around ξ_0 and C_{smooth} correlates nearby pixels — an RF-shaped weight prior.

Code. C is built in `ArcCosineKernel._compute_C_matrix` (`kernels.py:486-559`): the outer product $C = \text{Amp} * \alpha[:, \text{None}] * C_{\text{smooth}} * \alpha[\text{None}, :]$ (line 554) followed by $C = (C + C.T)/2$ (line 557).

3.4 Raw parameterization and constraints

For optimizer stability the code stores *unconstrained log-space raw parameters*, not β and ρ directly (registered in `kernels.py:229-242`, all `float64`):

$$\begin{aligned} \text{raw_m2log2beta} &= -2\log(2\beta) \quad \Rightarrow \quad \exp(\text{raw_m2log2beta}) = \frac{1}{4\beta^2}, \\ \text{raw_mlog2rho2} &= -\log(2\rho^2) \quad \Rightarrow \quad \exp(\text{raw_mlog2rho2}) = \frac{1}{2\rho^2}, \end{aligned} \tag{8}$$

and $\sigma_0 = \exp(\text{raw_sigma_0})$ under a `Positive(exp / log)` constraint (so the bias variance added in (2) is $\sigma_0^2 = (\exp(\text{raw_sigma_0}))^2$). Inside `_compute_C_matrix` the exponentials of the raw parameters are used *directly* as the exponent coefficients, so that $\alpha_i = \exp(-\frac{1}{4\beta^2} \|\xi_i - \xi_0\|^2)$ and $[C_{\text{smooth}}]_{ij} = \exp(-\frac{1}{2\rho^2} \|\xi_i - \xi_j\|^2)$ reproduce exactly the natural-parameter form (6) (verified numerically: $\beta = \rho = 0.1 \Rightarrow 1/(4\beta^2) = 25, 1/(2\rho^2) = 50$). The `.beta/.rho` properties invert the raw parameters back to the natural values, $\beta = \frac{1}{2} \exp(-\text{raw_m2log2beta}/2)$, $\rho = \frac{1}{\sqrt{2}} \exp(-\text{raw_mlog2rho2}/2)$, and round-trip cleanly.

Param	Constraint mechanism	Bounds (natural)	Bounds (raw)
σ_0	$\sigma_0 = \exp(\text{raw_sigma_0})$, <code>Positive(exp / log)</code>	$\sigma_0 > 0$	—
β	via <code>raw_m2log2beta</code> , clamped	[0.01, 0.3]	[1.02, 7.82]
ρ	via <code>raw_mlog2rho2</code> , clamped	[0.01, 0.5]	[0.69, 8.52]
ξ_0	direct clamp	$[-1, 1]$ per axis	—
Amp	<code>Amp = softplus(raw_Amp)</code> , clamp	(0, 1000]	—

Bounds are enforced two ways: `params_in_bounds()` (read-only rejection of an out-of-bounds trial step inside the LBFGS closure) and `clamp_hyperparameters()` (projected clamp after `optimizer.step()`).

Caveat. The β upper bound is **0.3**, not 1.0. It was tightened $1.0 \rightarrow 0.3$ on 2026-04-10 after a β -drift $0.12 \rightarrow 0.45$ made C cover all 11664 pixels (~ 519 MB) and OOM. Two stale code comments remain and should not be trusted: the inline comment on `kernels.py:180` reads `RAW_BETA_MIN #  $\approx 0.18$` , but $-2\log(2 \cdot 0.3) = 1.02$ (the block comment on line 172 and the arithmetic both give 1.02); and the `clamp_hyperparameters` docstring still says “`beta` $\in [0.01, 1.0]$ ” — also stale, the enforced bound is [0.01, 0.3].

Remark 3.6 (Naming hazard: two different “`beta`”/“`rho`”). “`beta`”/“`rho`” are overloaded. Use the *natural* β, ρ (RF half-width, smoothness SD — the `.beta/.rho` properties) for all reasoning and plotting. The local variables `beta` = $1/(4\beta^2)$ and `rho2` = $1/(2\rho^2)$ inside `_compute_C_matrix` are reciprocal-square coefficients, *not* the natural parameters; do not confuse them.

Remark 3.7 (Pixel masking). With `MASK_THRESHOLD`= 0.001, pixels whose locality weight satisfies $\alpha_i \geq 0.001$ are kept and the rest dropped, shrinking C from $(n_{\text{px}}, n_{\text{px}})$ to $(n_{\text{masked}}, n_{\text{masked}})$ (typically $\sim 100\times$ smaller). The mask is computed with *detached* parameters, so its *structure* is non-differentiable and stable while the retained C entries still carry gradients; `forward()` applies the same mask to the inputs, so only active pixels contribute to $x^\top Cx$. The mask radius is $\|\xi_i - \xi_0\| \leq 2\beta\sqrt{\ln 1000} \approx 5.25\beta$.

3.5 Kernel gradient with respect to the input, $\nabla_x K$

Choosing the next image (and steering diffusion, Part IV) requires the derivative of the kernel with respect to the stimulus. Treating C as symmetric and constant, σ_0^2 constant, and x' (hence $v_{x'}$) constant, the useful intermediate gradients are

$$\nabla_x v_x = 2Cx, \quad \nabla_x (x^\top Cx' + \sigma_0^2) = Cx', \quad \nabla_x \mathcal{M} = \sqrt{\frac{v_{x'}}{v_x}} Cx, \quad (9)$$

and the angular derivative $dJ/d\theta = -(\pi - \theta) \sin \theta$.

Proposition 3.8 (Input gradient of the arc-cosine kernel).

$$\nabla_x K(x, x') = \frac{1}{\pi} \left[(\pi - \theta) Cx' + \sin \theta \sqrt{\frac{v_{x'}}{v_x}} Cx \right] = \frac{1}{\pi} \left[(\pi - \theta) Cx' + \sin \theta \sqrt{\frac{x'^\top Cx' + \sigma_0^2}{x^\top Cx + \sigma_0^2}} Cx \right]. \quad (10)$$

By symmetry ($x \leftrightarrow x'$),

$$\nabla_{x'} K(x, x') = \frac{1}{\pi} \left[(\pi - \theta) Cx + \sin \theta \sqrt{\frac{x^\top Cx + \sigma_0^2}{x'^\top Cx' + \sigma_0^2}} Cx' \right]. \quad (11)$$

Proof. Write $K = \frac{1}{\pi} \mathcal{M} J(\theta)$ and apply the product rule $\nabla_x K \propto (\nabla_x \mathcal{M})J + \mathcal{M}(\nabla_x J)$. Using $\nabla_x J = (dJ/d\theta)\nabla_x \theta$ with $\nabla_x \theta = -(1/\sin \theta)\nabla_x(\cos \theta)$ and $dJ/d\theta = -(\pi - \theta) \sin \theta$, the $\sin \theta$ cancels and $\nabla_x J = (\pi - \theta)\nabla_x(\cos \theta)$ with $\nabla_x(\cos \theta) = \frac{1}{\mathcal{M}} Cx' - \frac{\cos \theta}{\mathcal{M}} \sqrt{v_{x'}/v_x} Cx$ (recall $\cos \theta = (x^\top Cx' + \sigma_0^2)/\mathcal{M}$). Collecting terms,

$$\pi \nabla_x K = \underbrace{\sin \theta \sqrt{\frac{v_{x'}}{v_x}} Cx}_{[A]} + \underbrace{(\pi - \theta) \cos \theta \sqrt{\frac{v_{x'}}{v_x}} Cx}_{[B]} + \underbrace{(\pi - \theta) Cx'}_{[C]} - \underbrace{(\pi - \theta) \cos \theta \sqrt{\frac{v_{x'}}{v_x}} Cx}_{[D]}.$$

Terms $[B]$ and $[D]$ are identical with opposite sign and cancel, leaving $[A] + [C]$, which is (10). $\square$

Every term carries a factor C , so $\nabla_x K$ lives in the column space of C ; this is the algebraic root of the subspace analysis in Part III (gradient ascent on the utility already concentrates on the top C -eigenvectors). *Code.* With `gradient_mode='autograd'` (default) this gradient is obtained by autodiff through `forward()`; the optional `'vjp'` mode (`analytical_gradients_vjp.py`) evaluates (10) explicitly, and all modes return the same K .

3.6 Kernel gradients with respect to the hyperparameters, $\partial C/\partial \theta$

The M-step (Part II training) maximizes the ELBO over the kernel hyperparameters and needs $\partial C/\partial \theta$, which then chains through $v_x = x^\top Cx$ and the cross-term into $\partial K/\partial \theta$ using the same quadratic-form structure as §3.5 with C replaced by $\partial C/\partial \theta$.

Proposition 3.9 (Structured-prior hyperparameter gradients (natural parameters)). *Writing $C = \text{Amp} \alpha C_{\text{smooth}} \alpha^\top$, the amplitude enters linearly and the shape parameters through their Gaussian exponents. Each shape-parameter derivative is C_{ij} — which already carries the Amp factor — times the derivative of its log-argument, while the amplitude derivative strips one factor of Amp:*

$$\begin{aligned}\frac{\partial C_{ij}}{\partial \text{Amp}} &= \alpha_i [C_{\text{smooth}}]_{ij} \alpha_j = \frac{C_{ij}}{\text{Amp}}, \\ \frac{\partial C_{ij}}{\partial \beta} &= C_{ij} \frac{\|\xi_i - \xi_0\|^2 + \|\xi_j - \xi_0\|^2}{2\beta^3}, \\ \frac{\partial C_{ij}}{\partial \rho} &= C_{ij} \frac{\|\xi_i - \xi_j\|^2}{\rho^3}, \\ \nabla_{\xi_0} C_{ij} &= \frac{C_{ij}}{2\beta^2} [(\xi_i - \xi_0) + (\xi_j - \xi_0)] \quad (2D \text{ vector}).\end{aligned}\tag{12}$$

Proof. For Amp: $C = \text{Amp} (\alpha C_{\text{smooth}} \alpha^\top)$ is linear in Amp, so $\partial C_{ij} / \partial \text{Amp} = \alpha_i [C_{\text{smooth}}]_{ij} \alpha_j = C_{ij} / \text{Amp}$. For β : $\partial_\beta [- (\|\xi_i - \xi_0\|^2 + \|\xi_j - \xi_0\|^2) / (4\beta^2)] = + (\|\xi_i - \xi_0\|^2 + \|\xi_j - \xi_0\|^2) / (2\beta^3)$. For ρ : $\partial_\rho [- \|\xi_i - \xi_j\|^2 / (2\rho^2)] = + \|\xi_i - \xi_j\|^2 / \rho^3$. For ξ_0 : $\nabla_{\xi_0} \|\xi - \xi_0\|^2 = 2(\xi_0 - \xi)$, giving $\nabla_{\xi_0} \log C_{ij} = (1/(2\beta^2))[(\xi_i - \xi_0) + (\xi_j - \xi_0)]$. $\square$

Remark 3.10 (Code differentiates the raw parameters). Because the code optimizes the raw parameters, `eigenspace_gradients.py` (lines 104–122) stores the chain-ruled *raw*-parameter gradients, which the M-step consumes directly (`eigenspace_mstep.py:320–330`):

$$\begin{aligned}\frac{\partial C_{ij}}{\partial (\text{raw_m2log2beta})} &= C_{ij} (\log \alpha_i + \log \alpha_j) = -C_{ij} \frac{\|\xi_i - \xi_0\|^2 + \|\xi_j - \xi_0\|^2}{4\beta^2}, \\ \frac{\partial C_{ij}}{\partial (\text{raw_mlog2rho2})} &= C_{ij} \log [C_{\text{smooth}}]_{ij} = -C_{ij} \frac{\|\xi_i - \xi_j\|^2}{2\rho^2}.\end{aligned}\tag{13}$$

and, since ξ_0 is a direct parameter, $\partial C_{ij} / \partial \xi_0 = (C_{ij} / (2\beta^2))[(\xi_i - \xi_0) + (\xi_j - \xi_0)]$ coincides with the natural form in (12). The two are related by the constraint-transform Jacobian $d(\text{raw})/d\beta = -2/\beta$, etc.; with the default `gradient_mode='autograd'` this chain rule is applied automatically.

3.7 Numerical stability

Three stability mechanisms act at three distinct levels; they must not be conflated.

1. **C matrix — symmetrization only.** $C \leftarrow (C + C^\top) / 2$ (`kernels.py:557`) cancels the float-rounding asymmetry from the outer product. *No diagonal jitter is added to C .*
2. **Kernel forward() — domain clamps.** With $\text{eps} = 10^{-7}$, $\cos \theta$ is clamped into $(-1, 1)$ so $\arccos$ is finite, and $1 - \cos^2 \theta$ is clamped at $\text{min} = \text{eps}$ so $\sin \theta = \sqrt{1 - \cos^2 \theta}$ never takes the square root of a negative rounding result (`kernels.py:634–639`). These keep the transcendental operations in-domain and are unrelated to Cholesky jitter.
3. **GP covariance / Cholesky — jitter on the gram matrices, not on C .** Jitter is added to the *output* gram matrices of the arc-cosine kernel — the inducing gram $\tilde{K}$ (i.e. $K(\tilde{Z}, \tilde{Z})$) and the predictive gram $K(X, X)$ — at the GPyTorch variational-strategy level, *not* to C . A pre-Cholesky jitter (`model.jitter`, default 10^{-4}) is added to $\tilde{K}/K(X, X)$; the kernel’s own `forward()` adds none (that would double-jitter). On a failed factorization `psd_safe_cholesky` promotes $\tilde{K}$ to `float64` (sequential Cholesky subtractions cause catastrophic cancellation in `float32`) and retries with escalating jitter $10^{-4}, 10^{-3}, 10^{-2}$ (`cholesky_max_tries`, default 3).

Caveat. Some intuition expects diagonal jitter on the prior C ; there is none. C receives only symmetrization. Jitter lives exclusively on the GP gram matrices $\widetilde{K}/K(X, X)$. Do not “add jitter to C ” thinking it is missing.

Part II

Inference: variational learning of the tuning

4 Sparse variational Gaussian-process inference

Part I fixed the generative model: a zero-mean latent Gaussian process $\lambda \sim \mathcal{GP}(0, K)$ over image space, an exponential link to the firing rate $f(x) = \exp(A\lambda(x) + \lambda_0)$, and Poisson spike counts $y_i \sim \text{Poisson}(f(x_i))$, where K is the first-order arc-cosine kernel of (2) built on the structured spatial prior covariance C of (6). Here we turn that model into a trainable posterior. Two current implementations realise *the same* inference; both are introduced abstractly in this section and made concrete in §5.

4.1 Poisson breaks conjugacy

Given a batch of images $X = (x_1, \dots, x_N)$ and spike counts $y = (y_1, \dots, y_N)$, the exact posterior over the latent and the marginal likelihood,

$$\begin{aligned} p(\lambda \mid y, X, \theta) &\propto \left[\prod_i \text{Poisson}(y_i \mid e^{A\lambda_i + \lambda_0}) \right] \mathcal{N}(\lambda \mid 0, K_\theta), \\ p(y \mid X, \theta) &= \int \prod_i \text{Poisson}(y_i \mid e^{A\lambda_i + \lambda_0}) \mathcal{N}(\lambda \mid 0, K_\theta) d\lambda, \end{aligned} \quad (14)$$

have no closed form: the exponential-Poisson likelihood is not conjugate to the Gaussian prior, so the integral over λ is intractable. We therefore approximate the posterior by a tractable Gaussian and maximise a variational lower bound (§4.4).

Remark 4.1 (The conjugate contrast). For a *Gaussian* observation $y = \lambda + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma_r^2 I)$, everything would be closed-form: the marginal is $y \mid X \sim \mathcal{N}(0, K + \sigma_r^2 I)$, the predictive at a test image x_* is Gaussian with mean $k_*^\top (K + \sigma_r^2 I)^{-1} y$ and variance $K(x_*, x_*) - k_*^\top (K + \sigma_r^2 I)^{-1} k_*$, and the log-evidence is $-\frac{1}{2} y^\top (K + \sigma_r^2 I)^{-1} y - \frac{1}{2} \log |K + \sigma_r^2 I| - \frac{N}{2} \log 2\pi$. None of these survive the Poisson link; the variational route below recovers a usable posterior in its place.

4.2 Inducing points and the variational posterior

Definition 4.2 (Sparse variational posterior). Introduce M *inducing points* $\tilde{Z} = \{\tilde{z}_1, \dots, \tilde{z}_M\}$ with latent values $\tilde{\lambda} = \lambda(\tilde{Z})$ (the code’s inducing values u), and place a Gaussian variational posterior on them,

$$q(\tilde{\lambda}) = \mathcal{N}(\tilde{\lambda} \mid m, V), \quad m \in \mathbb{R}^M, \quad V \in \mathbb{S}_{++}^M. \quad (15)$$

The posterior at *any* image x is recovered by marginalising the conditional prior $p(\lambda(x) \mid \tilde{\lambda})$ against $q(\tilde{\lambda})$.

Exact inference would need the $N \times N$ kernel over all training images, an $O(N^3)$ cost; the inducing construction reduces it to $O(NM^2)$.

Remark 4.3 (Closed-loop invariant $M = N$). In the active-learning loop the inducing set is taken equal to the training set, $X = \tilde{Z}$ (code `X_train == X_tilde`), so $M = N$ and $K = \tilde{K}$. The “sparse” posterior is then *exact*. The SVGP formalism is retained not for sparsity but for two other reasons: (i) it supplies the closed-form variational machinery of §4.3–§4.5, and (ii) its inducing structure admits the $O(M)$ rank-one online growth (§5.9) that appends one image per active step.

4.3 Projected posterior moments

Marginalising $p(\lambda(x) \mid \tilde{\lambda}) q(\tilde{\lambda})$ yields a Gaussian $q(\lambda(x)) = \mathcal{N}(\mu(x), \sigma^2(x))$ whose moments are the inducing-point moments projected through the kernel:

$$\begin{aligned}\mu(x) &= k(x)^\top \tilde{K}^{-1} m, \\ \sigma^2(x) &= K(x, x) + k(x)^\top \tilde{K}^{-1} (V - \tilde{K}) \tilde{K}^{-1} k(x),\end{aligned}\tag{16}$$

with the cross-covariance vector $k(x) = K(\tilde{Z}, x) \in \mathbb{R}^M$, the inducing gram $\tilde{K} = K(\tilde{Z}, \tilde{Z}) \in \mathbb{R}^{M \times M}$, and the prior self-variance $K(x, x)$ given by the self-kernel identity (5). Introducing the *projection vector* $u(x) = \tilde{K}^{-1} k(x)$ and the Schur complement $s(x) = K(x, x) - k(x)^\top \tilde{K}^{-1} k(x)$, the variance splits into a data-independent residual and an epistemic term,

$$\sigma^2(x) = \underbrace{[K(x, x) - k(x)^\top \tilde{K}^{-1} k(x)]}_{\text{residual (Nyström/Schur) } s(x)} + \underbrace{k(x)^\top \tilde{K}^{-1} V \tilde{K}^{-1} k(x)}_{\text{epistemic}} = s(x) + u(x)^\top V u(x),\tag{17}$$

and the mean reads $\mu(x) = u(x)^\top m$. The two forms of σ^2 are algebraically identical since $k^\top \tilde{K}^{-1} (V - \tilde{K}) \tilde{K}^{-1} k = k^\top \tilde{K}^{-1} V \tilde{K}^{-1} k - k^\top \tilde{K}^{-1} k$.

Remark 4.4 (Prior sanity check). At the prior $V = \tilde{K}$ the correction in (16) vanishes and $\sigma^2(x) = K(x, x)$ (the prior variance); as data drive V below $\tilde{K}$, $\sigma^2(x) < K(x, x)$.

Both code paths compute exactly (16): `vargp_direct` in the eigenbasis of $\tilde{K}$ (`eigenspace_model.py: lambda_moments_eigenspace`, §5.3), `default_gpy` through GPyTorch’s `VariationalStrategy` internals.

4.4 The ELBO

All parameters are learned by maximising the evidence lower bound on $\log p(y)$,

$$\mathcal{L} = \sum_i \mathbb{E}_{q(\lambda_i)} [\log \text{Poisson}(y_i \mid \lambda_i)] - \text{KL}(q(\tilde{\lambda}) \parallel p(\tilde{\lambda})),\tag{18}$$

which is tight when q equals the true posterior. Because the likelihood factorises over images and $q(\lambda_i) = \mathcal{N}(\mu_i, \sigma_i^2)$ is Gaussian, each expected log-likelihood term is closed-form:

$$\mathbb{E}_{q(\lambda_i)} [\log \text{Poisson}(y_i \mid \lambda_i)] = y_i (A\mu_i + \lambda_0) - \exp(A\mu_i + \frac{1}{2}A^2\sigma_i^2 + \lambda_0) + \text{const},\tag{19}$$

with $\text{const} = -\log(y_i!)$ independent of all parameters (dropped in code). `likelihoods.py: expected_log_prob` returns, summed over images,

```
log_prob = target*(A*mu + lambda0) \
           - exp(A*mu + 0.5*A**2*var + lambda0)
```

matching (19) term by term. The KL term is the standard Gaussian–Gaussian divergence between $q(\tilde{\lambda}) = \mathcal{N}(m, V)$ and the prior $p(\tilde{\lambda}) = \mathcal{N}(0, \tilde{K})$,

$$\text{KL}(q \parallel p) = \frac{1}{2} \left[\log \frac{|\tilde{K}|}{|V|} + \text{Tr}(\tilde{K}^{-1} V) + m^\top \tilde{K}^{-1} m - M \right].\tag{20}$$

Remark 4.5 (The $-M$ constant). The $-M$ in (20) is parameter-independent (it is the dimension of the inducing Gaussian) and carries zero gradient, so it affects only the reported loss magnitude, never the optimum or the predictions. The eigenspace evaluation of §5.4 carries the analogous $-n_b$ for the same reason.

The full objective is maximised by EM-style coordinate ascent (E-step on (m, V) , F-step on (A, λ_0) , M-step on the kernel hyperparameters); the update equations are derived in the training-algorithm section. Equation (18) is the single objective all three blocks share.

4.5 Expected firing rate via the Gaussian MGF

The exponential term in (19) is the *expected firing rate* under q ,

$$\bar{f}_i = \mathbb{E}_{q(\lambda_i)}[\exp(A\lambda_i + \lambda_0)] = \exp(A\mu_i + \tfrac{1}{2}A^2\sigma_i^2 + \lambda_0). \quad (21)$$

This is the Gaussian moment-generating function $\mathbb{E}[e^{tZ}] = \exp(t\mu + \frac{1}{2}t^2\sigma^2)$ applied to $Z = A\lambda_i + \lambda_0$ at $t = 1$ (code `likelihoods.py:expected_firing_rate`, identical to the loop’s `compute_f_mean`).

Remark 4.6 (Jensen inflation). The term $\frac{1}{2}A^2\sigma_i^2$ is strictly positive whenever $\sigma_i^2 > 0$, so posterior *uncertainty raises* the predicted rate above the point estimate $\exp(A\mu_i + \lambda_0)$ — a direct consequence of Jensen’s inequality for the convex map $\exp$. Confidence and rate are thus coupled: a poorly-constrained image is predicted to fire *more*, not less.

4.6 Prediction: expected spike count

The moments (16) do not depend on the observed y , so the same formulas serve as the predictive distribution at any novel image x_* . The expected spike count is the expected firing rate (21) evaluated there,

$$\langle y_* \rangle = \langle f(x_*) \rangle = \exp(A\mu(x_*) + \tfrac{1}{2}A^2\sigma^2(x_*) + \lambda_0), \quad (22)$$

with $\mu(x_*), \sigma^2(x_*)$ from (16) using freshly evaluated $k(x_*)$ and $K(x_*, x_*)$. The firing rate is log-normal, so its variance is

$$\text{Var}(f(x)) = \langle f(x) \rangle^2 (e^{A^2\sigma^2(x)} - 1). \quad (23)$$

4.7 The posterior cross-covariance pitfall

The self-variance of (16) extends formally to the *cross-covariance* between two distinct images x (a candidate sample location) and x_* (a query location):

$$\Sigma_{x,x_*} = \text{Cov}[\lambda(x), \lambda(x_*) \mid \mathcal{D}] = K(x, x_*) + u(x)^\top (V - \widetilde{K}) u(x_*), \quad u(\cdot) = \widetilde{K}^{-1}k(\cdot), \quad (24)$$

which reduces to $\sigma^2(x)$ at $x = x_*$.

Remark 4.7 (Failure outside the inducing hull). For a query x_* *outside* the inducing region, (24) produces cross-covariances that violate the Cauchy–Schwarz bound $|\Sigma_{x,x_*}| \leq \sqrt{\Sigma_{x,x} \Sigma_{x_*,x_*}}$. Measured example (RBF, $\ell = 0.2$, 20 inducing points in $[-1, 1]$, sample $x = 0$): at $x_* = \pm 1.5$ the manual formula gives $\Sigma \approx -4$ while GPyTorch’s covariance gives ≈ 0 ; with self-variances ~ 0.04 the valid maximum is ~ 0.04 , so -4 is two orders of magnitude out of range. Root cause: in the residual $K(x, x_*) - u(x)^\top \widetilde{K} u(x_*)$ the direct kernel $K(x, x_*) \rightarrow 0$ (finite lengthscale) outside the hull, but $u(x_*) = \widetilde{K}^{-1}k(x_*)$ can have large entries through $\widetilde{K}^{-1}$, and the two pieces fail to cancel.

Left unchecked, a value like $\Sigma_{x,x_*} \approx -4$ feeds the conditional moments $\mu_{\text{cond}}(x_*) = \mu(x_*) + (\Sigma_{x,x_*}/\Sigma_{x,x})(\lambda_{\text{obs}} - \mu(x))$ and $\sigma_{\text{cond}}^2(x_*) = \Sigma_{x_*,x_*} - \Sigma_{x,x_*}^2/\Sigma_{x,x}$ of the distribution-aware utility (Part III) with a regression coefficient ≈ -100 , giving nonsensical conditional means and spurious utility peaks at the edges of the inducing region. The fix is to never assemble the cross-block by hand: form the joint covariance with GPyTorch and slice it,

```
full_covar = model(torch.cat([x_samples, x_star])).covariance_matrix
Sigma      = full_covar[:N, N:]          # correct cross-block, Nystrom-consistent
```

Caveat. The shipped predict path already reads GPyTorch’s joint `covariance_matrix` (which internally includes the conditional correction term), so the pathology of (24) is confined to *hand-assembled* cross-covariances, not the production path. Any new code that forms a cross-covariance from the manual formula outside the inducing hull will nonetheless reintroduce it — use the slice above.

4.8 Whitening

The alternative reparameterisation (used by `default_gpy`, §5.6) is *Cholesky whitening*, which makes the prior a standard normal. With the Cholesky factor $\tilde{K} = L_{\tilde{K}} L_{\tilde{K}}^T$, define the whitened inducing values

$$\tilde{\lambda} = L_{\tilde{K}}^{-1}(\tilde{\lambda} - \mu_z) \sim \mathcal{N}(0, I) \quad \text{under the prior,} \quad (25)$$

where μ_z is the prior mean at the inducing points. Optimisation conditions much better in these coordinates. GPyTorch then stores the *whitened* variational parameters $(\tilde{m}, \tilde{V})$, related to the actual ones by

$$m - \mu_z = L_{\tilde{K}} \tilde{m}, \quad V = L_{\tilde{K}} \tilde{V} L_{\tilde{K}}^T; \quad \tilde{m} = L_{\tilde{K}}^{-1}(m - \mu_z), \quad \tilde{V} = L_{\tilde{K}}^{-1} V L_{\tilde{K}}^{-T}. \quad (26)$$

The whitened mean thus encodes the *deviation* from the prior mean function, not absolute function values. Reading these stored params inside a manual moment formula therefore requires three corrections: unwhiten the covariance ($V = L_{\tilde{K}} \tilde{V} L_{\tilde{K}}^T$), unwhiten the mean ($m - \mu_z = L_{\tilde{K}} \tilde{m}$), and add the mean function back, $\mu_* = \mu(x_*) + u(x_*)^T L_{\tilde{K}} \tilde{m}$ and $\sigma_*^2 = s(x_*) + u(x_*)^T L_{\tilde{K}} \tilde{V} L_{\tilde{K}}^T u(x_*)$.

Remark 4.8 (Zero mean in production). Both production paths use `ZeroMean` (`gpy_model.py`; the eigenspace path carries no mean module), so $\mu_z = 0$ and the “add the mean function” step is trivially zero. The whitening maps (26) themselves still apply whenever the whitened `VariationalStrategy` is selected; the mean-function machinery is retained only to guard a future non-zero mean.

5 The eigenspace representation and the two implementations

The two current code paths realise the *same* variational GP of §4 — same prior $\mathcal{N}(0, \tilde{K})$, same arc-cosine RF kernel, same Poisson(A, λ_0) likelihood, same expected rate (21), same ELBO (18). They differ only in *how* $q(\tilde{\lambda})$ is represented and optimised:

- `vargp_direct` (`DirectVGPMModel`) projects q into the eigenbasis of $\tilde{K}$ and runs a custom EM loop, using GPyTorch as a kernel calculator only;
- `default_gpy` (`VariationalGPMModel`) keeps q in the full M -dimensional inducing space via a stock GPyTorch `CholeskyVariationalDistribution` and optimises one joint ELBO.

5.1 Eigendecomposition of the inducing gram

The inducing gram $\widetilde{K}$ ($M \times M$) has effective rank far below M (typically ~ 10 – 50). `vargp_direct` eigendecomposes it and keeps only the well-conditioned directions (`eigenspace_utils.py: eigendecompose_K_tilde`):

$$\begin{aligned}\widetilde{K} &= B \Lambda B^\top, & \Lambda &= \text{diag}(\text{eigvals}_b), \\ B &\in \mathbb{R}^{M \times n_b}, & B^\top B &= I_{n_b},\end{aligned}\tag{27}$$

where the kept set is chosen by a *relative* threshold with an absolute floor,

$$\text{keep eigenpair } j \iff \Lambda_j > \max(\Lambda_{\max} \cdot \text{tol}, \text{tol}), \quad \text{tol} = \text{EIGVAL_TOL} = 10^{-4}.\tag{28}$$

`torch.linalg.eigh` returns eigenvalues in ascending order; B collects the kept eigenvectors as columns and Λ the kept eigenvalues. The kept count n_b is *dynamic*: it shifts across training iterations as the kernel hyperparameters move, and grows by ~ 1 when a point is appended (§5.9). Typical $n_b \approx 10$ – 11 for the closed-loop cells, up to ~ 50 elsewhere. Reprojection (§5.5) absorbs the dimension change.

Three payoffs motivate the projection: (i) dimensionality drops $M \rightarrow n_b$, so per-step cost falls $O(M^3) \rightarrow O(n_b^3)$; (ii) $\widetilde{K}$ becomes *diagonal* in the new basis, $\widetilde{K}_b = \Lambda$, so $\widetilde{K}^{-1}$ is the element-wise reciprocal $\text{diag}(1/\Lambda)$ — no matrix solve; (iii) dropping the small eigenvalues gives implicit regularisation.

5.2 The eigenspace state

The `DirectVariationalState` dataclass (`eigenspace_model.py`) is the eigenspace posterior together with all precomputed kernel projections:

field	shape	meaning
$\mathbf{m}_b = m_b$	(n_b)	variational mean in eigenspace, $m_b = B^\top m$
$\mathbf{V}_b = V_b$	(n_b, n_b)	variational covariance in eigenspace, $V_b = B^\top V B$ — dense
$B = B$	(M, n_b)	eigenvector matrix of $\widetilde{K}$ (the basis)
$\text{eigvals}_b = \Lambda$	(n_b)	kept eigenvalues of $\widetilde{K}$ (ascending)
$\mathbf{K_tilde}_b = \widetilde{K}_b$	(n_b, n_b)	inducing gram in eigenspace = $\text{diag}(\Lambda)$ — diagonal
$\mathbf{K}_b$	(N, n_b)	cross-kernel in eigenspace = $K(X, \widetilde{Z}) B$
$\mathbf{KKtilde_inv}_b = a$	(N, n_b)	projection $K \widetilde{K}^{-1}$ in eigenspace = $\mathbf{K}_b / \Lambda$ (element-wise)
$\mathbf{Kvec} = \mathbf{Kvec}$	(N)	diagonal self-kernel $K(x_i, x_i)$, the per-point prior variance
$\mathbf{K_tilde} = \widetilde{K}$	(M, M)	the full inducing gram, kept only to enable the $O(M)$ column-append (§5.9)
<code>mask</code>	(n_{pix})	RF-support pixel mask from the kernel (if <code>use_mask</code>)

Remark 5.1 (Only $\widetilde{K}_b$ is diagonal). V_b is a *dense* $n_b \times n_b$ matrix even though $\widetilde{K}_b$ is diagonal; never assume a diagonal V_b . The projection row $a_i = (\mathbf{K}_b / \Lambda)_i$ is exactly the eigenspace image of $k(x_i)^\top \widetilde{K}^{-1}$, computed by element-wise division rather than a solve.

The state is built in one pass by `_compute_eigenspace_quantities` (under `torch.no_grad`), which is shared by initialisation and the post- M -step recompute. Initialisation sets the variational params to the prior:

$$m_b = 0, \quad V_b = \widetilde{K}_b \quad (\text{i.e. } m = 0, V = \widetilde{K}, \text{ so KL} = 0 \text{ at start}).\tag{29}$$

5.3 Posterior moments in the eigenbasis

With $a = \text{KKtilde_inv_b} = K_b/\Lambda$, the projected moments (16) at the training points become (`eigenspace_model.py:_lambda_moments_eigenspace`):

$$\begin{aligned}\mu &= a m_b, \\ \sigma^2 &= \text{Kvec} + \text{diag}(a (V_b - \widetilde{K}_b) a^\top) = \text{Kvec} + \text{rowsum}(a \odot (a (V_b - \widetilde{K}_b))), \\ \sigma^2 &\leftarrow \max(\sigma^2, \text{LAMBDA_VAR_CLAMP} = 10^{-6}),\end{aligned}\tag{30}$$

where μ, σ^2 are the per-training-point vectors $\lambda_m, \lambda_{\text{var}}$ and $\odot$ is the element-wise product; the two variance expressions coincide because $\text{rowsum}(a \odot (aM)) = \text{diag}(aMa^\top)$ for any M . Term by term this equals (16): $\text{Kvec}_i = K(x_i, x_i)$ (the self-kernel identity (5)) and $a_i(V_b - \widetilde{K}_b)a_i^\top = k(x_i)^\top \widetilde{K}^{-1}(V - \widetilde{K})\widetilde{K}^{-1}k(x_i)$. For `test` points the same two lines run on a freshly projected $a = K(x_*, \widetilde{Z})B/\Lambda$ with $\text{Kvec} = K(x_*, x_*)$, so no kernel is recomputed for training points (`predict_eigenspace` returns $\{\text{f_pred}, \text{lambda_m}, \text{lambda_var}\}$ with $\text{f_pred} = \exp(A\lambda_m + \frac{1}{2}A^2\lambda_{\text{var}} + \lambda_0)$).

Remark 5.2 (Real method names). The expected firing rate lives on `PoissonLikelihood.expected_firing_rate(posterior)`; the `EigenspacePosterior` object exposes only `.mean` and `.variance` (computed once at construction). Older reference docs that place an `expected_firing_rate()` method on the posterior, or that name files `estep.py/mstep.py`, are stale — the live files are `eigenspace_estep.py`, `eigenspace_fstep.py`, `eigenspace_mstep.py`, `eigenspace_training.py`.

5.4 ELBO and KL in the eigenbasis

The eigenspace ELBO (`compute_elbo_eigenspace`) uses the log-likelihood (19) and, because $\widetilde{K}_b = \text{diag}(\Lambda)$, the KL (20) collapses to sums over the kept dimensions:

$$\begin{aligned}\text{KL}(q \parallel p) &= \frac{1}{2} \left[\text{Tr}(\widetilde{K}^{-1}V) + m^\top \widetilde{K}^{-1}m - n_b + \log|\widetilde{K}| - \log|V| \right] \\ &= \frac{1}{2} \left[\sum_j \frac{V_b[j, j]}{\Lambda_j} + \sum_j \frac{m_b[j]^2}{\Lambda_j} - n_b + \sum_j \log \Lambda_j - \log \det V_b \right].\end{aligned}\tag{31}$$

Remark 5.3 (Diagonal-trace assumption). The trace term uses only $\text{diag}(V_b)$, which is valid *only because* $\widetilde{K}_b$ is diagonal when the ELBO is evaluated — i.e. on a consistent (kernel, basis, params) triple right after initialisation, recompute, or the E/F-step, and before the M-step. `log_det V_b` uses the full `slogdet`; a non-positive-definite V_b warns and sets $\log|V| \rightarrow 0$. The $-n_b$ is a zero-gradient constant (cf. the $-M$ remark after (20)): it shifts only the reported loss magnitude.

5.5 The recompute seam after the M-step

An M-step changes the kernel hyperparameters, so $\widetilde{K}$ — and therefore its eigenbasis B and eigenvalues Λ — go stale. `DirectVGPMModel.recompute_eigenspace()` restores consistency in two steps:

1. recompute all eigenspace quantities from the *new* kernel (`_compute_eigenspace_quantities`);
2. *reproject* the variational params from the old basis to the new one by round-tripping through full M -space (`eigenspace_utils.py:reproject_variational_params`):

$$m_b^{\text{new}} = B_{\text{new}}^\top B_{\text{old}} m_b^{\text{old}}, \quad V_b^{\text{new}} = B_{\text{new}}^\top (B_{\text{old}} V_b^{\text{old}} B_{\text{old}}^\top) B_{\text{new}}.\tag{32}$$

In `train_eigenspace` this runs at the *start* of every iteration > 1 (guarded by `n_mstep>0` and `iteration>1`), so metrics and the ELBO (31) are always evaluated on a consistent triple. If n_b grows, the new dimensions carry no prior information and V_b^{new} may have tiny eigenvalues; the subsequent E-step restores positive-definiteness.

Caveat. $\widetilde{K}_b$ is diagonal *only* right after init/recompute and during the E/F-step. Inside the M-step closure, after a hyperparameter step but before the next recompute, the kernel has moved while the basis B is still the *old* one, so $\widetilde{K}_b = B^\top \widetilde{K}_{\text{new}} B$ is *not* diagonal — code there must use `torch.linalg.solve`, not `diag(1/Λ)`. Treating it as diagonal in that window is the diagonal-KL trap (a bug fixed Jan 2025).

5.6 The `default_gpy` path

`VariationalGPMModel` (`gpy_model.py`) realises the same model through stock GPyTorch, keeping $q(\tilde{\lambda})$ in full M -space and never touching the eigenbasis:

- `CholeskyVariationalDistribution(M)` stores $q(\tilde{\lambda}) = \mathcal{N}(m, L_{\tilde{K}} L_{\tilde{K}}^\top)$ via the Cholesky factor (numerically stable);
- `VariationalStrategy` (whitened, the default) or `UnwhitenedVariationalStrategy` (natural params), both with `jitter_val=JITTER=10-4`;
- `mean_module=ZeroMean()`, `covar_module=` the arc-cosine kernel; `forward(x)` returns the GP prior $\mathcal{N}(\text{ZeroMean}(x), K(x))$, and the strategy turns that plus $q(\tilde{\lambda})$ into $q(\lambda)$ at x .

It *is* an `nn.Module`, so kernel, likelihood, and variational params all live in one `state_dict`. Training (`gpy_training.py:train_gpy_default`) is a single joint ELBO maximisation over all parameters — no EM decomposition:

$$\begin{aligned} \text{loss} = & -\text{likelihood.expected_log_prob}(y, \text{model}(X)) \\ & + \text{model.variational_strategy.kl_divergence}(), \end{aligned} \quad (33)$$

minimised by LBFGS (`strong_wolfe`, `GPY_LBFGS_MAX_ITER=20`) or Adam. The LBFGS closure rejects a step (returns ∞) on out-of-bounds kernel/likelihood params, a NaN gram, or firing-rate divergence ($\bar{f}$ mean > 100 or max > 500); after each accepted step the kernel and likelihood are clamped (projected gradient descent). Cholesky stability comes from `cholesky_jitter(10-4)` with `cholesky_max_tries=3` (retries at $10^{-4}, 10^{-3}, 10^{-2}$). Prediction (`gpy_training.py:predict`) computes $\mu = \text{posterior.mean}$, $\sigma^2 = \max(\text{posterior.variance}, 10^{-6})$, and `f_pred = exp(Aμ + ½A²σ² + λ₀)` — the same (21), but returned under the key `lambda_mean` rather than `predict_eigenspace's lambda_m`.

5.7 Why `vargp_direct` manages parameters directly

The whitened `VariationalStrategy` stores params relative to $L_{\tilde{K}}$ (the Cholesky of $\widetilde{K}$), making the prior $\mathcal{N}(0, I)$; it *assumes joint autograd* optimisation, in which autograd differentiates through $L_{\tilde{K}}$ and keeps the whitened coupling consistent. A closed-form Newton E-step combined with a kernel-changing M-step (EM style) breaks that assumption. Write L_{old} and L_{new} for the Cholesky factor $L_{\tilde{K}}$ before and after such a step. Params stored as $m_{\text{stored}} = L_{\text{old}}^{-1} m_{\text{natural}}$ become stale the moment the M-step moves $L_{\text{old}} \rightarrow L_{\text{new}}$, so the next E-step reads $L_{\text{new}} m_{\text{stored}} \neq m_{\text{natural}}$ (empirically $\sim 8 \times$ errors in μ — the “ L mismatch”, DECISION_LOG Q26/Q27). There are two escapes:

1. `UnwhitenedVariationalStrategy` stores natural params ($L_{\tilde{K}}$ -independent), so EM is safe — but it is slower and empirically less accurate (KL-gradient instability, Q29);
2. `vargp_direct's` own eigenbasis whitens through B/Λ but *recomputes and reprojects the basis explicitly* after every M-step (§5.5) instead of relying on autograd.

This is exactly why `vargp_direct` owns its basis and runs a custom EM loop, while `default_gpy` stays in the whitened single-joint-ELBO regime that whitening was designed for. The eigenbasis B and GPyTorch’s $L_{\tilde{K}}$ play analogous roles (both diagonalise/whiten the prior); the difference is who keeps them in sync — `vargp_direct` does so explicitly, GPyTorch does so implicitly via autograd (which EM defeats).

5.8 Shared math, divergent representation

aspect	<code>vargp_direct</code> (<code>DirectVGPMModel</code>)	<code>default_gpy</code> (<code>VariationalGPMModel</code>)
posterior rep. of $q(\tilde{\lambda})$	eigenspace: m_b (n_b), dense V_b ($n_b \times n_b$) in basis B of $\tilde{K}$	full M -space: CholeskyVariationalDistribution , $q = \mathcal{N}(m, L_{\tilde{K}} L_{\tilde{K}}^T)$
$\tilde{K}^{-1}$	trivial $\text{diag}(1/\Lambda)$	GPyTorch Cholesky of $\tilde{K}$ + jitter
whitening	own eigenbasis, explicit reproject after each M-step	whitened $\mathcal{N}(0, I)$ (default) or unwhitened
optimiser	custom EM : E (Newton), F (A, λ_0), M (LBFGS) + recompute	single joint ELBO over all params (LBFGS/Adam)
GPyTorch role	kernel calculator only (<code>kernel(...).to_dense()</code>)	full ApproximateGP + VariationalStrategy
<code>nn.Module</code> ?	no (plain class; dataclass state)	yes
KL	manual eigenspace (31)	<code>variational_strategy.kl_divergence()</code>
dim. reduction	$M \rightarrow n_b$ eigenspace	none (full M)
online rank-1 update	<code>extend_model_with_new_point</code> (§5.9)	none provided
checkpoint	<code>eigenspace_checkpoint.py</code> (m_b, V_b ; no B)	<code>checkpoint.py</code> (<code>state_dict</code>)
predict return key	<code>f_pred, lambda_m, lambda_var</code>	<code>f_pred, lambda_mean, lambda_var</code>

Table 1: The two implementations of the identical model (§4): the ELBO, moments, likelihood, and firing rate are shared; only the representation of $q(\tilde{\lambda})$ and the optimiser differ.

Table 1 contrasts the two. Everything above the representation line is shared: the GP prior, the arc-cosine RF kernel, the Poisson(A, λ_0) likelihood, $\bar{f}$ of (21), the ELBO decomposition (18), the divergence guards ($\bar{f}$ mean/max 100/500), the ELBO early-stopping logic, $\text{JITTER} = 10^{-4}$, and the inducing-points-are-training-points invariant (Remark 4.3).

5.9 Online growth and the checkpoint seam

Rank-one update. Phase-2 active learning grows the model by one image per step (`rank1_update.py:extend_model_with_new_point`), producing a *new* `DirectVGPMModel` with $M+1$ points warm-started from the M -point model:

1. **Column-append to $\tilde{K}$.** Compute only the new column $K(\tilde{Z}_{\text{new}}, x_{\text{new}})$ ($O(M)$ kernel evals) and stitch it as a new row/column onto the stored `state.K_tilde` — this is *why* the full $\tilde{K}$ is retained (§5.2) — avoiding the $O(M^2)$ full recompute. (The $(M+1)$ -eigendecomposition is $O(M^3)$ either way.)
2. **Eigendecompose** the new $\tilde{K} \rightarrow B_{\text{new}}, \Lambda$ (n_b^{new}).
3. **Derived quantities** using the active-loop invariant $X = \tilde{Z}$ (so $K = \tilde{K}$): $\text{K_b} = \tilde{K}_{\text{new}} B_{\text{new}}$, $\tilde{K}_b = \text{diag}(\Lambda)$, $a = \text{K_b}/\Lambda$; `Kvec` extended by the single new diagonal $K(x_{\text{new}}, x_{\text{new}})$.
4. **Warm-start** the variational params (the legacy three-move): expand to full space $m_{\text{full}} = B_{\text{old}} m_b^{\text{old}}$, $V_{\text{full}} = B_{\text{old}} V_b^{\text{old}} B_{\text{old}}^T$ (symmetrised); pad to $M+1$ with a new mean entry $\text{mean}(m_{\text{full}})$ and a covariance $V_{\text{ext}} = I_{M+1}$ with $V_{\text{ext}}[:, M] = V_{\text{full}}$ (the new dimension

gets *unit variance, zero cross-covariance* — an identity placeholder for the yet-unseen point); reproject into the new basis $m_b^{\text{new}} = B_{\text{new}}^T m_{\text{ext}}$, $V_b^{\text{new}} = B_{\text{new}}^T V_{\text{ext}} B_{\text{new}}$.

5. Hand the precomputed state to `DirectVGPMModel(..., _precomputed_state=state)` (skipping the constructor’s own eigenspace build). The kernel and likelihood must be deep-copied by the caller, since `train_eigenspace` mutates the kernel in place.

Net: one active image appends one column to $\tilde{K}$, bumps n_b by ~ 1 , and re-warm-starts (m_b, V_b) — the online counterpart of the batch initialisation.

Checkpoint seam. `eigenspace_checkpoint.py` persists a `DirectVGPMModel` (a plain class, hence a bespoke checkpoint). It *saves* `kernel.state_dict()`, `likelihood.state_dict()`, the variational params m_b, V_b , `pool_indices` (indices into the shared image pool), pool integrity tags, and the RF-centre box. It does *not* save X (reconstructed as `X_pool[pool_indices]`, keeping checkpoints ~ 1 KB) nor the entire derived eigenspace $(B, \Lambda, \tilde{K}_b, K_b, a, K_{\text{vec}})$ — on load, `DirectVGPMModel.__init__` recomputes the eigenspace fresh from the loaded kernel, and the saved m_b, V_b are re-injected via `update_variational_params`.

Caveat. The saved m_b, V_b are expressed in the *save-time* eigenbasis B , but are injected into a *freshly recomputed* B with **no** reprojection — the code assumes the recomputed eigendecomposition equals the save-time one (“deterministic on the same hardware”). This is the documented basis-drift fragility: under a different torch version the load-time eigendecomposition of $\tilde{K}$ differs subtly (sign flips, near-threshold eigenvectors dropping in or out of the kept set, near-degenerate subspace rotation), so the reloaded posterior can drift by $\sim 5\%$. Note the asymmetry with the *in-memory* `restore_best` path, which *does* save B and reprojects (32); only the *on-disk* checkpoint omits the basis. Integrity is guarded for the data pool (shape exact, dtype warn, sum within tolerance), not for the basis.

Remark 5.4 (E/F/M objects only). The custom EM loop mutates this eigenspace state — the E-step Newton update of (m_b, V_b) (which `detaches` both, to stop the autograd graph chaining across iterations through A and blowing up memory, and symmetrises V_b), the F-step on (A, λ_0) , and the M-step LBFGS on the kernel hyperparameters. The update *equations* belong to the training-algorithm section; this section fixes only the objects each step reads and writes.

6 The training algorithm: EM-style coordinate ascent

The model assembled in Parts I–II has three groups of free parameters, and all three sit on the *single* evidence lower bound $\mathcal{L}$ of (18):

- (a) the **variational moments** m_b, V_b of the eigenspace posterior $q(\tilde{\lambda}) = \mathcal{N}(m, V)$ (Part II, §6 inputs);
- (b) the **firing-rate parameters** of the Poisson likelihood, the gain A and the offset λ_0 , defining $f = \exp(A\lambda + \lambda_0)$;
- (c) the **kernel hyperparameters** $\{\sigma_0, \text{Amp}, \beta, \rho, \varepsilon_{0x}, \varepsilon_{0y}\}$ of the structured arc-cosine prior (Part I).

Training maximises the one bound $\mathcal{L}$ by **block coordinate ascent**: each step improves one block while holding the other two fixed. The three steps are the E-step (variational moments), the F-step (firing-rate parameters), and the M-step (kernel hyperparameters), run once per outer iteration in a fixed order (§6.1). Ground truth throughout is the current eigenspace implementation

(`eigenspace_*.py`); the deprecated `vargp_old` engine is out of scope. We note only that the current E-step *numerically reproduces* the legacy *m*-update (§6.2), so no legacy code is needed to state the algorithm.

6.1 Overview: three coordinate blocks, one ELBO, one loop

Each step optimises one block and freezes the rest:

Step	Free block	Held fixed	Method / file
E	m_b, V_b	kernel; $A, \lambda_0; B$	closed-form Newton, no autograd (<code>eigenspace_estep.py</code>)
F	A (λ_0 closed-form)	m_b, V_b (via μ, σ^2); kernel	LBFSG on $\log A$ or damped Newton (<code>eigenspace_fstep.py</code>)
M	$\{\sigma_0, \text{Amp}, \beta, \rho, \xi_0\}$	$m_b, V_b; A, \lambda_0; B$	LBFSG + analytical grads (<code>eigenspace_mstep.py</code>)

(In the M-row, the RF centre $\xi_0 = (\varepsilon_{0x}, \varepsilon_{0y})$ counts as the two coordinate hyperparameters, so the free block has six scalars.)

Loop order (one outer iteration). The driver `eigenspace_training.py:train_eigenspace` runs, per iteration:

1. **Re-sync the eigenspace** — `recompute_eigenspace()` rebuilds the eigenbasis B from the hyperparameters the *previous* M-step moved, then refreshes $\mu, \sigma^2, \bar{f}$. (Guarded: only when $n_{\text{mstep}} > 0$ and iteration > 1 ; this is where the deferred B update from the last M-step lands.)
2. **E-step** — n_{estep} closed-form Newton steps on m_b, V_b (§6.2).
3. **F-step** — fit A (and closed-form λ_0), unless the F-step is interleaved into the E-loop (§6.3).
4. **Metrics + ELBO + early stopping** — computed *deliberately before* the M-step.
5. **M-step** — update the kernel hyperparameters (§6.4); the B update is *deferred* to step 1 of the next iteration.

Remark 6.1 (Why metrics precede the M-step). The M-step changes the kernel parameters *without* recomputing the eigenbasis B (that is deferred, for cost and stability). Every logged quantity — the ELBO, the early-stopping metric — must therefore be read while the model state is still internally consistent, i.e. *before* the M-step perturbs the kernel. Early stopping is on the ELBO (`es_metric='elbo'`, the only supported metric), with patience and best-restore.

6.2 The E-step: a Newton update of the variational posterior

At *fixed* kernel and fixed A, λ_0 , the E-step maximises the ELBO (18) over the variational moments. Writing the inducing-space posterior $q(\tilde{\lambda}) = \mathcal{N}(m, V)$ against the prior $p(\tilde{\lambda}) = \mathcal{N}(0, \tilde{K})$,

$$\mathcal{L}(m, V) = \underbrace{\sum_i \mathbb{E}_{q(\lambda_i)} [\log \text{Poisson}(y_i | \lambda_i)]}_{\mathcal{L}_{\text{data}}} - \text{KL}(q(\tilde{\lambda}) \| p(\tilde{\lambda})), \quad (34)$$

with the projected posterior moments μ_i, σ_i^2 of (16) and the KL contributing, through the E-step variables, $-\frac{1}{2}m^\top \tilde{K}^{-1}m$ (couples to m) and $\frac{1}{2} \log |V| - \frac{1}{2} \text{Tr}(\tilde{K}^{-1}V)$ (couples to V).

Poisson expected log-likelihood and its moment derivatives. The data term is closed-form via the Gaussian moment-generating function (`likelihoods.py:expected_log_prob`), with the expected firing rate $\bar{f}_i$ of (21):

$$\mathcal{L}_{\text{data}} = \sum_i [y_i(A\mu_i + \lambda_0) - \bar{f}_i], \quad \bar{f}_i = \exp(A\mu_i + \frac{1}{2}A^2\sigma_i^2 + \lambda_0), \quad (35)$$

(the $-\log y_i!$ constant is dropped). The three point-derivatives that assemble the Newton step follow by the chain rule on $\bar{f}_i$:

$$\frac{\partial \bar{f}_i}{\partial \mu_i} = A \bar{f}_i, \quad \frac{\partial \bar{f}_i}{\partial \sigma_i^2} = \frac{1}{2}A^2 \bar{f}_i, \quad \frac{\partial}{\partial \mu_i} \mathbb{E}_q[\log \text{Poisson}] = A(y_i - \bar{f}_i). \quad (36)$$

The two Newton helpers. Accumulating those derivatives through the projection $\mu_i = k_i^\top \widetilde{K}^{-1}m$ defines the gradient helper g_E and the (Poisson-Fisher) curvature G_E :

$$g_E = A \sum_i k_i(y_i - \bar{f}_i) \in \mathbb{R}^M, \quad G_E = A^2 \sum_i \bar{f}_i k_i k_i^\top \in \mathbb{R}^{M \times M}. \quad (37)$$

g_E is the kernel-projected mismatch between observed and expected counts; G_E is the Poisson Fisher information and is PSD because every $\bar{f}_i > 0$. (Note g_E carries the subscript E specifically to keep it distinct from the log-firing rate $g = A\lambda + \lambda_0$ of the utility part.)

The V-update (closed form). With μ_i independent of V , setting $\nabla_V \mathcal{L} = 0$ gives $V^{-1} = \widetilde{K}^{-1}(\widetilde{K} + G_E)\widetilde{K}^{-1}$, hence

$$V = \widetilde{K}(\widetilde{K} + G_E)^{-1}\widetilde{K} \quad (38)$$

— a *symmetric* sandwich of two symmetric-PD factors, so V is a valid covariance. This symmetry is the substance of the “corrected” E-step: the superseded form $V = (\widetilde{K} + G_E)^{-1}\widetilde{K}$ is a product of two non-commuting symmetric matrices and is therefore *not* symmetric (Cholesky/PSD failures).

The m-update (code form). The exact Newton step $m \leftarrow m - H_m^{-1}\nabla_m \mathcal{L}$, with $\nabla_m \mathcal{L} = \widetilde{K}^{-1}(g_E - m)$ and Hessian $H_m = -\widetilde{K}^{-1}(\widetilde{K} + G_E)\widetilde{K}^{-1}$, gives the corrected form $m \leftarrow m + \widetilde{K}(\widetilde{K} + G_E)^{-1}(g_E - m)$. The **shipped code** does *not* use this; it uses the algebraically older map

$$m \leftarrow \widetilde{K}(\widetilde{K} + G_E)^{-1}(G_E \widetilde{K}^{-1}m + g_E) \quad (\text{eigenspace_estep.py}) \quad (39)$$

(equal to $V_{\text{new}}(G_E \widetilde{K}^{-1}m + g_E)$ with V_{new} from (38)). The two maps differ only in the first term ($\widetilde{K}(\widetilde{K} + G_E)^{-1}G_E \widetilde{K}^{-1}m$ vs. $G_E(\widetilde{K} + G_E)^{-1}m$); they coincide iff $\widetilde{K}$ and G_E commute.

Caveat. The code m-update (39) is presented as *the* algorithm (code is ground truth). It differs *per iteration* from the corrected Newton step whenever $\widetilde{K}$ and G_E do not commute, but both maps share the same fixed point $m^* = g_E$, so the *converged* posterior is identical — the discrepancy is a transient path difference, not a different solution. The code carries this as an open TODO; it is left as-is (works empirically). The V-update (38) is unambiguously correct, and it is V that the active-learning variance estimates depend on.

Remark 6.2 (Fixed-point equivalence). Let $T_N(m) = m + \widetilde{K}(\widetilde{K} + G_E)^{-1}(g_E - m)$ (corrected) and $T_C(m) = \widetilde{K}(\widetilde{K} + G_E)^{-1}(G_E \widetilde{K}^{-1}m + g_E)$ (code). At $m = g_E$: $T_N(g_E) = g_E$, and $T_C(g_E) = \widetilde{K}(\widetilde{K} + G_E)^{-1}(G_E \widetilde{K}^{-1} + I)g_E = \widetilde{K}(\widetilde{K} + G_E)^{-1}(G_E + \widetilde{K})\widetilde{K}^{-1}g_E = \widetilde{K}\widetilde{K}^{-1}g_E = g_E$. Both are valid fixed-point iterations onto the same stationary condition $m^* = g_E = A \sum_i k_i(y_i - \bar{f}_i)$.

6.2.1 Why the update is cheap: the eigenspace realisation

The E-step never touches the M -dimensional inducing space directly. It works in the eigenbasis B of $\widetilde{K}$, keeping only the n_b leading directions (eigenvalues above $\max(\lambda_{\max} \cdot \text{tol}, \text{tol})$, $\text{tol} = 10^{-4}$; in practice $n_b \approx 10$ –11 while M may be large). With $\widetilde{K} = B\Lambda B^\top$ the projected gram $\widetilde{K}_b = B^\top \widetilde{K} B = \text{diag}(\Lambda)$ is *diagonal*, so $\widetilde{K}^{-1}$ is trivial. Using the eigenspace projection $a = K\widetilde{K}^{-1}$ (`KKtilde_inv_b`, shape $N \times n_b$), the code forms the *transformed* helpers (`eigenspace_estep.py`):

$$g_{E,b} = A a^\top (y - \bar{f}) = \widetilde{K}^{-1} g_E \quad (\text{eigenbasis coords}), \quad (40)$$

$$G_{E,b} = A^2 a^\top (\bar{f} \odot a) = \widetilde{K}^{-1} G_E \widetilde{K}^{-1}, \quad (41)$$

and applies (38)–(39) entirely in the tiny n_b -space:

$$V_b \leftarrow (I + \widetilde{K}_b G_{E,b})^{-1} \widetilde{K}_b, \quad m_b \leftarrow V_b (G_{E,b} m_b + g_{E,b}), \quad V_b \leftarrow \frac{1}{2}(V_b + V_b^\top). \quad (42)$$

The first line equals (38) exactly, via the identity $(I + PQ^{-1})^{-1} = Q(Q + P)^{-1}$ with $P = G_E$, $Q = \widetilde{K}$: $(I + \widetilde{K}_b G_{E,b})^{-1} \widetilde{K}_b = (I + G_E \widetilde{K}^{-1})^{-1} \widetilde{K} = \widetilde{K}(\widetilde{K} + G_E)^{-1} \widetilde{K}$. Prior initialisation is $m_b = 0$, $V_b = \widetilde{K}_b = \Lambda$. The dimensional collapse $M \rightarrow n_b \approx 10$ — not any special-casing — is what makes the Newton step cheap.

6.2.2 Iteration, damping, and the divergence guard

`estep_eigenspace` performs a *single, undamped, closed-form* Newton step. Iteration and stabilisation live in the driver:

- **Coupling variable $\bar{f}$.** After each step the posterior is recomputed and $\bar{f} = \exp(A\mu + \frac{1}{2}A^2\sigma^2 + \lambda_0)$ refreshed, then fed into the next step’s $g_{E,b}, G_{E,b}$. A, λ_0 are detached (fixed) during the E-step.
- **Revert-on-divergence guard (the effective damping).** Each iteration snapshots (m_b, V_b) ; if the post-step $\bar{f}$ mean or max exceeds a threshold, or $\bar{f}$ has NaNs, it *reverts* (m_b, V_b) (and A, λ_0 if interleaved), recomputes $\bar{f}$, and breaks the loop.
- **Optional interleaved F-step.** With `interleave_fstep`, a *damped* Newton update of (A, λ_0) (§6.3, $\alpha = 0.25$) runs after every E-iteration, so the firing-rate parameters track (m_b, V_b) as they move.

6.3 The F-step: fitting the firing-rate function

Here “F” denotes the **firing-rate function** f : the F-step fits the Poisson-rate parameters A, λ_0 that define $\bar{f} = \exp(A\mu + \frac{1}{2}A^2\sigma^2 + \lambda_0)$, holding the variational moments (through frozen μ, σ^2) and the kernel fixed. In this codebase A is the only searched variable; λ_0 is solved in closed form for the current A .

Caveat. The F-step is *undocumented* in the LaTeX/handoff sources (they cover only the kernel/M-step gradients). Its definition rests entirely on the code (`eigenspace_fstep.py`, `utils.py:lambda0_given_A`), which is ground truth.

Closed-form λ_0 given A . From $\partial\mathcal{L}/\partial\lambda_0 = 0$ on $\sum_i [y_i \lambda_0 - \exp(\lambda_0) \exp(A\mu_i + \frac{1}{2}A^2\sigma_i^2)]$:

$$\lambda_0 = \log\left(\sum_i y_i\right) - \log\left(\sum_i \exp(A\mu_i + \frac{1}{2}A^2\sigma_i^2)\right) \quad (43)$$

(requires $\sum_i y_i > 0$; a zero-spike cell is a data problem). It is re-evaluated whenever A changes.

LBFGS on A with an analytic gradient. The standard F-step (`fstep_eigenspace`) runs LBFGS over $\text{raw}_A = \log A$, re-solving λ_0 each closure and using the closed-form derivative (no autograd), with μ, σ^2 the frozen inputs:

$$\frac{\partial \mathcal{L}}{\partial A} = \sum_i [y_i \mu_i - (\mu_i + A \sigma_i^2) \bar{f}_i], \quad \frac{\partial \mathcal{L}}{\partial(\log A)} = A \frac{\partial \mathcal{L}}{\partial A}, \quad \text{raw}_A.\text{grad} = -A \frac{\partial \mathcal{L}}{\partial A}. \quad (44)$$

(The sign is because LBFGS minimises $-\mathcal{L}$.) Guards reject a trial step whose λ_0 over/underflows or whose $\bar{f}$ blows past the mean/max thresholds, and revert $(\text{raw}_A, \lambda_0)$ if the final λ_0 overflows.

Interleaved joint damped Newton on (A, λ_0) . When `interleave_fstep=True`, a joint damped Newton step (matching the reference paper’s `updateA`) is called *inside* the E-loop. With $\psi = [A, \lambda_0]$ and per image $f_i = \exp(A\mu_i + \frac{1}{2}A^2\sigma_i^2 + \lambda_0)$, $d_i = \mu_i + A\sigma_i^2$:

$$R = \begin{bmatrix} \sum_i (y_i \mu_i - d_i f_i) \\ \sum_i (y_i - f_i) \end{bmatrix}, \quad H = - \begin{bmatrix} \sum_i (\sigma_i^2 f_i + d_i^2 f_i) & \sum_i d_i f_i \\ \sum_i d_i f_i & \sum_i f_i \end{bmatrix}, \quad \psi \leftarrow \psi - \alpha H^{-1} R, \quad \alpha = 0.25. \quad (45)$$

H is negative-definite (the log-likelihood is concave in ψ); the damping $\alpha = 0.25$ and the same $\bar{f}$ -threshold rejections stabilise the step.

6.4 The M-step: updating the kernel hyperparameters

The M-step maximises the ELBO (18) over the kernel/prior hyperparameters while m_b, V_b , the firing-rate parameters A, λ_0 , and *the eigenbasis* B are held fixed. The genuine free hyperparameters (all owned by the arc-cosine kernel) are the six

$$\theta \in \{ \sigma_0, \text{Amp}, \beta, \rho, \varepsilon_{0x}, \varepsilon_{0y} \}, \quad (46)$$

carried in raw form as $\{\text{raw_sigma_0}, \text{raw_Amp}, \text{raw_m2log2beta} = -2 \log(2\beta), \text{raw_mlog2rho2} = -\log(2\rho^2), \varepsilon_{0x}, \varepsilon_{0y}\}$. B is deliberately *not* recomputed inside the M-step (an approximation for cost/stability); it is re-synced at step 1 of the next outer iteration.

Remark 6.3 (The amplitude Amp is optimized by default). The amplitude prefactor Amp of the structured prior ($C = \text{Amp} \alpha C_{\text{smooth}} \alpha^T$, (6)) is a genuine sixth kernel hyperparameter, not a fixed constant: the analytical M-step computes and applies its gradient $\text{raw_Amp.grad} = \partial_{\text{Amp}} \mathcal{J} \cdot \text{sigmoid}(\text{raw_Amp})$ (the softplus Jacobian; `eigenspace_mstep.py`), so Amp is optimized alongside the other kernel hyperparameters. The optional flag `fix_Amp` (default `False`) instead pins $\text{Amp} = 1$ by calling `raw_Amp.requires_grad_(False)` (`eigenspace_training.py`), reproducing the reference paper’s Amp-free kernel; left at its default, Amp trains with the rest.

Caveat. Constraint transforms. $\sigma_0 = \exp(\text{raw})$ and $\text{Amp} = \text{softplus}(\text{raw})$ (`kernels.py`). The `MSTEP_ANALYTICAL_HANDOFF` claim of “softplus for sigma_0” is *wrong*: σ_0 uses $\exp$, only Amp uses softplus.

6.4.1 The analytical gradient chain

The shipped M-step (`mstep_eigenspace_analytical`, run under `@torch.no_grad()`) differentiates the eigenspace ELBO analytically and writes `param.grad` directly. Writing the minimised loss $\mathcal{J} := -\mathcal{L}$, the per-hyperparameter chain is

$$\begin{aligned} \partial_\theta C &\rightarrow \{\partial_\theta K, \partial_\theta \widetilde{K}, \partial_\theta \text{Kvec}\} \rightarrow \{\partial_\theta K_b, \partial_\theta \widetilde{K}_b\} \rightarrow \{\partial_\theta \mu, \partial_\theta \sigma^2\} \\ &\rightarrow \{\partial_\theta \log\text{-lik}, \partial_\theta \text{KL}\} \rightarrow \partial_\theta \mathcal{J} \rightarrow \text{param.grad}. \end{aligned} \quad (47)$$

Steps 1–3: kernel gradients and their projection. Step 1 computes the masked spatial covariance C and its six gradients $\partial_\theta C$ (Part I, (12)); σ_0 enters the arc-cosine kernel K directly, not through C . Step 2 evaluates the kernel three times to get $\widetilde{K} = K(\widetilde{Z}, \widetilde{Z})$, $K = K(X, \widetilde{Z})$, $\text{Kvec} = \text{diag } K(X, X)$ and their $\partial_\theta \widetilde{K}$, $\partial_\theta K$, $\partial_\theta \text{Kvec}$ from $\partial_\theta C$ by the arc-cosine forward chain (Part I). Step 3 projects onto the fixed basis B :

$$\widetilde{K}_b = B^\top \widetilde{K} B, \quad K_b = K B, \quad \partial_\theta \widetilde{K}_b = B^\top (\partial_\theta \widetilde{K}) B, \quad \partial_\theta K_b = (\partial_\theta K) B, \quad \widetilde{K}_b^{-1} = \text{solve}(\widetilde{K}_b, I), \quad (48)$$

where $\widetilde{K}_b^{-1}$ is a *full* linear solve (matching the reference engine), not an eigenvalue reciprocal.

Remark 6.4 (Two different kernel gradients). The M-step needs the *hyperparameter* gradient $\partial_\theta K$ (through $\partial_\theta C$, (12)). This is distinct from the *input* gradient $\nabla_x K$ of (10), which feeds the utility and the diffusion guidance (Parts III–IV), not training.

Step 4: posterior-moment gradients. With the projection $a = K_b \widetilde{K}_b^{-1}$, the moments are $\mu = a m_b$ (cf. (16)) and $\sigma^2 = \text{Kvec} + \text{diag}(a(V_b - \widetilde{K}_b)a^\top)$ (clamped at a floor). Their hyperparameter derivatives (`compute_lambda_moments_and_gradients`) are

$$\partial_\theta a = (\partial_\theta K_b - a \partial_\theta \widetilde{K}_b) \widetilde{K}_b^{-1}, \quad (49)$$

$$\partial_\theta \mu = \partial_\theta a \cdot m_b, \quad (50)$$

$$\partial_\theta \sigma^2 = \partial_\theta \text{Kvec} + 2 \text{diag}(\partial_\theta a V_b a^\top) - \text{diag}(\partial_\theta K_b a^\top) - \text{diag}(K_b \partial_\theta a^\top). \quad (51)$$

Steps 5–6: loss gradients and the constraint transform. Splitting $\mathcal{J} = -\log\text{-lik} + \text{KL}$ (`compute_loss_gradients`), and letting $\cdot$ denote the sum over images:

$$\partial_\theta \log\text{-lik} = A(y \cdot \partial_\theta \mu) - A(\bar{f} \cdot \partial_\theta \mu) - \frac{1}{2} A^2(\bar{f} \cdot \partial_\theta \sigma^2), \quad (52)$$

$$\partial_\theta \text{KL} = \frac{1}{2} \text{Tr}(P_\theta) - \frac{1}{2} \text{Tr}(c P_\theta) - \frac{1}{2} w^\top (P_\theta m_b), \quad c = V_b \widetilde{K}_b^{-1}, \quad w = \widetilde{K}_b^{-1} m_b, \quad (53)$$

$$P_\theta = \partial_\theta \widetilde{K}_b \widetilde{K}_b^{-1}, \quad (54)$$

$$\partial_\theta \mathcal{J} = -\partial_\theta \log\text{-lik} + \partial_\theta \text{KL}.$$

Step 6 applies the constraint Jacobian $d(\text{value})/d(\text{raw})$ to write each raw parameter’s gradient (`eigenspace_mstep.py`):

$$\text{raw_sigma_0.grad} = \partial_{\sigma_0} \mathcal{J} \cdot \sigma_0, \quad \text{raw_Amp.grad} = \partial_{\text{Amp}} \mathcal{J} \cdot \text{sigmoid}(\text{raw_Amp}), \quad (55)$$

with $\sigma_0 = \exp(\text{raw}) \Rightarrow d\sigma_0/d\text{raw} = \sigma_0$ and $\text{Amp} = \text{softplus}(\text{raw}) \Rightarrow d\text{Amp}/d\text{raw} = \text{sigmoid}(\text{raw})$. The remaining four raw parameters — $\varepsilon_{0x}, \varepsilon_{0y}$ and `raw_m2log2beta`, `raw_mlog2rho2` — take `grad = \partial_{(\cdot)} \mathcal{J}` *directly*, with no transform factor. Both σ_0 (through $\exp$) and Amp (through softplus) carry a non-identity constraint Jacobian, because their $\partial C/\partial K$ were taken w.r.t. the *values*. The RF centre ξ_0 is its own raw value, and the two `raw_m*` gradients are already derivatives w.r.t. the raw log-parameter (the $\partial_\theta C$ formulas of (12) differentiate the raw directly).

Remark 6.5 (Local symbols renamed from the source notes). The source code reuses B for both the eigenbasis B and the KL matrix $\partial_\theta \widetilde{K}_b \widetilde{K}_b^{-1}$, and b for both the eigenbasis subscript and the vector $\widetilde{K}_b^{-1} m_b$. To keep B unambiguous, this document renames the KL matrix to P_θ and the KL vector to w (their values are unchanged).

6.4.2 Two gradient systems — and why the VJP is not the M-step

The codebase carries *two* analytical-gradient systems that share the $\partial_\theta C / \partial_\theta K$ formulas but differ in API and caller:

- **System 1 — kernel-level:** the Jacobian (`analytical_gradients.py`) and VJP (`analytical_gradients_vjp.py`) `autograd.Function` wrappers, selected by `gradient_mode`. Their backward takes `grad_output =  $\partial\mathcal{J}/\partial K$` and returns per-hyperparameter grads. Used when the kernel is one node in a general autograd graph.
- **System 2 — M-step level** (`eigenspace_gradients.py` + `mstep_eigenspace_analytical`). *Not* an `autograd.Function`; runs under `no_grad`, computes the entire chain (47) analytically, writes `param.grad`. **This is the system that actually differentiates the ELBO w.r.t. hyperparameters during training.**

The VJP formulation (System 1). The VJP computes the *same* $\partial\mathcal{J}/\partial\theta$ as the Jacobian version but never materialises the per-hyperparameter $\partial_\theta K$ matrices. It forms $\partial\mathcal{J}/\partial C$ *once* (a single $n_x \times n_x$ matrix, from outer products of the masked stimuli) and contracts it against the cheap $\partial_\theta C$ structure:

$$\frac{\partial\mathcal{J}}{\partial\text{Amp}} = \frac{(\partial_C\mathcal{J} \odot C)_\Sigma}{\text{Amp}}, \quad \frac{\partial\mathcal{J}}{\partial\alpha} = 2((\partial_C\mathcal{J} \odot \text{Amp } C_{\text{smooth}})\alpha), \quad \frac{\partial\mathcal{J}}{\partial\sigma_0} = 2\sigma_0\left(\sum \frac{\partial\mathcal{J}}{\partial V_1} + \sum \frac{\partial\mathcal{J}}{\partial V_2} + \sum \frac{\partial\mathcal{J}}{\partial x_1 C x_2}\right), \quad (56)$$

and chains $\partial\mathcal{J}/\partial\alpha$ to $\beta, \varepsilon_{0x}, \varepsilon_{0y}$ and $\partial\mathcal{J}/\partial C_{\text{smooth}}$ to ρ . This is one backward sweep, $O(n_x^2 n)$, versus materialising six $\partial_\theta K$ matrices (~ 15 large matmuls) — roughly 10–15 $\times$ cheaper.

Caveat. The VJP is *not* used for the LBFGS M-step. The M-step loss is not a plain function of K : it flows $K \rightarrow (a, \mu, \sigma^2) \rightarrow \text{log-lik/KL}$ with the eigenspace projection and the Poisson likelihood in between, and LBFGS’s strong-Wolfe line search re-evaluates the loss many times per step (a *different* $\partial\mathcal{J}/\partial K$ each time). System 2 therefore recomputes $C/\partial C/K/\partial K$ *inside* the closure on every evaluation and runs the full moment $\rightarrow \text{log-lik/KL}$ chain (47); it never forms a single $\partial\mathcal{J}/\partial K$ contraction. The “compute ∂K once / single $(\partial_K \mathcal{J} \cdot \partial K) \cdot \text{sum}()$ reduction” described in the handoff docs is System 1’s kernel-level backward, which those docs conflated with the M-step.

6.4.3 Numerical guardrails (M-step closure)

Inline in `mstep_eigenspace_analytical`: (1) out-of-bounds trial $\rightarrow +\infty$ reject; (2) $\cos\theta$ clamped to $[-1 + \epsilon, 1 - \epsilon]$ before $\arccos$, $\sin\theta = \sqrt{\max(1 - \cos^2\theta, \epsilon)}$, and division by the magnitude $\mathcal{M}$ jittered by $\epsilon = 10^{-7}$; (3) $\widetilde{K}, \widetilde{K}_b$ symmetrised $(M + M^\top)/2$; (4) $\widetilde{K}_b^{-1}$ via `torch.linalg.solve`; (5) $\bar{f}$ mean over threshold or any NaN $\rightarrow$ reject; (6) $\log|\widetilde{K}_b|$ via Cholesky ($2 \sum \log \text{diag } L_{\widetilde{K}}$), Cholesky failure $\rightarrow$ reject; (7) loss NaN/Inf or $\log|V_b| \text{ sign} \leq 0 \rightarrow$ reject; (8) gradient NaN/Inf counted and warned. σ^2 floored at `lambda_var_clamp`; an LBFGS line-search crash keeps the pre-step parameters; float64 throughout (kernel values can reach $\sim 10^4$).

Equation-to-code map

Object	Equation	Code (gpytorch_porting/)
E-step V, m update	(38), (39)	eigenspace_estep.py:estep_eigenspace
E-step helpers g_E, G_E	(37)	likelihoods.py:PoissonLikelihood
E/F/M loop order	§6.1	eigenspace_training.py: train_eigenspace
F-step λ_0 closed form	(43)	utils.py:lambda0_given_A
F-step A grad; damped Newton	§6.3	eigenspace_fstep.py
M-step gradient chain	(47)	eigenspace_mstep.py + eigenspace_gradients.py
Kernel-level Jacobian / VJP	§6.4	analytical_gradients.py / analytical_gradients_vjp.py

Part III

Active learning: the information utility

7 The acquisition objective

Active learning closes the loop: after each spike-count observation the model is refit, and the next natural image to display is the one whose observation is expected to teach us the most about the neuron’s tuning. “Teach us the most” is made precise as an *information gain* — an expected reduction in the entropy of what we do not yet know about the retinal ganglion cell’s firing behaviour. This section builds that utility on the generative model and variational posterior of Parts I–II: the Poisson-lognormal predictive of the spike count, its entropy, and the two utilities the codebase actually computes.

Throughout, the candidate (query) image is x^* ; a spike count is the response random variable R with realizations r (this is the spike count written y elsewhere in the document — we keep it as r here to match the count-domain code variables $r_{\max}$ and the Laplace mode $\bar{g}_r$). The neuron’s mean firing rate to image x is $f(x) = \exp(A\lambda(x) + \lambda_0)$ with latent $\lambda(x)$, gain A , and offset λ_0 ; the spike count is $R \mid f(x) \sim \text{Poisson}(f(x))$. It is convenient to work with the *log-firing rate* $g(x) = A\lambda(x) + \lambda_0$, so $f = e^g$, whose GP posterior at any image is Gaussian with the log-rate moments

$$\mu_g = A\mu(x^*) + \lambda_0, \quad \sigma_g^2 = A^2\sigma^2(x^*), \quad (57)$$

the affine push-forward of the latent posterior moments $\mu(x^*)$, $\sigma^2(x^*)$ read off the variational posterior (16).

Remark 7.1 (Two utilities, one idea — and which one ships). The codebase carries *two* acquisition functions. Both express the same idea — *utility*(x^*) = *information the observation R at x^* gives about the neuron’s tuning* = *a drop in entropy* — and they differ only in *which* uncertainty they score (Table 2). The **standard utility** U_{std} scores the firing rate *at x^* itself*; the **distribution-aware utility** U_{DA} scores the firing rates at the natural images $x \sim p(x)$ we ultimately want to predict. The production closed-loop experiment selects images with U_{std} (verified: `run_active_loop.py:241` calls `standard_utility`). U_{DA} is the theoretically richer object derived in the internal write-ups, but in the current code it is a research instrument used only under *investigations/* — and it carries a norm-exploitation divergence (§7.6, Thm. 8.9) that the finite image pool masks in production. We label U_{std} = production, U_{DA} = research, throughout.

	Standard U_{std} (production)	Distribution-aware U_{DA} (research)
Formula	$H_{\text{marg}}(x^*) - \mathbb{E}[H_{\text{noise}}(x^*)]$	$H_{\text{marg}}(x^*) - \mathbb{E}_{x \sim p(x)}[H_{\text{cond}}(x^*)]$
As MI	$I(f(x^*); R \mid x^*, \mathcal{D})$	$I(f(X); R \mid X, x^*, \mathcal{D}), X \sim p(x)$
Scores	rate <i>at x^* itself</i>	rates <i>at natural images $x \sim p(x)$</i>
Needs	marginal moments at x^*	marginal moments <i>and</i> cross-cov. to $p(x)$
Code	<code>standard_utility</code> → <code>nd_utility_new</code>	<code>distribution_aware_utility</code>
Where	<code>run_active_loop.py:241</code> (real experiment)	<i>investigations/ only</i>

Table 2: The two utilities. Both are “information gain = entropy reduction”, about different targets. U_{std} ships; U_{DA} is a research variant.

7.1 Utility as expected entropy reduction and mutual information

Fix a candidate x^* and the data $\mathcal{D} = \{(x_i, r_i)\}$. The utility is the mutual information between the response R we would record at x^* and the firing behaviour we want to learn. What “firing behaviour” means is exactly the difference between the two utilities.

The distribution-aware objective. Let $Z := (X, f(X))$ with $X \sim p(x)$ a natural image and $f(X)$ its mean firing rate. Choosing x^* to maximize the information R carries about Z ,

$$U_{\text{DA}}(x^*) := I(Z; R \mid x^*, \mathcal{D}) = I((X, f(X)); R \mid x^*, \mathcal{D}). \quad (58)$$

The chain rule for mutual information splits this into a term for *which* natural image we imagine and a term for its *rate*:

$$I((X, f(X)); R \mid x^*, \mathcal{D}) = \underbrace{I(X; R \mid x^*, \mathcal{D})}_{=0} + I(f(X); R \mid X, x^*, \mathcal{D}). \quad (59)$$

The first term vanishes: *which* natural image X we intend to ask about is independent of the response R recorded at x^* , given x^* and $\mathcal{D}$. Hence

$$U_{\text{DA}}(x^*) = I(f(X); R \mid X, x^*, \mathcal{D}) = \mathbb{E}_{x \sim p(x)}[I(f(x); R \mid x, x^*, \mathcal{D})]. \quad (60)$$

Entropy decomposition. For a fixed natural image x , the per-image mutual information decomposes into a marginal-entropy term minus an expected conditional-entropy term,

$$I(f(x); R \mid x, x^*, \mathcal{D}) = H(R \mid x^*, \mathcal{D}) - \mathbb{E}_{f(x)}[H(R \mid x^*, f(x), \mathcal{D})], \quad (61)$$

using (i) $H(R \mid x, x^*, \mathcal{D}) = H(R \mid x^*, \mathcal{D})$ — the marginal of R at x^* does not depend on which natural image we intend to predict — and (ii) that conditioning on the latent value $f(x)$ leaves the expected entropy $\mathbb{E}_{f(x)}[H(R \mid x^*, f(x), \mathcal{D})]$. Averaging (61) over $x \sim p(x)$ and naming the two pieces gives the distribution-aware utility as a difference of entropies,

$$U_{\text{DA}}(x^*) = H_{\text{marg}}(x^*) - \mathbb{E}_{x \sim p(x)}[H_{\text{cond}}(x^*)], \quad (62)$$

with

$$H_{\text{marg}}(x^*) := H(R \mid x^*, \mathcal{D}), \quad (63)$$

$$H_{\text{cond}}(x^*) := \mathbb{E}_{x \sim p(x), \lambda(x) \sim q}[H(R \mid x^*, \lambda(x), \mathcal{D})]. \quad (64)$$

By the symmetry of mutual information, $U_{\text{DA}}(x^*)$ equals the reduction in uncertainty about the firing rates $\{f(x) : x \sim p(x)\}$ produced by observing R at x^* — the precise sense of “how much does showing x^* teach us about the cell’s tuning on natural images.”

Remark 7.2 (Two expectations inside H_{cond}). H_{cond} in (64) carries *two* expectations: over natural images $x \sim p(x)$ and over the posterior latent $\lambda(x) \sim q$. The latent is integrated because $f(x) = \exp(A\lambda(x) + \lambda_0)$ is itself uncertain — only the posterior of $\lambda(x)$ is known, not its value. Replacing the inner expectation by an evaluation at the mean $\lambda(x) = \mu(x)$ is *biased*; the necessity of the inner sampling is the subject of §7.5.

The *standard* utility instead scores the rate at the query point itself. It is the special case in which the “natural image we predict” is x^* , so U_{DA} ’s outer expectation collapses and H_{cond} becomes the aleatoric Poisson noise entropy at x^* ; we develop it in §7.3 once H_{marg} is in hand.

7.2 The marginal spike-count entropy H_{marg}

Both utilities share the marginal predictive entropy of the spike count at x^* ,

$$H_{\text{marg}}(x^*) = - \sum_{r=0}^{\infty} p(r | x^*, \mathcal{D}) \log p(r | x^*, \mathcal{D}), \quad (65)$$

where the marginal predictive integrates the Poisson likelihood against the Gaussian posterior of the log-rate $g(x^*) \sim \mathcal{N}(\mu_g, \sigma_g^2)$ (cf. (57)):

$$p(r | x^*, \mathcal{D}) = \int \text{Poisson}(r | e^g) \mathcal{N}(g | \mu_g, \sigma_g^2) dg = \frac{1}{r!} \int \exp(rg - e^g) \mathcal{N}(g | \mu_g, \sigma_g^2) dg. \quad (66)$$

This integral has no closed form. The code evaluates it by a **Laplace expansion** of the integrand about its mode $\bar{g}_r$ (the mode of the log-integrand $rg - e^g - (g - \mu_g)^2/2\sigma_g^2$):

$$\log p(r | x^*, \mathcal{D}) \approx \bar{g}_r r - e^{\bar{g}_r} - \frac{(\bar{g}_r - \mu_g)^2}{2\sigma_g^2} - \frac{1}{2} \log(1 + \sigma_g^2 e^{\bar{g}_r}) - \log r!. \quad (67)$$

Lemma 7.3 (Closed-form Laplace mode via Lambert W). *The stationarity condition $r - e^g - (g - \mu_g)/\sigma_g^2 = 0$ rearranges to $g = r\sigma_g^2 + \mu_g - \sigma_g^2 e^g$. Substituting $w = \sigma_g^2 e^g$ gives $w + \log w = \log \sigma_g^2 + r\sigma_g^2 + \mu_g$, i.e. $w = W_0(\sigma_g^2 e^{r\sigma_g^2 + \mu_g})$ with W_0 the principal branch of the Lambert W function. Hence the mode is closed-form,*

$$\bar{g}_r = r\sigma_g^2 + \mu_g - W_0(\sigma_g^2 e^{r\sigma_g^2 + \mu_g}). \quad (68)$$

The infinite sum in (65) decays fast (retinal firing rates are low) and is truncated at r_{max} (§7.6 handles the truncation artifact). At very small variance the Laplace form is unstable, and the code falls back to the exact Poisson log-pmf $\log p(r) = r\mu_g - e^{\mu_g} - \log r!$ via a differentiable branch (threshold $\sigma_g^2 < 10^{-6}$). The mode is computed in log-space — $W_0(e^y)$ with $y = \log \sigma_g^2 + r\sigma_g^2 + \mu_g$, Newton-iterated with a custom backward $dW/dy = W/(1 + W)$ — so no $e^{r\sigma_g^2 + \mu_g}$ is ever formed (overflow-safe, gradient-safe).

7.3 The standard (production) utility

The production acquisition is the mutual information between the response and the latent firing rate *at the query point itself*,

$$U_{\text{std}}(x^*) = H_{\text{marg}}(x^*) - \mathbb{E}[H_{\text{noise}}(x^*)] = H(R | x^*, \mathcal{D}) - \mathbb{E}_{f \sim \text{post.}}[H(R | f)] = I(f(x^*); R | x^*, \mathcal{D}). \quad (69)$$

It scores “how much would a spike count at x^* pin down the firing rate at x^* ” — it depends on the marginal GP moments at x^* only, with no $p(x)$ and no cross-covariance. It is exactly U_{DA} with the predicted image taken to be the query point, so the second term is the *aleatoric* Poisson noise entropy rather than an epistemic conditional entropy.

7.3.1 The aleatoric noise entropy $\mathbb{E}[H_{\text{noise}}]$ in closed form

Given a rate f , the entropy of $\text{Poisson}(f)$ is $H_{\text{noise}}(f) = f(1 - \log f) + \mathbb{E}_{r \sim \text{Poisson}(f)}[\log r!]$. Writing $g = \log f \sim \mathcal{N}(\mu_g, \sigma_g^2)$ and using the lognormal moment identities

$$\mathbb{E}[e^g] = \exp(\mu_g + \tfrac{1}{2}\sigma_g^2) = \bar{f}(x^*), \quad \mathbb{E}[g e^g] = \exp(\mu_g + \tfrac{1}{2}\sigma_g^2) (\mu_g + \sigma_g^2), \quad (70)$$

(the first is the expected firing rate $\bar{f}$ of (21)), the first piece integrates in closed form:

$$\mathbb{E}_g[f(1 - \log f)] = \mathbb{E}_g[e^g(1 - g)] = \exp(\mu_g + \tfrac{1}{2}\sigma_g^2)(1 - (\mu_g + \sigma_g^2)) = -\exp(\mu_g + \tfrac{1}{2}\sigma_g^2)(\mu_g + \sigma_g^2 - 1). \quad (71)$$

The remaining $\mathbb{E}[\log r!]$ is taken under the marginal predictive $p(r \mid x^*, \mathcal{D})$ of (66). Hence

$$\mathbb{E}[H_{\text{noise}}(x^*)] = -\exp(\mu_g + \tfrac{1}{2}\sigma_g^2)(\mu_g + \sigma_g^2 - 1) + \sum_r p(r \mid x^*, \mathcal{D}) \log r!. \quad (72)$$

This is the closed form used by `nd_utility_new` (its code comment cites Eq. 33 of Goldin et al. 2023, PNAS). The production path `standard_utility` (i) reads the marginal latent moments $\mu(x^*), \sigma^2(x^*)$ via `get_gp_marginal_moments`, (ii) transforms them to μ_g, σ_g^2 by (57), (iii) optionally sets r_{max} adaptively, and (iv) returns $U_{\text{std}} = H_{\text{marg}} - \mathbb{E}[H_{\text{noise}}]$ from `nd_utility_new`.

7.4 The distribution-aware (research) utility

The distribution-aware utility (62) reduces uncertainty about firing rates at the natural images we actually want to predict. It weights a candidate x^* by how informative its observation is about responses to typical natural images — i.e. by the cross-covariance between x^* and images $x \sim p(x)$ (“distribution-aware” = aware of the natural-image distribution $p(x)$). Evaluating H_{cond} needs the predictive moments of $\lambda(x^*)$ *after a hypothetical observation* $\lambda(x) = \lambda_i$ at a natural image x .

7.4.1 Conditional GP moments via Gaussian conditioning

Let $u(x) = \widetilde{K}^{-1}k(x)$ be the SVGP projection and $q(\tilde{\lambda}) = \mathcal{N}(m, V)$ the variational posterior on the inducing values. The marginal latent posterior moments at x and x^* are the projected moments of (16), $\mu(x) = u(x)^\top m$, $\sigma^2(x) = s(x) + u(x)^\top V u(x)$ (and likewise at x^*), where $s(x) = k(x, x) - u(x)^\top k(x)$ is the prior Schur complement. The object that couples the two points is the **prior conditional covariance** of $\lambda(x), \lambda(x^*)$ given the inducing values — the term an earlier hand-assembled formulation omitted:

$$c = k(x, x^*) - u(x)^\top \widetilde{K} u(x^*). \quad (73)$$

By the law of total covariance the full posterior cross-covariance is

$$\Sigma_{x^*} = c + u(x)^\top V u(x^*) = k(x, x^*) + u(x)^\top (V - \widetilde{K}) u(x^*), \quad (74)$$

and standard Gaussian conditioning on the joint of $(\lambda(x), \lambda(x^*))$ gives the conditional latent moments at x^* after observing $\lambda(x) = \lambda_i$,

$$\mu_{\text{cond}}(x^*) = \mu(x^*) + \frac{\Sigma_{x^*}}{\sigma^2(x)} (\lambda_i - \mu(x)), \quad (75)$$

$$\sigma_{\text{cond}}^2(x^*) = \sigma^2(x^*) - \frac{\Sigma_{x^*}^2}{\sigma^2(x)}. \quad (76)$$

These are transformed to log-rate space, $A\mu_{\text{cond}} + \lambda_0$ and $A^2\sigma_{\text{cond}}^2$, and fed to the same entropy functional (65) to give the per-image conditional entropy $H(R \mid x^*, \lambda(x), \mathcal{D})$. The full Gaussian-conditioning derivation (augmented inducing matrix, block inverse, and the equivalence to the rank-1 posterior update) is the deep-layer result (84); and the conditional mean (75) obtained by conditioning on the joint posterior coincides with the mean obtained by augmenting the inducing set with the hypothetical observation and re-integrating (Thm. 8.5). Neither is re-derived here.

Remark 7.4 (The code is correct-by-construction). `get_gp_conditional_moments` builds the joint $[x; x^*]$, reads the *full covariance matrix* from the GPyTorch posterior, and takes $\mu_{\text{cond}}, \sigma_{\text{cond}}^2$ from its blocks (with a clamp $\sigma_{\text{cond}}^2 \geq 10^{-8}$). Because the variational posterior covariance between any two points already equals $c + u^\top V u^* = \Sigma_{x^*}$ — the c term included — the code never forms the buggy $u^\top V u^*$ -only cross-covariance. The “missing- c ” correction in the write-ups is a warning about *hand-assembled* cross-covariances (and about the deferred `vargp_direct` augmented-matrix path), not a defect in the shipped predict path.

Monte-Carlo estimator (one λ per image). Given pre-drawn samples $\{x_i\}_{i=1}^{N_{\text{mc}}} \sim p(x)$:

1. $H_{\text{marg}}(x^*) = \text{entropy}$ (65) at the marginal moments μ_g, σ_g^2 of x^* .
2. For $i = 1, \dots, N_{\text{mc}}$: read $(\mu(x_i), \sigma^2(x_i))$; draw $\lambda_i = \mu(x_i) + \sqrt{\sigma^2(x_i)}\varepsilon$, $\varepsilon \sim \mathcal{N}(0, 1)$ (if `sample_lambda`; else $\lambda_i = \mu(x_i)$); form the conditional moments (75)–(76) at x^* ; set $H_{\text{cond}}^{(i)} = \text{entropy at } (A\mu_{\text{cond}} + \lambda_0, A^2\sigma_{\text{cond}}^2)$.
3. $H_{\text{cond}} = \frac{1}{N_{\text{mc}}} \sum_i H_{\text{cond}}^{(i)}$; $U_{\text{DA}} = H_{\text{marg}} - H_{\text{cond}}$.

The x_i -posterior and the reparameterized draw λ_i are computed under `torch.no_grad()` (they do not depend on x^* , so excluding them avoids $O(N_{\text{mc}})$ graph accumulation); the conditional moments at x^* are differentiable in x^* . `sample_lambda` is *not* a debug flag: `False` uses the mean $\lambda_i = \mu(x_i)$ (a deterministic, reproducible sanity check — §7.4.2); `True` is the correct unbiased estimator (§7.5).

7.4.2 The deterministic special case

Setting $\lambda_i = \mu(x)$ (the posterior mean) makes the innovation $\lambda_i - \mu(x) = 0$, so (75)–(76) collapse to $\mu_{\text{cond}} = \mu(x^*)$ (mean unchanged) and

$$\sigma_{\text{cond}}^2(x^*) = \sigma^2(x^*)(1 - \rho_\lambda^2), \quad \rho_\lambda^2 = \frac{\Sigma_{x^*}^2}{\sigma^2(x)\sigma^2(x^*)}, \quad (77)$$

where ρ_λ is the posterior latent correlation between $\lambda(x)$ and $\lambda(x^*)$. In log-rate coordinates the conditioning map is $(\mu_g, \sigma_g^2) \mapsto (\mu_g, \sigma_g^2(1 - \rho_\lambda^2))$: same mean, variance scaled by $1 - \rho_\lambda^2$. Writing $H_{\text{marg}}(\cdot, \cdot)$ for the spike-count entropy (65) as a function of the log-rate moments, the utility is the entropy difference

$$U_{\text{DA}}(x^*) = H_{\text{marg}}(\mu_g, \sigma_g^2) - H_{\text{marg}}(\mu_g, \sigma_g^2(1 - \rho_\lambda^2)). \quad (78)$$

So the deterministic U_{DA} depends on exactly three quantities: the operating point μ_g , the marginal log-rate variance σ_g^2 (how much entropy to start with), and ρ_λ^2 (the fraction of that variance conditioning removes). ρ_λ^2 is governed by the angle between x^* and the conditioning image in C -space: a small angle gives $\rho_\lambda^2 \rightarrow 1$ (near-total variance removal), a large angle $\rho_\lambda^2 \rightarrow 0$ (no conditioning). This same ρ_λ^2 sets the sampling bias of §7.5.

7.5 Why we sample the latent instead of using its mean

H_{cond} is a double expectation (Remark 7.2), over $x \sim p(x)$ and over $\lambda(x) \sim q$. The tempting shortcut replaces the inner expectation by evaluation at the mean,

$$\tilde{H}_{\text{cond}}(x^*) = \mathbb{E}_{x \sim p(x)}[H(R \mid x^*, \lambda(x) = \mu(x), \mathcal{D})]. \quad (79)$$

Proposition 7.5 (The mean shortcut is biased). $\mathbb{E}_{\lambda(x)}[H(R \mid x^*, \lambda(x), \mathcal{D})] \neq H(R \mid x^*, \lambda(x) = \mu(x), \mathcal{D})$ in general. The predictive mean of $\lambda(x^*)$ is linear in the conditioning value, $\mathbb{E}[\lambda(x^*) \mid \lambda(x), \mathcal{D}] = \mu(x^*) + \alpha(\lambda(x) - \mu(x))$ with regression coefficient $\alpha = \text{Cov}(\lambda(x^*), \lambda(x) \mid \mathcal{D}) / \sigma^2(x)$, while the predictive variance does not depend on the value of $\lambda(x)$. Entropy is nonlinear in the predictive mean, so a second-order Taylor expansion of H about $\lambda(x) = \mu(x)$, with $\mathbb{E}[\lambda(x) - \mu(x)] = 0$, leaves the leading bias

$$\mathbb{E}_{\lambda(x)}[H] - H|_{\lambda(x)=\mu(x)} \approx \frac{1}{2} \alpha^2 \sigma^2(x) \frac{\partial^2 H_{\text{marg}}}{\partial \mu^2} \Big|_{x^*}. \quad (80)$$

Since $\text{Cov}(\lambda(x^*), \lambda(x)) = \rho_\lambda \sqrt{\sigma^2(x^*)\sigma^2(x)}$, we have $\alpha^2 \sigma^2(x) = \rho_\lambda^2 \sigma^2(x^*)$, and the $\sigma^2(x)$ cancels:

$$\text{Bias} \approx \frac{\rho_\lambda^2}{2} \sigma^2(x^*) \frac{\partial^2 H_{\text{marg}}}{\partial \mu^2} \Big|_{x^*}. \quad (81)$$

The bias vanishes *only* when $\rho_\lambda = 0$ (x uninformative about x^* — but then the utility contribution is ≈ 0 anyway), or $\sigma^2(x^*) = 0$ (nothing left to learn), or $\partial^2 H_{\text{marg}} / \partial \mu^2 = 0$ (entropy linear in the mean — false for the Poisson-lognormal). Notably $\sigma^2(x) = 0$ is *not* a zero-bias condition: it cancels in (81).

Remark 7.6 (The bias is largest exactly where it matters). $\text{Bias} \propto \rho_\lambda^2 \sigma^2(x^*)$ is maximal when the sampled image x is most correlated with x^* (most relevant to the utility) *and* when x^* is most uncertain (where the acquisition should be steering us) — i.e. largest as $\rho_\lambda \rightarrow 1$. A systematic bias does not shrink with more Monte-Carlo samples: it converges to the *wrong* value and corrupts the optimizer’s direction. Sampling λ is unbiased and its variance falls as $O(1/N)$. Hence: sample λ .

One λ per image is optimal (nested MC). With $g(x, \lambda) = H(R \mid x^*, \lambda, \mathcal{D})$, let $\tau^2 = \text{Var}_x[\mathbb{E}_\lambda g]$ (between-image) and $\mathbb{E}_x[\sigma_x^2]$ (within-image). For a fixed budget of M entropy evaluations, using N_λ latent samples per image gives variance $(N_\lambda \tau^2 + \mathbb{E}_x[\sigma_x^2]) / M$, strictly worse for $N_\lambda > 1$. Spending the budget on *more images*, one λ each, is optimal — which is what the implementation does.

Caveat. The “practical magnitude” analysis in the source (`why_sample_lambda.tex`) is self-flagged “review carefully.” Pushed to log-rate space the curvature carries a gain factor, $\partial^2 H_{\text{marg}} / \partial \mu^2 = A^2 \partial^2 H_{\text{marg}} / \partial g^2$, so $\text{Bias} \propto A^2$; for a small gain (this codebase typically has $A \approx 0.03$) the bias is far below the MC standard error, and the deterministic and sampled U_{DA} nearly coincide. The regime guide $A \ll 1$ negligible / $A \sim 1$ moderate / $A \gg 1$ essential, and the specific numbers, are plausibility arguments, not settled results. The core claim — bias (81) does not vanish with N — is solid; the magnitude estimate is provisional. The *correct* estimator remains the sampled one regardless.

7.6 The entropy landscape over image space

Monotone shape — utility is not pure epistemic uncertainty. The spike-count entropy $H_{\text{marg}}(\mu_g, \sigma_g^2)$ increases monotonically in *both* arguments: with μ_g (more predicted firing) and with σ_g^2 (more GP/epistemic uncertainty). A candidate therefore scores high for a *mixture* of reasons

Function	Input moment convention	Sum index
<code>compute_H</code> ($\mu, \sigma^2, r_{\max}, A, \lambda_0$)	<i>raw</i> λ moments (transformed inside)	$r = 0, \dots, r_{\max} - 1$
<code>nd_utility_new</code> ($\mu_g, \sigma_g^2, r_{\max}$)	<i>log-rate</i> g moments (pre-transformed)	$r = 0, \dots, r_{\max}$

Table 3: The two entropy entry points differ in *both* the moment convention and the truncation range. U_{std} transforms manually then calls `nd_utility_new`; U_{DA} passes raw moments $+(A, \lambda_0)$ to `compute_H`.

— it is predicted to fire a lot *and* it is epistemically uncertain. The utility is thus not a pure epistemic-uncertainty acquisition; predicted firing rate is baked in. This is a deliberate modelling choice: a normalized arc-cosine kernel would force $K(x, x) = 1$, killing the norm channel so that utility tracked only angular/RF alignment (pure epistemic), but held-out `test_r` then drops by $\sim 25\%$ (a fitted PNAS cell, $M = 100$: $0.79 \rightarrow 0.59$) — image norm is genuinely informative for neural encoding, so the production kernel keeps the norm dependence.

The $r_{\max}$ truncation artifact. With a fixed $r_{\max} = 100$ the Laplace sum (65) misses the probability mass above $r_{\max}$ once the Poisson peak climbs past it, and H_{marg} *collapses to* ≈ 0 for $\mu_g \gtrsim \log 100 \approx 4.6$ (and at lower μ_g for large σ_g^2). This is a numerical artifact, not real entropy: the computable region is roughly triangular in (μ_g, σ_g^2) . A cheap pre-check without running the Laplace sum is $z_{\text{safe}} = (\log r_{\max} - \mu_g) / \sqrt{\sigma_g^2} - z_{\text{safe}} > 2$ safe, < 1.5 unreliable, < 0.5 broken. Natural images from the pool sit at $z_{\text{safe}} \gg 10$, always safe; trouble arises only for artificially amplified images. The **adaptive- $r_{\max}$ guard** (§7.7) removes the collapse.

Norm-exploitation divergence (research path). Because the arc-cosine kernel is positively 1-homogeneous ($K(\kappa x, y) = \kappa K(x, y)$ when the bias $\sigma_0 = 0$), scaling $x^* \rightarrow \kappa x^*$ scales $\mu(x^*) \sim \kappa$ and $\sigma^2(x^*) \sim \kappa^2$, so H_{marg} grows while a directionally aligned conditioning image keeps $\rho_\lambda \approx 1$ and $\sigma_{\text{cond}}^2 \approx 0$. Under free-space gradient ascent U_{DA} then grows $\sim \kappa^{1.9}$ (numerically) — the optimizer “diverges” by amplifying image norm rather than finding angularly informative images. This is intrinsic to the homogeneous kernel, *not* a bug; the full statement and the $\sigma_0^2 > 0$ damping (alignment peak strictly at $\kappa = 1$) are Thm. 8.9 in the deep layer. Over the finite natural-image *pool* (production selection over discrete candidates) norms are bounded and this never fires.

Caveat. Utility accuracy ceiling (known limitation). At high firing the two utilities fail in opposite directions through the same $r_{\max} = 100$ truncation: U_{std} can run to $+\infty$ while U_{DA} silently collapses toward 0. The firing -rate guard ($f_{\max}$, default 100) keeps generation in the accurate regime; a dedicated DA-path look-up table was deliberately not built. Present as a limitation of the fixed- $r_{\max}$ path, cured on the standard path by the adaptive guard.

7.7 Code realization and conventions

Entry points. U_{std} runs `standard_utility`, which calls `nd_utility_new`. The research routine `distribution_aware_utility` computes H_{marg} with `compute_H`, and per MC sample forms the conditional moments (§7.4.1) and calls `compute_H` again for $H_{\text{cond}}^{(i)}$. Its marginal and conditional latent moments come from the two `get_gp` moment helpers of §7.4.1.

The adaptive- $r_{\max}$ guard. `compute_adaptive_rmax` sizes the truncation from the moments: with an upper log-rate $\log f_\uparrow = \mu_g + k\sqrt{\sigma_g^2}$ ($k = 3$ by default), $r_{\max} = \lceil f_\uparrow + 5\sqrt{f_\uparrow} + 10 \rceil$ (a 5-SD

Poisson margin), floored at `min_rmax` (default 200). It *raises* rather than silently truncating when the needed $r_{\max}$ would exceed `max_rmax` (default 10000) — an aggressive-truncation guard, since a wrong entropy would otherwise pass unnoticed. This is what keeps H_{marg} from collapsing when μ_g is high; non-adaptive callers pass a fixed $r_{\max}$ (production default 100).

Gradient compatibility. Every production entropy/utility function is differentiable in x^* : the Laplace path uses `torch.where` (not indexed assignment, which breaks autograd), and the Lambert W has a proper custom backward $dW/dy = W/(1+W)$. The caller chooses to track gradients (pass `requires_grad` tensors, enabling gradient-ascent x^* optimization — and exposing the divergence above) or to suppress them (`torch.no_grad()`).

Caveat. A/λ_0 convention split. `compute_H` takes *raw* λ moments and applies $\mu_g = A\mu + \lambda_0$, $\sigma_g^2 = A^2\sigma^2$ internally, whereas `nd_utility_new` and the low-level `_diff_laplace_log_probs` take *already-transformed* g moments (Table 3). Both deliver the same math to the Laplace pipeline, but silently passing raw moments where g moments are expected (or vice versa) is a genuine bug source — keep the mapping explicit at every call.

Caveat. $r_{\max}$ off-by-one. `compute_H` sums $r = 0, \dots, r_{\max} - 1$ while `nd_utility_new` sums $r = 0, \dots, r_{\max}$. Both are finite truncations of (65); the one-bin difference is immaterial at low firing but is a real inconsistency between the two entry points — do not treat $r_{\max}$ as bit-identical across them.

Caveat. Deprecated helper. The current local `utils.py` has *no* `lambda_moments`: that name is a legacy varGP routine, living in the deprecated repo-root `utils.py`. Its role is now filled by `get_gp_marginal_moments` (the marginal moments μ, σ^2) and `get_gp_conditional_moments` (the conditional pair $\mu_{\text{cond}}, \sigma_{\text{cond}}^2$). Do not resurrect it.

Remark 7.7 (Mode restriction and an analysis-only routine). U_{DA} currently requires a model exposing `model(X).covariance_matrix` (the `default_gpy` GPyTorch VariationalGP); the eigenspace `vargp_direct` path is deferred (it would need the augmented-matrix conditioning of (84)). Separately, `compute_H_MC` (sample $g \sim \mathcal{N}$, $r \sim \text{Poisson}(e^g)$, average the clipped Laplace $\log p(r)$) exists *only* for pedagogy/analysis: it is non-differentiable (discrete Poisson sampling) and biased (clipping), and carries an in-code “RAISE TO USER” warning about hard-coded values — it is never a production acquisition.

8 The deep layer: conditioning, subspace, divergence, and numerics

Part III (§7) *defined* the two acquisition utilities and their entropies; this section proves *why* those formulas behave as they do. It supplies the theorems that sit underneath the moment code, the exact predictive-conditioning derivation, the subspace-validity argument, the norm-scaling divergence theorems that explain the utility’s blow-up under unconstrained image optimization, the two kernel-level fixes, and the four-failure-mode numerical audit of the standard utility.

Two objects recur, and their “subtracted term” is *not* the same (the single most load-bearing clash for a pooled reader; the top-level exposition is in Part III):

- the **standard / production** utility $U_{\text{std}}(x^*) = H_{\text{marg}}(x^*) - \mathbb{E}_g[H_{\text{noise}}(x^*)]$ of (69), which the pool-selection loop maximizes (§8.6 audits it); its subtracted term is the expected Poisson *noise* entropy $\mathbb{E}_g[H_{\text{noise}}]$, an average over the point’s *own* latent, no second image involved;

- the **distribution-aware (DA)** research utility $U_{\text{DA}}(x^*) = H_{\text{marg}}(x^*) - \mathbb{E}_{x \sim p(x)}[H_{\text{cond}}(x^*)]$ of (62), whose subtracted term is the entropy *after Gaussian-conditioning* the GP on a hypothetical observation of $\lambda(x_t)$ at a target image x_t .

The two coincide only in the $\rho_\lambda^2 = 1$ deterministic limit, where H_{cond} collapses to a single Poisson entropy (§8.4). The divergence theorems of §8.4 and the remedies of §8.5 concern *gradient ascent on the image x^** (a super-stimulus / image-synthesis setting), which is distinct from the production loop’s selection from a fixed pool.

8.1 GP moments and Gaussian conditioning (Proof III)

This is the moments reference grounding `get_gp_marginal_moments` and `get_gp_conditional_moments` (the current helpers; the deprecated `lambda_moments` no longer exists).

8.1.1 The self-kernel identity

Recall the order-1 arc-cosine kernel of Part I, (2),

$$K(x, x') = \frac{1}{\pi} \mathcal{M} J(\theta), \quad \mathcal{M} = \sqrt{v_x v_{x'}}, \quad \cos \theta = \frac{x^\top C x' + \sigma_0^2}{\mathcal{M}}, \quad J(\theta) = \sin \theta + (\pi - \theta) \cos \theta,$$

with self-magnitude $v_x = x^\top C x + \sigma_0^2$.

Lemma 8.1 (Self-kernel / prior variance). *For every x , $K(x, x) = \|x\|_C^2 + \sigma_0^2 = v_x$, where $\|x\|_C^2 \equiv x^\top C x$.*

Proof. Set $x' = x$: $\cos \theta = v_x / v_x = 1 \Rightarrow \theta = 0$, and $J(0) = \sin 0 + \pi \cos 0 = \pi$. Hence $K(x, x) = \frac{1}{\pi} v_x \pi = v_x = x^\top C x + \sigma_0^2$. $\square$

This restates (5): *the prior variance at any point equals its squared C-norm (plus the bias variance)*. It is the single identity from which the entire norm-scaling divergence of §8.4 grows.

8.1.2 Variational posterior moments

With M inducing points $\{\tilde{z}_j\}$ and variational posterior $q(\tilde{\lambda}) = \mathcal{N}(m, V)$, write the inducing gram $\tilde{K} = K(\tilde{Z}, \tilde{Z})$, the cross-kernel vector $k(x) = [K(x, \tilde{z}_1), \dots, K(x, \tilde{z}_M)]^\top$, the projection $u(x) = \tilde{K}^{-1} k(x)$, and the (training-fixed) correction matrix

$$W = \tilde{K}^{-1}(V - \tilde{K})\tilde{K}^{-1}.$$

The marginal moments are

$$\begin{aligned} \mu(x^*) &= k(x^*)^\top \tilde{K}^{-1} m, \\ \sigma^2(x^*) &= K(x^*, x^*) + k(x^*)^\top W k(x^*) \\ &= \|x^*\|_C^2 + \sigma_0^2 + k(x^*)^\top W k(x^*). \end{aligned} \tag{82}$$

These ground `get_gp_marginal_moments`, which returns `posterior.mean` and `posterior.variance`. The first term of σ^2 is the prior variance (Lemma 8.1); the second is the posterior correction (negative when $V \prec \tilde{K}$, i.e. data shrink variance). Between two points,

$$\Sigma_q(x^*, x_t) = K(x^*, x_t) + k(x^*)^\top W k(x_t), \quad \Sigma_q(x^*, x^*) = \sigma^2(x^*), \quad \Sigma_q(x_t, x_t) = \sigma^2(x_t),$$

and the squared posterior correlation — the Pearson correlation of the joint Gaussian $[\lambda(x^*), \lambda(x_t)]$ under q — is

$$\rho_\lambda^2(x^*, x_t) = \frac{\Sigma_q(x^*, x_t)^2}{\sigma^2(x^*) \sigma^2(x_t)} \in [0, 1]. \tag{83}$$

8.1.3 Conditioning on an observed latent

Observing λ_t at x_t and applying the standard Gaussian conditioning identity to $[\lambda(x^*), \lambda(x_t)]$ gives the conditional moments grounding `get_gp_conditional_moments`:

$$\begin{aligned}\mu_{\text{cond}}(x^*) &= \mu(x^*) + \frac{\Sigma_q(x^*, x_t)}{\sigma^2(x_t)}(\lambda_t - \mu(x_t)), \\ \sigma_{\text{cond}}^2(x^*) &= \sigma^2(x^*) - \frac{\Sigma_q(x^*, x_t)^2}{\sigma^2(x_t)} = \sigma^2(x^*)(1 - \rho_\lambda^2).\end{aligned}\tag{84}$$

Proposition 8.2 (ρ_λ^2 is the fractional variance reduction). *The conditional variance at x^* is the marginal variance scaled by $(1 - \rho_\lambda^2)$; equivalently, conditioning on $\lambda(x_t)$ removes a fraction ρ_λ^2 of the variance at x^* .*

Proof. Substitute the correlation definition (83) into the variance line of (84):

$$\sigma^2(x^*) - \frac{\Sigma_q(x^*, x_t)^2}{\sigma^2(x_t)} = \sigma^2(x^*) \left[1 - \frac{\Sigma_q(x^*, x_t)^2}{\sigma^2(x^*) \sigma^2(x_t)} \right] = \sigma^2(x^*)(1 - \rho_\lambda^2).$$

□

The code (`get_gp_conditional_moments`) reads the *full joint covariance matrix* of $[x_{\text{sample}}; x^*]$ from the model and applies exactly (84) (with a floor $\sigma_{\text{cond}}^2 \geq 10^{-8}$). By the theorem of §8.2 this joint route is the *exact* augmented predictive, not the sparse approximation.

8.1.4 Log-firing transform and the DA utility in three scalars

Under $g = A\lambda + \lambda_0$ and $R \sim \text{Poisson}(e^g)$, the GP moments push to g -space as $\mu_g = A\mu + \lambda_0$, $\sigma_g^2 = A^2\sigma^2$ (marginal) and $\mu_{g,\text{cond}} = A\mu_{\text{cond}} + \lambda_0$, $\sigma_{g,\text{cond}}^2 = A^2\sigma_{\text{cond}}^2 = \sigma_g^2(1 - \rho_\lambda^2)$ (the variance reduction carries through). Hence, writing $H_{\text{marg}}(\mu_g, \sigma_g^2)$ for the marginal Poisson–log-normal entropy (65),

$$U_{\text{DA}}(x^*) = H_{\text{marg}}(\mu_g, \sigma_g^2) - H_{\text{marg}}(\mu_{g,\text{cond}}, \sigma_{g,\text{cond}}^2).$$

In deterministic mode ($\lambda_t = \mu(x_t)$, `sample_lambda=False`) the conditional mean is unchanged, $\mu_{g,\text{cond}} = \mu_g$, and (62) reduces to the deterministic distribution-aware utility (78) of §7.4.2, a function of exactly three scalars:

$$U_{\text{DA}}(x^*) = H_{\text{marg}}(\mu_g, \sigma_g^2) - H_{\text{marg}}(\mu_g, \sigma_g^2(1 - \rho_\lambda^2))\tag{85}$$

the operating point $\mu_g = A\mu(x^*) + \lambda_0$, the marginal variance $\sigma_g^2 = A^2\sigma^2(x^*)$, and the fractional reduction ρ_λ^2 . Conditioning slides $(\mu_g, \sigma_g^2) \rightarrow (\mu_g, \sigma_g^2(1 - \rho_\lambda^2))$: same mean, reduced variance; the utility is the entropy gap. This is the object whose divergence §8.4 analyses.

8.2 The predictive distribution conditioned on a new observation

The DA utility integrates the entropy of $p(\lambda(x^*) \mid \lambda(x), \mathcal{D})$: the predictive over the latent at a query x^* after a *hypothetical* new latent observation at x . We derive it two ways — sparse-approximated and exact/augmented — and prove the two agree in mean.

Define the *prior conditional covariance* (the residual not captured by the inducing points)

$$c(x, x') \equiv K(x, x') - u(x)^\top \widetilde{K} u(x') = K(x, x') - k(x)^\top \widetilde{K}^{-1} k(x'),$$

whose diagonal is the residual / aleatoric variance $\sigma_r^2(x) \equiv c(x, x) = K(x, x) - k(x)^\top \widetilde{K}^{-1} k(x) \geq 0$.

8.2.1 The rank-1 posterior update

Lemma 8.3 (Rank-1 update). *Let the new point x have projection $u = u(x)$ and residual variance $\sigma_r^2 = \sigma_r^2(x)$, so the conditional likelihood is $p(\lambda(x) \mid \tilde{\lambda}) = \mathcal{N}(u^\top \tilde{\lambda}, \sigma_r^2)$. Conditioning the variational posterior $\mathcal{N}(m, V)$ on the observed $\lambda(x)$ yields $\mathcal{N}(m', V')$ with*

$$m' = m + \frac{Vu}{\sigma_r^2 + u^\top V u}(\lambda(x) - u^\top m), \quad V' = V - \frac{Vu u^\top V}{\sigma_r^2 + u^\top V u}. \quad (86)$$

Proof. Under the variational approximation the joint of $(\lambda(x), \tilde{\lambda})$ is Gaussian:

$$\begin{bmatrix} \lambda(x) \\ \tilde{\lambda} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} u^\top m \\ m \end{bmatrix}, \begin{bmatrix} \sigma_r^2 + u^\top V u & u^\top V \\ V u & V \end{bmatrix}\right).$$

The Gaussian conditioning identity on $\tilde{\lambda}$ given $\lambda(x)$, with scalar lower-right-to-upper-left Schur term $\Sigma_{11} = \sigma_r^2 + u^\top V u$ and cross-block Vu , gives (86) directly. $\square$

8.2.2 Sparse vs. augmented predictive

Let $u^* = \tilde{K}^{-1}k(x^*)$ and $\sigma_r^2(x^*) = K(x^*, x^*) - k(x^*)^\top \tilde{K}^{-1}k(x^*)$.

Sparse route. Assuming conditional independence $p(\lambda(x^*) \mid \lambda(x), \tilde{\lambda}) \approx p(\lambda(x^*) \mid \tilde{\lambda})$ and integrating the conditional prior against the updated posterior of Lemma 8.3,

$$\mu_{\text{pred}} = u^{*\top} m', \quad \sigma_{\text{pred}}^2 = \sigma_r^2(x^*) + u^{*\top} V' u^* = K(x^*, x^*) - u^{*\top} (\tilde{K} - V') u^*.$$

Augmented (exact) route. Keep the direct $\lambda(x)$ – $\lambda(x^*)$ correlation. With augmented latent $\tilde{\lambda}_{\text{aug}} = [\lambda(x); \tilde{\lambda}]$ and augmented gram / cross-kernel

$$\tilde{K}_x = \begin{bmatrix} K(x, x) & k(x)^\top \\ k(x) & \tilde{K} \end{bmatrix}, \quad k_{\text{aug}}(x^*) = \begin{bmatrix} K(x, x^*) \\ k(x^*) \end{bmatrix},$$

the non-approximated conditional prior is $p(\lambda(x^*) \mid \tilde{\lambda}_{\text{aug}}) = \mathcal{N}(u_{\text{aug}}^\top \tilde{\lambda}_{\text{aug}}, S^*)$, $u_{\text{aug}} = \tilde{K}_x^{-1} k_{\text{aug}}(x^*)$, $S^* = K(x^*, x^*) - u_{\text{aug}}^\top \tilde{K}_x u_{\text{aug}}$. Since $\lambda(x)$ is *observed* (variance 0) while $\tilde{\lambda}$ follows the updated posterior $\mathcal{N}(m', V')$, the augmented posterior covariance is $V'' = \begin{bmatrix} 0 & 0^\top \\ 0 & V' \end{bmatrix}$, and

$$\mu_{\text{pred}} = u_{\text{aug}}^\top [\lambda(x); m'], \quad \sigma_{\text{pred}}^2 = S^* + u_{\text{aug}}^\top V'' u_{\text{aug}} = K(x^*, x^*) - u_{\text{aug}}^\top (\tilde{K}_x - V'') u_{\text{aug}}.$$

Remark 8.4 (Augmented route fixes the sparse self-variance artifact). When $x = x^*$, u_{aug} aligns with the first column of $\tilde{K}_x$ and, because the (1, 1) block of V'' is 0, the predictive variance is forced to 0. A self-consistent predictive *must* have zero variance at an already-observed point; the sparse route leaves a spurious nonzero self-variance there. This is the same pathology as the hand-assembled cross-covariance that can violate Cauchy–Schwarz (Part II §4.7) — the shipped code sidesteps both by reading the model’s full joint covariance.

8.2.3 Equivalence of predictive means

Theorem 8.5 (Equivalence of predictive means). *The augmented-integration predictive mean equals the mean obtained by standard Gaussian conditioning on the joint predictive $[\lambda(x), \lambda(x^*)]$ — exactly, with no cross-covariance approximation.*

Proof. Method 1 (joint conditioning). The joint moments given $\mathcal{D}$ are $\mu_x = u^\top m$, $\mu_{x^*} = u^{*\top} m$,

$$\Sigma_{11} = \sigma_r^2 + u^\top V u, \quad \Sigma_{12} = \underbrace{(K(x, x^*) - u^\top \widetilde{K} u^*)}_{=c} + u^{*\top} V u,$$

where the residual cross term is exactly $c = c(x, x^*)$. The Gaussian conditioning identity gives the target

$$\mathbb{E}[\lambda(x^*) \mid \lambda(x), \mathcal{D}] = \mu_{x^*} + \frac{\Sigma_{12}}{\Sigma_{11}}(\lambda(x) - \mu_x) = u^{*\top} m + \frac{\Sigma_{12}}{\Sigma_{11}}(\lambda(x) - u^\top m). \quad (87)$$

Method 2 (augmented integration). Block-invert $\widetilde{K}_x$. Writing the residual-correlation coefficient $\vartheta \equiv c/\sigma_r^2$ (the source’s predictive α , renamed here to avoid the RF locality mask α_i),

$$u_{\text{aug}} = \begin{bmatrix} \vartheta \\ u^* - \vartheta u \end{bmatrix}, \quad \vartheta = \frac{K(x, x^*) - u^\top \widetilde{K} u^*}{\sigma_r^2} = \frac{c}{\sigma_r^2}.$$

Then, with m' from (86),

$$\mu_{\text{aug}} = u_{\text{aug}}^\top [\lambda(x); m'] = \vartheta \lambda(x) + (u^{*\top} - \vartheta u^\top) m' = \vartheta \lambda(x) + (u^{*\top} - \vartheta u^\top) \left[m + \frac{V u}{\Sigma_{11}} (\lambda(x) - u^\top m) \right].$$

Group by the prior mean $u^{*\top} m$ and the innovation $(\lambda(x) - u^\top m)$, adding and subtracting $\vartheta u^\top m$:

$$\mu_{\text{aug}} = u^{*\top} m + \underbrace{\left[\vartheta + \frac{u^{*\top} V u - \vartheta u^\top V u}{\Sigma_{11}} \right]}_{\mathcal{C}} (\lambda(x) - u^\top m).$$

Put $\mathcal{C}$ over the common denominator $\Sigma_{11} = \sigma_r^2 + u^\top V u$ and use $\vartheta \sigma_r^2 = c$:

$$\mathcal{C} = \frac{\vartheta \sigma_r^2 + \vartheta u^\top V u + u^{*\top} V u - \vartheta u^\top V u}{\Sigma_{11}} = \frac{\vartheta \sigma_r^2 + u^{*\top} V u}{\Sigma_{11}} = \frac{c + u^{*\top} V u}{\Sigma_{11}} = \frac{\Sigma_{12}}{\Sigma_{11}}.$$

Hence $\mu_{\text{aug}} = u^{*\top} m + (\Sigma_{12}/\Sigma_{11})(\lambda(x) - u^\top m)$, identical to (87). $\square$

Remark 8.6. Theorem 8.5 certifies that the production conditional-moment code (§8.1, `get_gp_conditional_moments`), which conditions on the full joint covariance, computes the *exact* augmented predictive mean — it does not incur the sparse cross-covariance error. Setting the residual cross term $c = 0$ would enforce the sparse factorization

$$p(\lambda^*, \lambda \mid \tilde{\lambda}) \approx p(\lambda^* \mid \tilde{\lambda}) p(\lambda \mid \tilde{\lambda});$$

the inducing (epistemic) cross-covariance $u^{*\top} V u$ can still be nonzero, because changing belief about an inducing point moves both $\lambda(x^*)$ and $\lambda(x)$.

8.3 Subspace theory

Why the utility may be optimized in a low-dimensional subspace of image space (11664 pixels full; ~ 900 – 2356 in the RF-masked region) instead of full pixel space. Setting: optimize x^* to maximize $U_{\text{DA}}(x^*)$ by gradient ascent, constrained to $x_{\text{rf}} = \bar{x} + (\text{basis}) \cdot \zeta$ over the subspace coordinate ζ ; the offset $\bar{x}$ is the RF-pixel dataset mean, so $\zeta = 0$ maps to the mean image.

8.3.1 The kernel touches x only through the C -eigenspace

Let $C = \sum_i \gamma_i e_i e_i^\top$ be the eigendecomposition of the structured PSD matrix of Part I (§3.3), with $\gamma_1 \geq \dots \geq \gamma_d \geq 0$ and orthonormal C -eigenvectors e_i .

Proposition 8.7 (*C -eigenspace validity*). *Every kernel evaluation depends on x only through the quadratic forms $x^\top C x = \sum_i \gamma_i (e_i^\top x)^2$ and $x^\top C x'$; and $\nabla_x K$ lies in the column space of C , with its component along e_i scaled by γ_i . Consequently, gradient ascent already concentrates movement along the top C -eigenvectors, and truncating to the leading C -eigen-subspace is (at full rank) an exact reparameterization and (at reduced rank) drops only near-zero-gradient directions.*

Proof. The kernel enters x solely via $x^\top C x + \sigma_0^2 = v_x$ and $x^\top C x' + \sigma_0^2$ (through $\mathcal{M}$ and $\cos \theta$); in the C -eigenbasis $x^\top C x = \sum_i \gamma_i (e_i^\top x)^2$, so a direction e_i with $\gamma_i \approx 0$ contributes ≈ 0 to *every* kernel value. The input gradient of the arc-cosine kernel (Part I §3.5, (10)) is

$$\nabla_x K(x, x') = \frac{1}{\pi} \left[(\pi - \theta) C x' + \sin \theta \sqrt{v_{x'}/v_x} C x \right],$$

every term of which is $C \cdot (\text{vector})$; projecting onto e_i multiplies by γ_i . Thus movement along small- γ_i directions is intrinsically suppressed. Removing those directions (a) does not change the optimum (they carried near-zero gradient) and (b) removes flat directions, giving a better-conditioned, lower-effective-dimension problem. $\square$

So the C -eigenspace projection is a *convergence aid* $\not\subset$ *exact reparameterization*, not a naturalness constraint (analogous to the eigenvalue threshold on $\bar{K}$ in the direct-VGP path of Part II §5.1).

8.3.2 Three bases and the Szegő/Fourier equivalence

All three write $x_{\text{rf}} = \bar{x} + (\text{basis}) \cdot \zeta$ and share one optimizer/diagnostics; they differ only in the basis.

- **PCA.** Columns $\Phi_K = [\phi_1, \dots, \phi_K] = \text{top-}K$ eigenvectors of the data covariance $\Sigma_{\text{data}} = \frac{1}{N} X_c^\top X_c$ (eigenvalues $\nu_1 \geq \nu_2 \geq \dots$). The offset $\bar{x}$ is *mathematically intrinsic* (centering before SVD); truncation is a *genuine data-driven constraint* (excludes low-data-variance, “unnatural” directions). At variance threshold 0.80: $K = 197$ of 2356 RF pixels. Gradient projects as $\nabla_\zeta U_{\text{DA}} = \Phi_K^\top \nabla_x U_{\text{DA}}$. PCA captures *second-order statistics only* (mean + covariance); it is blind to phase structure (edges/contours), sparsity, and non-Gaussian marginals.
- **C -eigenspace.** Columns $E_K = [e_1, \dots, e_K] = \text{top-}K$ C -eigenvectors. The offset is *not* intrinsic (the eigendecomposition passes through the origin) — see the caveat below. Truncation is the reparameterization of Proposition 8.7. The spectrum is extremely skewed (top eigenvalue $\sim 44\%$ of the total): at relative threshold 10^{-3} , $K = 28$ of 2356 captures 99.7% of the eigenvalue mass. (Float32 noise floor: eigenvalues below $\sim 10^{-7} \gamma_1$ are numerical noise; thresholds below $\sim 10^{-6}$ pull in garbage eigenvectors.)
- **Combined.** Project C into the PCA subspace and re-diagonalize: $C_{\text{pca}} = \Phi_K^\top C \Phi_K = Q \Gamma_Q Q^\top$, keep the columns Q above the eigen-threshold, basis = $\Phi_K Q$. Directions that are *both* natural (PCA) and kernel-visible (C -eigen). At $(0.80, 10^{-3})$: $2356 \rightarrow K_{\text{pca}} = 197 \rightarrow K_{\text{comb}} = 28$, achieving near-identical utility to PCA with $\sim 7\times$ fewer dimensions.

Proposition 8.8 (PCA $\approx$ Fourier under stationarity (Szegő)). *If the image process is second-order stationary on a regular grid, its covariance is (block-)Toeplitz, $\Sigma[i, j] = f(|i - j|)$. By Szegő’s theorem, as $d \rightarrow \infty$ the eigenvectors converge to Fourier modes and the eigenvalues to the power spectral*

density $S(\omega) = \sum_k f(k)e^{-i\omega k}$. Natural images have $S(\omega) \sim 1/|\omega|^2$, so Σ_{data} 's eigenvectors approach the Fourier basis and PCA-truncation approximates keeping the K lowest-frequency Fourier modes.

The equivalence is weakened by the irregular masked region, the non-stationarity the α mask injects (RF-center pixels have higher variance, breaking Toeplitz structure), finite $N \approx 10^4$ in $d \approx 900$, and residual non-stationarity of natural images; at $\sim 30 \times 30$ patches it is reasonable.

8.3.3 Subspace $\neq$ naturalness constraint

All three are *subspace* constraints, not *distributional* ones. None guarantees x^* is a plausible sample of $p(x)$: an image with the right power spectrum but wrong phase (correlated noise) satisfies all three. A true distributional constraint needs higher-order structure (phases, edges, sparsity) — i.e. a generative model, which is the motivation for the guided-diffusion synthesis of Part IV. The subspace approach is a useful *diagnostic* (does restricting to the right power spectrum change utility? does the GP utility operate primarily at second order?), not a full fix.

Caveat. The C -eigenspace offset is not mathematically motivated. Unlike PCA, where the mean offset is intrinsic to centering, the C -eigendecomposition passes through the origin, so setting $\bar{x} = \text{mean}$ has no principled basis — it is forced by *limiter precision*: the non-capturable eigenvectors collapse to 0 instead of to the mean, and the code compensates by setting offset = mean. This changes the utility landscape (the kernel sees $C(\bar{x} + E_K\zeta)$, not $C(E_K\zeta)$) and should be investigated. It is an acknowledged modeling wrinkle, not a resolved result.

8.4 Divergence theorems (Proof I): norm scaling and the utility blow-up

The arc-cosine kernel is positively homogeneous; scaling an image along a fixed direction inflates the posterior variance as the square of the scale while preserving the conditioning benefit, so unconstrained gradient ascent on the DA utility diverges. The three source proof documents each state this phenomenon independently; it is proved once here and reused by §8.5. Throughout, $x^* = \kappa x_t$ with scale $\kappa > 0$, and $q \equiv x_t^\top C x_t$.

Theorem 8.9 (Cross-kernel proportionality, $\sigma_0^2 = 0$). *Let $\sigma_0^2 = 0$ and $x^* = \kappa x_t$, $\kappa > 0$. Then $K(x^*, z) = \kappa K(x_t, z)$ for every z , and hence $k(x^*) = \kappa k(x_t)$.*

Proof. With $\sigma_0^2 = 0$, $v_{x^*} = \kappa^2 q$ and $v_{x_t} = q$. For any z with $v_z = z^\top C z$,

$$\cos \theta(x^*, z) = \frac{(\kappa x_t)^\top C z}{\sqrt{\kappa^2 q \cdot v_z}} = \frac{\kappa x_t^\top C z}{\kappa \sqrt{q v_z}} = \frac{x_t^\top C z}{\sqrt{q v_z}} = \cos \theta(x_t, z),$$

so $\theta(x^*, z) = \theta(x_t, z)$ and $J(\theta(x^*, z)) = J(\theta(x_t, z))$. Then $K(x^*, z) = \frac{1}{\pi} \sqrt{\kappa^2 q v_z} J(\theta) = \kappa \cdot \frac{1}{\pi} \sqrt{q v_z} J(\theta) = \kappa K(x_t, z)$. Applying this to each inducing point $z = \tilde{z}_j$ gives $k(x^*) = \kappa k(x_t)$. $\square$

Corollary 8.10 (Perfect posterior correlation). *Under Theorem 8.9, $\rho_\lambda^2(x^*, x_t) = 1$.*

Proof. From $k(x^*) = \kappa k(x_t)$ and (with $x^* \parallel x_t$, $\theta = 0$, $J(0) = \pi$) $K(x^*, x_t) = \kappa K(x_t, x_t) = \kappa v_{x_t}$, substitute into Σ_q with $W = \tilde{K}^{-1}(V - \tilde{K})\tilde{K}^{-1}$:

$$\Sigma_q(x^*, x_t) = \kappa v_{x_t} + \kappa k(x_t)^\top W k(x_t) = \kappa \Sigma_q(x_t, x_t), \quad \Sigma_q(x^*, x^*) = \kappa^2 \Sigma_q(x_t, x_t),$$

so $\rho_\lambda^2 = [\kappa \Sigma_q(x_t, x_t)]^2 / [\kappa^2 \Sigma_q(x_t, x_t) \cdot \Sigma_q(x_t, x_t)] = 1$. $\square$

Corollary 8.11 (Moment scaling). *Under Theorem 8.9, $\mu(\kappa x_t) = \kappa \mu(x_t)$ and $\sigma^2(\kappa x_t) = \kappa^2 \sigma^2(x_t)$.*

Proof. Substitute $k(x^*) = \kappa k(x_t)$ into $\mu = k^\top \widetilde{K}^{-1} m$ (linear in k) and $\sigma^2 = K(x^*, x^*) + k^\top W k$ (quadratic in k , with $K(x^*, x^*) = \kappa^2 v_{x_t}$). $\square$

By Corollary 8.10, $\rho_\lambda^2 = 1 \Rightarrow \sigma_{\text{cond}}^2(x^*) = 0$: conditioning removes *all* the variance. Yet by Corollary 8.11 the marginal moments still grow ($\mu_g \propto \kappa$, $\sigma_g^2 \propto \kappa^2 \Rightarrow H_{\text{marg}} \rightarrow \infty$), so U_{DA} still diverges. In the exact parallel case ($\theta = 0, \sigma_0^2 = 0$), $\mu_{\text{cond}} = \mu$, $\sigma_{\text{cond}}^2 = 0$, and H_{cond} collapses to a single Poisson entropy $H_{\text{Poisson}}(e^{\mu_g})$, $\mu_g = A\mu(x^*) + \lambda_0$; then (85) becomes $U_{\text{DA}}(\kappa x_t) = H_{\text{marg}}(\mu_g, \sigma_g^2) - H_{\text{Poisson}}(e^{\mu_g})$ with $\mu_g = A\kappa\mu(x_t) + \lambda_0$, $\sigma_g^2 = A^2\kappa^2\sigma^2(x_t)$.

8.4.1 Growth rate of the diverging utility

Proposition 8.12 (Gaussian observation model: $\log \kappa$ growth). *If observations were Gaussian, $r(x) \sim \mathcal{N}(\lambda(x), \tau^2)$ with fixed τ^2 , then for $\rho_\lambda^2 = 1$, $\sigma^2 = \kappa^2\sigma^2(x_t)$, the DA utility grows as $\log \kappa$.*

Proof. $H_{\text{marg}} = \frac{1}{2} \log(2\pi e(\sigma^2 + \tau^2))$ and, since $\sigma_{\text{cond}}^2 = 0$, $H_{\text{cond}} = \frac{1}{2} \log(2\pi e \tau^2)$, so $U_{\text{DA}} = \frac{1}{2} \log(1 + \sigma^2/\tau^2)$. With $\sigma^2 = \kappa^2\sigma^2(x_t)$ and large κ , $U_{\text{DA}} \approx \frac{1}{2} \log(\kappa^2\sigma^2(x_t)/\tau^2) = \log \kappa + \text{const}$. $\square$

Unbounded but slow: $dU_{\text{DA}}/d\kappa \sim 1/\kappa$, so ascent slows but never stops. Norm scaling is *not* unique to Poisson — the distinction is the *rate*.

Proposition 8.13 (Poisson-GP: κ^2 heuristic bound). *For the Poisson-GP case, under the heuristic $H_{\text{marg}} \geq H_{\text{Poisson}}(\mathbb{E}[f])$ with $\mathbb{E}[f] = e^{\mu_g + \sigma_g^2/2}$ and the large-rate Poisson entropy $H_{\text{Poisson}}(\lambda) \approx \frac{1}{2} \log(2\pi e \lambda)$,*

$$H_{\text{marg}} \geq \frac{1}{2} \log(2\pi e) + \frac{1}{2}(\mu_g + \sigma_g^2/2), \quad \text{dominant term } \frac{1}{4}\sigma_g^2 = \frac{1}{4}A^2\kappa^2\sigma^2(x_t).$$

Since $H_{\text{cond}} = H_{\text{Poisson}}(e^{\mu_g})$ depends only on μ_g (grows as κ , not κ^2),

$$U_{\text{DA}} = H_{\text{marg}} - H_{\text{cond}} \gtrsim \frac{1}{4}\sigma_g^2 \propto \kappa^2$$

(faster than the Gaussian $\log \kappa$).

Caveat. The bound $H_{\text{marg}} \geq H_{\text{Poisson}}(\mathbb{E}[f])$ is *plausible but not proved*. A Poisson mixture over log-normal rates is overdispersed (“more spread”), but the Poisson is *not* the max-entropy law on $\mathbb{N}_0$ under a mean constraint alone (the geometric is); it is max-entropy only under $\text{Var} = \text{mean}$, so the naive argument does not directly apply. Numerically (`verify_conclusions.py`, test C4), fitting $U_{\text{DA}} \sim \kappa^\alpha$ over the Laplace-valid range $\kappa \in \{2, 5, 10\}$ (all $\sum p(r) > 0.95$) gives $\alpha \approx 1.9$, consistent with κ^2 . The Laplace approximation truncates at r_{max} and breaks once σ_g^2 puts mass above r_{max} ; the safety score $z_{\text{safe}} = (\log r_{\text{max}} - \mu_g)/\sqrt{\sigma_g^2}$ falls below 2 (unreliable) at $\kappa \approx 10$ in-model, limiting the verifiable range.

8.4.2 The $\sigma_0^2 > 0$ damping and the alignment peak at $\kappa = 1$

With $\sigma_0^2 > 0$ the results hold only approximately:

$$v_{\kappa x_t} = \kappa^2 q + \sigma_0^2 \neq \kappa^2 v_{x_t}, \quad \cos \theta(\kappa x_t, z) = \frac{\kappa x_t^\top C z + \sigma_0^2}{\sqrt{(\kappa^2 q + \sigma_0^2) v_z}} \neq \cos \theta(x_t, z).$$

The fractional error $|K(\kappa x, z) - \kappa K(x, z)|/|\kappa K(x, z)|$ converges to a *constant* $O(\sigma_0^2/v_{x_t})$ as $\kappa \rightarrow \infty$ (not to zero: σ_0^2 shifts the angle by an amount $\propto \sigma_0^2/v_{x_t}$, independent of κ). In a trained model ($v_{x_t} = q + \sigma_0^2 \approx 669$, $\sigma_0^2 \approx 0.98$, so $\sigma_0^2/v_{x_t} \approx 0.0015$): the measured fractional error in $K(\kappa x_t, \tilde{z}_j) \rightarrow \approx$

Table 4: Trained model (seed 42, cell 8, $M = 50$, $N = 50$; $A = 0.0265$, $\lambda_0 = -1.686$). The small gain A compresses the raw GP mean into the useful band, so the DA loophole ($U_{\text{DA}} = 0.800$) is real, not a numerical artifact; the standard-optimized point overflows float32.

Image	μ	σ^2	μ_g	σ_g^2	$\theta(\text{rad})$	H_{marg}	U_{DA}
x_t (natural target)	7.86	70.9	-1.48	0.050	0	0.593	0.010
x_{DA}^*	101	3240	0.99	2.28	0.114	2.836	0.800
x_{Std}^*	68497	2.85×10^8	1816	2.0×10^5	0.866	~ 0	∞

0.6% for large κ , $\rho_\lambda^2 \rightarrow \approx 0.987$, and the residual variance ratio $\approx 1.3\%$ (conditioning stays very effective, not perfect). Crucially, the cosine similarity

$$\cos \theta(\kappa) = \frac{\kappa q + \sigma_0^2}{\sqrt{(\kappa^2 q + \sigma_0^2)(q + \sigma_0^2)}}$$

is *strictly maximized at* $\kappa = 1$ when $\sigma_0^2 > 0$ (and $\equiv 1$ for all κ when $\sigma_0^2 = 0$). So σ_0^2 breaks scale invariance: the kernel *wants* to match the target in pattern *and* intensity — but the magnitude still scales as $K(x^*, x^*) = \kappa^2 q + \sigma_0^2 = O(\kappa^2)$, so the bounded correlation reward cannot compete with the unbounded variance reward, and ascent still runs $\kappa \rightarrow \infty$.

Remark 8.14 (Root cause and remedy). The mechanism is a decoupling of norm from direction: $K(\kappa x, y) = \kappa K(x, y)$ at $\sigma_0^2 = 0$ lets ascent raise variance (hence entropy and utility) via the norm degree of freedom without sacrificing directional alignment. Any practical gradient-based image optimization with this kernel needs a norm constraint or a normalized kernel (§8.5). The blow-up is strongly angle-dependent (test C8): effective at small angles ($\theta < 0.2$, strong conditioning), negligible at large angles ($\theta > 0.6$).

Table 4 shows the trained-model evidence: the DA-optimized image finds a *genuine* loophole ($U_{\text{DA}} = 0.800$ at a small angle $\theta = 0.114$, variance ratio $0.060 \Rightarrow \rho_\lambda^2 \approx 0.94$), while the standard-optimized image diverges to $U_{\text{std}} = \infty$ by numerical overflow at $\mu_g = 1816$ — two different phenomena (§8.6 separates them).

8.5 Kernel solutions (Proof II)

Two kernel-level remedies bound the variance term that §8.4 showed to be the culprit. Decompose the utility (Poisson link, $f = e^\lambda$) schematically as

$$U_{\text{std}}(x^*) \approx \underbrace{\frac{1}{2}\sigma^2(x^*)}_{\text{variance term, } O(\kappa^2)} + \underbrace{I(\rho_\lambda^2)}_{\text{correlation term, } \leq \text{const}},$$

where the correlation term is bounded (maximized at $\kappa = 1$ once $\sigma_0^2 > 0$) but the variance term is unbounded; to fix the divergence, bound the variance term.

8.5.1 Normalized arc-cosine kernel (project onto the unit sphere)

Proposition 8.15 (Scale-invariant normalized kernel). *Define $\bar{K}(x, x') = K(x, x') / \sqrt{K(x, x) K(x', x')}$. For the order-1 arc-cosine kernel, $\bar{K}(x, x') = J(\theta) / \pi$, and for any $\kappa > 0$, $\bar{K}(\kappa x, \kappa x) = 1 = \bar{K}(x, x)$ — the prior variance is strictly constant, independent of $\|x\|$.*

Proof. By Lemma 8.1 $K(x, x) = v_x$, so $\bar{K}(x, x') = [\frac{1}{\pi} \sqrt{v_x v_{x'}} J(\theta)] / \sqrt{v_x v_{x'}} = J(\theta) / \pi$, which depends only on θ . In particular $\bar{K}(x, x) = J(0) / \pi = 1$; and $\bar{K}(\kappa x, \kappa x) = K(\kappa x, \kappa x) / \sqrt{K(\kappa x, \kappa x)^2} = 1$. $\square$

With $\bar{K}(x, x) = 1$ and cross terms $\bar{K}(x, \tilde{z}_j) = J(\theta_j) / \pi \in [0, 1]$ bounded, the whole posterior-correction term is bounded, so $\bar{\sigma}^2(x)$ is bounded regardless of $\|x\|_C$ and U_{DA} cannot diverge by scaling: the only way up is to improve ρ_λ^2 (small angle to x_t ; with $\sigma_0^2 > 0$ retained inside θ , minimized only when $x \rightarrow x_t$ in both direction and magnitude).

Caveat. The normalized kernel discards all norm information, but image norm carries genuine neural-encoding signal (neurons respond differently to high/low contrast). Empirically, training with $\bar{K}$ drops test correlation by $\sim 25\%$ — the norm encodes real structure the GP needs. A fix for the image-synthesis divergence is not a free improvement to the production model.

8.5.2 Biologically plausible saturation kernel (arc-sin)

The arc-cosine kernel is the covariance of infinite ReLU networks, which do *not* saturate; real neurons have refractory periods and maximum firing rates. The arc-sin (probit-activation) kernel models saturation:

$$K_{\text{sat}}(x, x') = \sigma_f^2 \cdot \frac{2}{\pi} \arcsin \left(\frac{x^\top C x' + \sigma_0^2}{\sqrt{(1 + x^\top C x + \sigma_0^2)(1 + x'^\top C x' + \sigma_0^2)}} \right).$$

Proposition 8.16 (Bounded self-variance). $\lim_{\|x\| \rightarrow \infty} K_{\text{sat}}(x, x) = \sigma_f^2 \cdot \frac{2}{\pi} \arcsin(1) = \sigma_f^2$.

Proof. As $\|x\| \rightarrow \infty$ the ratio inside arcsin tends to $(x^\top C x) / (1 + x^\top C x) \rightarrow 1$, and $\arcsin(1) = \pi/2$, so $K_{\text{sat}}(x, x) \rightarrow \sigma_f^2 \cdot (2/\pi)(\pi/2) = \sigma_f^2$. $\square$

Unlike $K(x, x) \propto \|x\|^2$, the variance saturates to the constant σ_f^2 . This fixes the divergence *without* normalizing the kernel: H_{marg} cannot grow indefinitely with norm, so to raise utility the optimizer must lower H_{cond} by finding x^* highly correlated with x_t ; “super-stimuli” of infinite norm yield diminishing returns, embedding the physical constraint that contrast beyond a point yields no more signal variance.

8.6 Numerical analysis of the standard utility

This subsection is entirely about the standard/production utility (69), $U_{\text{std}} = H_{\text{marg}} - \mathbb{E}_g[H_{\text{noise}}] = I(R; g \mid x^*, \mathcal{D})$, computed by `nd_utility_new`. (Formulas verified against the live engine; the `utility_numerical_analysis.tex` line citations from the 2026-05-22 audit have since drifted, but every formula matches the code.) The spike-count index r below is the model’s spike count (Part I’s y); the count random variable is $R \sim \text{Poisson}(e^g)$.

8.6.1 The closed-form decomposition

With $\lambda(x^*) \mid \mathcal{D} \sim \mathcal{N}(\mu, \sigma^2)$, $g = A\lambda + \lambda_0$, so $g \mid \mathcal{D} \sim \mathcal{N}(\mu_g, \sigma_g^2)$, and $R \mid g \sim \text{Poisson}(e^g)$:

$$H_{\text{marg}} = - \sum_{r=0}^{\infty} p(r) \log p(r), \quad p(r) = \int \text{Poisson}(r; e^g) \mathcal{N}(g; \mu_g, \sigma_g^2) dg,$$

the marginal (Poisson–log-normal) entropy of (65) (no closed form; Laplace-approximated below). The subtracted expected-noise entropy *is* closed-form via the log-normal MGF ($\mathbb{E}[e^g] = e^{\mu_g + \sigma_g^2/2}$, $\mathbb{E}[ge^g] = e^{\mu_g + \sigma_g^2/2}(\mu_g + \sigma_g^2)$):

$$\mathbb{E}_g[H_{\text{noise}}] = \underbrace{-e^{\mu_g + \sigma_g^2/2}(\mu_g + \sigma_g^2 - 1)}_{\text{exact (MGF), closed form}} + \underbrace{\sum_{r=0}^{\infty} p(r) \log r!}_{\text{needs } p(r), \text{ Laplace}}, \quad (88)$$

so $U_{\text{std}} \approx -\sum_{r=0}^{r_{\text{max}}} p(r) \log p(r) - (-e^{\mu_g + \sigma_g^2/2}(\mu_g + \sigma_g^2 - 1) + \sum_{r=0}^{r_{\text{max}}} p(r) \log r!)$; both sums truncate at the same r_{max} . Ground truth: `nd_utility_new` computes

```
E_H_noise = -torch.exp(mu_g + 0.5*sigma2_g)*(mu_g + sigma2_g - 1) + p_times_logr_sum
```

8.6.2 Laplace approximation and the log-space Lambert-W fix

The saddle point of $p(r)$ is at the mode $\bar{g}_r$ of $\text{Poisson}(r; e^g) \mathcal{N}(g; \mu_g, \sigma_g^2)$, solving $r = e^{\bar{g}_r} + (\bar{g}_r - \mu_g)/\sigma_g^2$:

$$\bar{g}_r(r) = r\sigma_g^2 + \mu_g - W_0(\sigma_g^2 e^{r\sigma_g^2 + \mu_g}), \quad (89)$$

W_0 the principal Lambert branch. **The historic bug** (legacy `argmax_g_old`) formed $\sigma_g^2 e^{r\sigma_g^2 + \mu_g}$ literally and, in float32, *overflowed* whenever $r\sigma_g^2 + \mu_g \gtrsim 88$ ($e^{88} \sim 10^{38}$); masking the overflowing r let the truncated $\sum p(r)$ exceed 1 by large factors. This is the historic “exp(μ) diverges.” **The fix** (current `_LambertWLogFunction + _diff_argmax_g`) never forms e^y : set $y = \log \sigma_g^2 + r\sigma_g^2 + \mu_g$ (finite for finite inputs) and solve

$$w + \log w = y \iff w = W_0(e^y)$$

by a fixed 10 Newton iterations directly on the log-form (custom autograd; backward uses $dW/dy = W/(1+W)$, in which e^y cancels), then $\bar{g}_r(r) = r\sigma_g^2 + \mu_g - w$. Since $w \approx y - \log y$ for large y , $\bar{g}_r(r) \approx -\log \sigma_g^2 + \log y \sim \log r$, so $e^{\bar{g}_r(r)} \approx r$ (exactly right: the Poisson likelihood is maximized when rate = count) and stays in float32 range for any r — so $\exp(\bar{g}_r)$ is no longer an overflow risk post-fix.

8.6.3 Four independent residual failure modes

Distinct thresholds and remedies (they coincide only because all worsen at large μ_g/σ_g^2); (a) and (b) are the pair older docs conflate.

(a) Laplace approximation error in $\log p(r)$. A local quadratic model of the integrand; inaccurate when it departs from a quadratic-log shape. Two regimes: $\sigma_g^2 \rightarrow 0$ (prior degenerates, saddle singular — the code falls back to *exact* Poisson $\log p = \mu_g r - e^{\mu_g} - \log r!$ at $\sigma_g^2 < 10^{-6}$); σ_g^2 large (flat prior; the asymmetric Poisson factor skews the integrand). *Independent of r_{max}* — each $p(r)$ has its own bias, observable as $\sum p(r) \neq 1$ even with no missing mass (up to 1.015; that is Laplace error, not truncation). Measured (vs `scipy.quad`): error < 1% for $\sigma_g^2 \lesssim 1$ at every μ_g , growing to 5–20% at the peak by $\sigma_g^2 \sim 50$ (worst where the predictive peak slides onto $r = 0$). The driver is σ_g^2 , not μ_g ; float32 vs float64 shifts $\log p(r)$ by $\leq 3.7 \times 10^{-4}$ nats. *Physical gate*: an RGC cannot fire more than ~ 100 spikes in the window, so the sensible regime is $\sigma_g^2 \lesssim 6$, within which (a) is a few-percent effect.

(b) Truncation at $r_{\max}$. If the predictive puts non-negligible mass above $r_{\max}$, both $-p(r) \log p(r)$ and $p(r) \log r!$ are clipped, biasing *both* H_{marg} and the closed-sum term low. The moments are

$$\mathbb{E}[R] = e^{\mu_g + \sigma_g^2/2}, \quad \text{Var}(R) = e^{\mu_g + \sigma_g^2/2} + e^{2\mu_g + \sigma_g^2}(e^{\sigma_g^2} - 1),$$

the second (log-normal-of-rate) term dominating once $\sigma_g^2 \gtrsim 1$. As σ_g^2 grows the predictive is strongly right-skewed and its centres separate: the mean $e^{\mu_g + \sigma_g^2/2}$ climbs, the median stays $\approx e^{\mu_g}$, the mode drifts *down* toward 0 ($\approx e^{\mu_g - \sigma_g^2}$) — so “ $p(r)$ peaks at e^{μ_g} ” is only the small- σ_g^2 limit. A sufficient benign-truncation condition is $r_{\max} \gtrsim \mathbb{E}[R] + k\sqrt{\text{Var}(R)}$. The code `compute_adaptive_rmax` is stricter (a k - σ upper tail in g -space, exponentiated to a rate, plus a Poisson std, plus a margin):

$$r_{\max}^{\text{adapt}} = e^{\mu_g + k\sigma_g} + 5\sqrt{e^{\mu_g + k\sigma_g}} + 10, \quad k = 3,$$

clamped to $[200, 10000]$ with a `ValueError` if the needed $r_{\max}$ exceeds the ceiling (never silently over-truncates; a further guard raises if $\max_rmax > e^{80}$ so the overflow clamp cannot hide failures). The fixed default $r_{\max} = 100$ is benign for confident predictions but *not safe in general*: it fails whenever $\mu_g \gtrsim \log 100 \approx 4.6$ or σ_g^2 grows large — exactly the high-uncertainty regime active learning explores.

(c) Float32 overflow of the exact closed-form term. The exact term $-e^{\mu_g + \sigma_g^2/2}(\mu_g + \sigma_g^2 - 1)$ overflows float32 when $\mu_g + \frac{1}{2}\sigma_g^2 \gtrsim 88$, sending $\mathbb{E}_g[H_{\text{noise}}] \rightarrow -\infty$ and $U_{\text{std}} \rightarrow +\infty$. This is the *only* residual “exp can diverge” issue, and it lives in an *exact analytical* term, not the Laplace approximation — distinct from the historic bug. In practice truncation (b) bites long first ($\sigma_g \sim 1 \Rightarrow$ truncation near $\mu_g \sim 1$ at $r_{\max} = 100$, vs $\mu_g \sim 88$ here).

(d) Catastrophic cancellation. Inherent to the decomposition (survives even with exact Laplace and no truncation). *Inside* $\mathbb{E}_g[H_{\text{noise}}]$ (large, earliest): both $-e^{\mu_g + \sigma_g^2/2}(\mu_g + \sigma_g^2 - 1)$ and $\sum p(r) \log r!$ grow $\sim e^{\mu_g}$, while the true $\mathbb{E}_g[H_{\text{noise}}]$ grows only *linearly* ($\sim \frac{1}{2}\mu_g$); at $\mu_g = 10$ each term $\sim 2 \times 10^5$ vs a true value ~ 6 (~ 4.5 digits cancel), at $\mu_g = 15 \sim 6.7$ digits cancel — essentially noise in float32’s ~ 7 -digit mantissa (and truncating the second sum makes this *worse*). *Between* H_{marg} and $\mathbb{E}_g[H_{\text{noise}}]$ (smaller, second-level): by Jensen $H_{\text{marg}} \geq \mathbb{E}_g[H_{\text{noise}}]$ while both grow linearly in μ_g , a second cancellation costing fewer digits. Unfixable without rewriting the decomposition so cancelling pieces are computed together (e.g. summing $-p(r) \log p(r) - p(r) \log r!$ as a single term before any large analytical exponential). Flagged, out of scope.

8.6.4 “Utility diverges under gradient ascent,” re-examined

The claim is *two* distinct things. (i) **Kernel-driven input blow-up:** $K(\kappa x, \kappa x) = \kappa^2 K(x, x) \Rightarrow$ ascending U_{std} with no norm constraint sends $\sigma^2 \propto \|x\|^2 \rightarrow \infty$, hence $\sigma_g^2 \rightarrow \infty$ (the mechanism of §8.4–8.5). (ii) The **computed** utility then misbehaves via the four numerical modes, typically in order: truncation collapse of H_{marg} (b), then Laplace error at large σ_g^2 (a), then cancellation inside $\mathbb{E}_g[H_{\text{noise}}]$ (d) as μ_g grows, finally float32 overflow (c) at extreme scales. The kernel makes the inputs unbounded; then numerics make the computed utility ill-behaved — not the same thing, and the four numerical effects are not the same thing either. Numerical fixes alone will not cure it: either constrain the optimizer (norm projection, or the normalized kernel of Proposition 8.15) or rewrite the closed-form term in an asymptotically stable form.

Caveat. Utility-accuracy ceiling (known limitation). The standard utility can run to $+\infty$ and the DA utility can silently collapse, both via the $r_{\max} = 100$ truncation, at high firing; the production loop’s $f_{\max}$ guard keeps generation in the accurate regime, and a DA-path lookup-table was deliberately not built. This is a documented limitation, not a resolved result.

Part IV

From selection to synthesis: guided diffusion

9 Guided diffusion for stimulus synthesis

Parts I–III chose the next stimulus by *selecting* the image of highest information utility from a fixed pool. This part develops the natural extension: *synthesize* a new natural-image-like stimulus that maximizes the same Gaussian-process (GP) utility for a target retinal ganglion cell (RGC). A denoising diffusion model supplies a learned prior over natural images, and the GP utility gradient—the very object of Part III—steers generation toward informative stimuli. The two ingredients divide the labor cleanly: the utility gradient (smooth, low-pass; §9.1) fixes *where* in image space to move, and the diffusion score supplies the high-frequency naturalness a GP utility gradient can never produce on its own.

Caveat. This is an exploratory, mid-flight investigation, represented honestly. The mathematics below is sound and matches the current code, but as of the 2026-07 runtime audit the two generation scripts are *stale against the current GP engine* (they crash out-of-the-box, §9.4) and guided generation is *fragile* (some random seeds collapse). Nothing here has been validated on real neurons. Ground truth for “what is implemented” is the code, not the older `.tex/.md` design notes, several of which are superseded (§9.4).

9.1 Motivation: synthesize rather than select

Why direct gradient ascent on the utility fails. Suppose we optimize an image x^* directly to maximize a GP utility $U(x^*)$ by gradient ascent. The gradient flows through the arc-cosine kernel’s input-gradient $\nabla_x K$ (Part I, (10)), which carries the structured spatial prior C and hence its Gaussian smoothing block C_{smooth} —a convolution of the pixel field with a Gaussian, i.e. a *low-pass filter*. Two consequences follow, independent of the inducing points, the training data, or which utility is used:

1. the utility gradient is *inherently smooth*, so ascent images lack the high-frequency structure (edges, textures, local contrast) of natural images; and
2. ascent pushes pixels outside the physical display (DMD) range. The arc-cosine self-kernel is positively homogeneous, $K(x, x) = x^T C x + \sigma_0^2 = \|x\|_C^2 + \sigma_0^2$, so larger norms are rewarded and the utility diverges under input scaling (the norm-scaling divergence of Part III).

Such images cannot serve as physiological stimuli: to be shown to real RGCs they must be indistinguishable from natural images.

Why a subspace constraint is not enough. Restricting the optimization to a low-dimensional subspace (PCA space, or the C -eigenspace of Part III) does not cure this. PCA space fixes only the *second-order* statistics; under approximate stationarity of natural-image statistics its eigenvectors tend to Fourier modes (Szegő’s theorem), so PCA-space ascent is essentially Fourier low-pass filtering. Neither PCA space nor the C -eigenspace enforces the higher-order structure (phase coherence,

sparse edges) that makes an image look natural: a subspace constraint is a convergence aid, not a naturalness constraint.

The decomposition, and why diffusion. The synthesis task has two objectives that pull apart cleanly: (i) *informativeness*— x^* maximizes the utility (reduces response uncertainty); and (ii) *naturalness*— x^* is a plausible draw from the natural-image distribution $p(x)$. Gradient ascent handles (i) but not (ii). A diffusion model trained on natural images learns the *score* $\nabla_x \log p_t(x)$ at every noise level t ; at low noise this encodes the full statistical structure of natural images. Guided generation adds the two:

$$\underbrace{s_\theta(x_t, t)}_{\text{high-freq. naturalness}} + \underbrace{w \nabla_{x_t} U(\hat{x}_0)}_{\text{low-freq. utility bias}}. \quad (90)$$

Remark 9.1 (Division of labor). The smoothness that ruined direct optimization becomes acceptable under (90): the utility only has to *bias* generation toward informative regions, not prescribe individual pixels. The score model supplies all orders of image statistics (trained once, reused across cells and utilities); the utility gradient supplies the cell-specific, low-frequency direction. They act at complementary spatial-frequency scales.

Closed-loop connection. In the fixed-pool active-learning loop, the experiment *picks* the pool image of highest expected information gain about the neuron’s tuning. Diffusion-guided synthesis is the extension: *manufacture* a stimulus of even higher utility that is still natural enough to present on the DMD—closing the gap between “best available pool image” and “best possible informative stimulus.” The package is framed by three questions: (1) can we generate images that are simultaneously natural-looking and high-utility? (2) how does that image change as the GP sharpens from $N = 50$ to 300 training images? (3) how much utility does the naturalness constraint cost—the gap between the unconstrained pure-utility optimum and the diffusion-guided one, as a function of guidance strength w ?

9.2 Diffusion fundamentals

9.2.1 Forward (noising) process

Let $x_0 \in \mathbb{R}^d$ be a clean image ($d = 64 \times 64 = 4096$ in the implementation). Fix a variance schedule $\beta_t \in (0, 1)$, $t = 1, \dots, T$, and define the per-step signal retention $\alpha_t = 1 - \beta_t$ and its cumulative product $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s = \prod_{s=1}^t (1 - \beta_s)$. The forward process adds Gaussian noise one step at a time,

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad (91)$$

and admits a closed-form marginal at arbitrary t (the key property—no intermediate steps are needed):

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I). \quad (92)$$

Equivalently, by the reparameterization

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (93)$$

The schedule is chosen so that $\bar{\alpha}_T \approx 0$, hence $q(x_T | x_0) \approx \mathcal{N}(0, I)$: at the end of the forward process all information about x_0 is destroyed.

9.2.2 Score $\leftrightarrow$ noise, and the training objective

Differentiating the log of the forward marginal (92) gives the score in closed form, which identifies it with the added noise:

$$\nabla_{x_t} \log q(x_t | x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{1 - \bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}} \implies \nabla_{x_t} \log p_t(x_t) \approx s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}. \quad (94)$$

A network trained to predict the noise therefore approximates the score. Training minimizes the simplified denoising-score-matching (DSM) L_2 loss

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{t \sim U(1, T)} \mathbb{E}_{x_0 \sim p(x)} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [\|\epsilon_\theta(x_t, t) - \epsilon\|^2], \quad (95)$$

with x_t formed by (93). Each mini-batch draws a fresh (x_0, t, ϵ) ; the network learns all noise levels though it sees each infrequently.

9.2.3 Reverse (denoising) process and the Tweedie estimate

The generative reverse process is Gaussian, $p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$, with mean read off from the noise predictor

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{1 - \beta_t}} \left[x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right], \quad (96)$$

and fixed variance σ_t^2 , either the simple choice $\sigma_t^2 = \beta_t$ or the posterior form $\tilde{\beta}_t = \beta_t(1 - \bar{\alpha}_{t-1})/(1 - \bar{\alpha}_t)$; in practice $\sigma_t^2 = \beta_t$ works well. Sampling runs $t = T, \dots, 1$ from $x_T \sim \mathcal{N}(0, I)$, drawing $x_{t-1} = \mu_\theta + \sigma_t z$ with $z \sim \mathcal{N}(0, I)$ ($z = 0$ at $t = 1$). At any t the noise prediction yields the *Tweedie estimate* of the clean image—the model’s posterior mean $\mathbb{E}[x_0 | x_t]$:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \quad (97)$$

It is coarse at high noise and increasingly detailed as $t \downarrow$. The Tweedie estimate (97) is the workhorse of guidance (§9.3): it maps a noisy intermediate to a clean image on which the GP utility can be evaluated, and it is a differentiable function of x_t .

Remark 9.2 ($T - 1$ start). At $t = T$ the schedules used here have $\bar{\alpha}_T \sim 10^{-3}$, so $1/\sqrt{\bar{\alpha}_T} \sim 31$ and the first reverse step amplifies any prediction error $\sim 31\times$, causing divergence. Both models start the loop at $t = T - 1$ (where $1/\sqrt{\bar{\alpha}} \approx 2$, stable).

9.2.4 The U-Net noise predictor and DDIM acceleration

Because $\epsilon_\theta(x_t, t)$ maps a noisy image to a same-size tensor, it is parameterized by an encoder–decoder with *skip connections* (a U-Net): a plain bottleneck would discard exactly the fine detail that distinguishes a natural image from noise, whereas skip connections carry per-resolution detail directly to the decoder. The scalar timestep t enters through a sinusoidal embedding (Transformer-style),

$$\text{emb}(t)_{2k} = \sin(t/10000^{2k/d_{\text{emb}}}), \quad \text{emb}(t)_{2k+1} = \cos(t/10000^{2k/d_{\text{emb}}}),$$

passed through a small MLP and *added* (broadcast over space) to each block’s feature map—a global “how much to activate at this noise level” modulation.

Full DDPM sampling needs $T - 1 = 999$ sequential U-Net passes. *DDIM* replaces the stochastic reverse chain by a deterministic one over a subsequence of S timesteps $\tau_S > \dots > \tau_1$ (e.g. $S = 50$). Going from $t = \tau_i$ to $t' = \tau_{i-1}$,

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'} - \sigma^2} \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}} + \sigma z, \quad (98)$$

with $\hat{x}_0$ from (97). Here $\sigma = 0$ gives a fully deterministic, reproducible sampler; the DDPM choice of σ recovers stochastic sampling. Quality degrades below $S \approx 20$; $S = 50$ is the default.

9.3 Guidance: steering the reverse process with the GP utility gradient

9.3.1 Classifier-style guidance

Model “high utility” as a label y with $p(y \mid x_0) \propto \exp(wU(x_0))$, $w > 0$. Bayes’ rule splits the *conditional* score into a naturalness term and a guidance term:

$$\nabla_{x_t} \log p(x_t \mid y) = \underbrace{s_\theta(x_t, t)}_{\text{unconditional score, (94)}} + w \nabla_{x_t} U_{\text{std}}(\hat{x}_0(x_t, t)). \quad (99)$$

The utility is defined on *clean* images, so it is evaluated on the Tweedie estimate (97) (the standard Dhariwal–Nichol trick). Any of the Part III utilities may play the role of U in (99): the production standard utility U_{std} ((69)) shown here, or the distribution-aware utility U_{DA} ((62)), which is in fact the generation script’s default. Define the *guidance gradient*

$$g_t = \nabla_{x_t} U_{\text{std}}(\hat{x}_0(x_t, t)). \quad (100)$$

9.3.2 The guided reverse update (two equivalent forms)

Converting the guided score (99) back to a noise prediction, and substituting into the Tweedie map (97), yields two algebraically identical updates:

$$\epsilon_{\text{guided}} = \epsilon_\theta(x_t, t) - w \sqrt{1 - \bar{\alpha}_t} g_t, \quad (101)$$

$$\hat{x}_0^{\text{guided}} = \hat{x}_0 + \frac{w(1 - \bar{\alpha}_t)}{\sqrt{\bar{\alpha}_t}} g_t. \quad (102)$$

Form (101) feeds the DDPM mean (96) in place of ϵ_θ ; form (102) feeds the DDIM update (98) in place of $\hat{x}_0$. The *ramp factor* $(1 - \bar{\alpha}_t)/\sqrt{\bar{\alpha}_t}$ in (102) is large at high noise ($t \approx T$, ~ 158), giving bold utility steering while global structure is still being decided, and small at low noise ($t \approx 1$, ~ 0.3), preserving fine detail.

9.3.3 The GP link: what the guidance signal is

The guidance gradient (100) is the gradient of the GP information utility of Part III, back-propagated through Tweedie. Both utilities are differentiable in the image, and the gradient flows through the arc-cosine kernel’s input-gradient $\nabla_x K$ (Part I, (10)). Because $\nabla_x K$ carries the factor C —and thus the Gaussian spatial smoother C_{smooth} —the utility gradient is exactly the smooth, low-pass signal of §9.1. This is precisely why guidance works: the utility supplies a *low-frequency* informative direction, and the score model supplies the *high-frequency* naturalness the smooth utility gradient can never produce.

The receptive-field mask $\Rightarrow$ guidance touches only RF pixels. The structured prior applies a Gaussian RF envelope to each pixel j ,

$$\alpha_j = \exp\left(-\frac{\|\xi_j - \xi_0\|^2}{4\beta^2}\right), \quad (103)$$

where ξ_j is pixel j 's coordinate, ξ_0 the RF center, and β the *GP kernel RF half-width*. Pixels with $\alpha_j < 10^{-3}$ are masked out (zero contribution), so $\nabla_x U$ is nonzero *only at RF pixels*. The mask area scales as β^2 : at the default $\beta = 0.1$ the RF covers $\approx 21\%$ of the 64×64 image (≈ 869 pixels), versus 5.4% at $\beta = 0.05$ and 1.7% at $\beta = 0.03$. The utility therefore guides only the RF region, and *the diffusion model freely generates every remaining pixel* for naturalness—an advantage over the L_2 approach (§9.3.5) whose non-RF pixels are frozen at a start image. Because β is a learnable kernel parameter, it is frozen during GP training for synthesis, or the intended mask size drifts.

Caveat. The symbol β in (103) is the *GP kernel RF half-width* (Part I), **not** the diffusion noise-schedule variance β_t of (91); likewise the RF envelope α_j is the GP locality mask, not the diffusion signal-retention α_t . These collisions (β , α) are the most dangerous in the whole document; the macros keep them apart.

9.3.4 Full versus approximate guidance gradient

By the chain rule g_t factors through the utility gradient in clean-image space and the Tweedie Jacobian,

$$\frac{\partial \hat{x}_0}{\partial x_t} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(I - \sqrt{1 - \bar{\alpha}_t} \frac{\partial \epsilon_\theta}{\partial x_t} \right), \quad (104)$$

where $\partial \epsilon_\theta / \partial x_t$ is the $d \times d$ U-Net Jacobian, never formed explicitly (PyTorch computes the vector-Jacobian product via `backward()` on $-U(\hat{x}_0)$ with x_t a leaf tensor and the U-Net weights frozen).

- **Full gradient** (`-use-full-gradient`): retains $\partial \epsilon_\theta / \partial x_t$ in (104); one U-Net forward *and* backward, $\approx 2\times$ the cost of a plain step.
- **Approximate gradient** (default): run the U-Net under `no_grad`, treating its Jacobian as negligible. Then $\hat{x}_0$ depends on x_t only through the direct $x_t / \sqrt{\bar{\alpha}_t}$ term, so

$$g_t^{\text{approx}} = \frac{1}{\sqrt{\bar{\alpha}_t}} \nabla_{\hat{x}_0} U_{\text{std}}(\hat{x}_0). \quad (105)$$

No U-Net backward pass ($\approx \frac{1}{2}$ the cost). It is justified when ϵ_θ is a “local correction” whose Jacobian is roughly zero-mean (does not systematically align with or oppose the utility gradient), which holds best at *low* noise (small $\sqrt{1 - \bar{\alpha}_t}$); a hybrid (approximate for early $t > 200$, full for late $t < 200$) captures most of the benefit cheaply.

9.3.5 The four approaches (A–D) and the adopted method

Four guidance paradigms were catalogued; all attempt to add a naturalness term counteracting the utility gradient’s drift off the natural-image manifold $\mathcal{M}$. Let x_c denote a current image being optimized.

A — L_2 penalty toward the Tweedie estimate. Corrupt x_c at a *fixed* noise level t^* , predict $\hat{\epsilon}$, form $\hat{x}_0$ ((97)), *detach* it, and penalize the distance to it:

$$\mathcal{L}_A = -U_{\text{DA}}(x_c) + \frac{\gamma_{\text{reg}}}{2} \|x_c - \hat{x}_0\|^2, \quad x_c \leftarrow x_c + \eta \nabla U_{\text{DA}} + \eta \gamma_{\text{reg}} (\hat{x}_0 - x_c). \quad (106)$$

Here γ_{reg} is the approach-A regularization weight (code `-lambda-diff`). Since $x_c - \hat{x}_0 \propto (\hat{\epsilon} - \epsilon)$, (106) is a Score-Distillation-Sampling gradient with a different time weighting. *Weaknesses:* no signal near $\mathcal{M}$ (for natural x_c , $\hat{x}_0 \approx x_c$ so $\hat{\epsilon} - \epsilon$ is just reconstruction noise—the penalty is reactive, not preventive); the Tweedie estimate of an amplified image is itself amplified, so norm growth is not prevented; t^* is arbitrary; a single fixed noise sample is biased.

- B — direct score.** Use $s_\theta(x_t, t^*) = -\hat{\epsilon}/\sqrt{1 - \bar{\alpha}_{t^*}}$ directly, chain-ruled to x_c -space. Uses $\hat{\epsilon}$ alone (not $\hat{\epsilon} - \epsilon$), so it has signal on $\mathcal{M}$ but high variance, and estimates the score of the *blurred* p_{t^*} , not p_0 .
- C — score without added noise.** Feed x_c at $t = 1$ directly. Appealing ($p_1 \approx p_0$) but fails: a utility-distorted x_c is far out-of-distribution for a network expecting “natural + $\sigma \approx 0.01$ noise,” and dividing by $\sqrt{1 - \bar{\alpha}_1} \approx 0.01$ amplifies the unreliable prediction $\sim 100\times$.
- D — full guided reverse diffusion (*adopted*).** Do not optimize an existing image—*generate* from noise, nudging each denoising step by the utility gradient via (99)–(102). The U-Net is *always* in-distribution (x_t is genuinely at noise level t because it came from the reverse process); generation traverses coarse $\rightarrow$ fine over all noise levels; there is no t^* to choose; the output is a proper sample from the guided distribution. Costs: the U-Net forward (with gradients in full mode), T (or ~ 50 DDIM) sequential steps, utility evaluated on rough early Tweedie estimates, stochastic output (best-of- K), and no start-point control.

Remark 9.3 (Why the score is a genuine restoring force). Naive per-pixel clipping $\Pi_{\mathcal{B}}$ (each x_i clamped to $[x_{\min}, x_{\max}]$) acts on each pixel independently and ignores the joint distribution $p(x)$: it lands on the boundary of the display box but far from $\mathcal{M}$. In the guided score (99) the score term is instead a *structured* restoring force on the joint distribution—if a patch saturates and loses its natural spectrum, $p_t(x_t)$ drops sharply and the score rearranges pixels to restore natural correlations while accommodating the utility bias. The limitation is out-of-distribution failure: ϵ_θ was trained only on the forward trajectory (natural + Gaussian noise), so too large a w drives x_t where ϵ_θ extrapolates and the restoring force becomes unreliable. Thus w must be large enough to explore high-utility regions yet small enough to keep x_t where the score approximation is valid.

9.3.6 The firing-rate guard

At high noise the Tweedie estimate is rough and can produce extreme pixels, hence unphysical firing rates that fall in the regime where the truncated utility is inaccurate (§9.4). The generator therefore zeroes the guidance gradient whenever the predicted log-firing rate is too high:

$$g_t \leftarrow 0 \quad \text{whenever} \quad \exp(\mu_g) \geq f_{\max}, \quad \mu_g = A\mu + \lambda_0, \quad (107)$$

with $f_{\max} = 100$ and μ_g the predicted log-firing mean (gain A times the posterior latent mean μ plus the offset λ_0). This guard fires mostly in the first ~ 10 steps (high t) and keeps generation in the low-firing regime where the (truncated) utility matches Monte-Carlo truth. Two further practical guards accompany it: the raw-pixel/model-space rescale $\hat{x}_0^{\text{raw}} = \hat{x}_0 \cdot s$ ($s \approx 2.478$) *must* stay inside the autograd graph or the gradient from U back to x_t vanishes; and, because the utility landscape over seeds is sparse (most seeds land at $U \approx 0$), generation is run best-of- K (keep the highest-utility image over K random seeds).

9.4 Implementation status and open issues

The investigation is honest about being mid-flight. The math above is faithful to the current code, but the surrounding tooling and validation are incomplete.

Two models, two approaches. Approach A and Approach D use *different* diffusion models—a point on which several design notes are wrong.

	Small “from-scratch”	Large “pretrained”
Params	2.16M	99.5M
Schedule	cosine, $T = 1000$	linear $\beta_t \in [10^{-4}, 0.02]$, $T = 1000$
Training	PNAS only	ImageNet-pretrained $\rightarrow$ PNAS-finetuned
Used by	Approach A (<code>guided_optimization.py</code>)	Approach D (<code>guided_reverse.py</code>)

The small model gives decent *unconditional* DDPM samples but fails under DDIM (blurry gradients, not images); the 99.5M model is sharp under both DDPM and DDIM, which is why it was adopted for the guided reverse path.

Caveat. Doc-vs-code (item 9 of the reconciliation worklist). (i) `guided_reverse_diffusion.tex` states Approach D uses the 2.16M/cosine U-Net; **it actually loads the 99.5M/linear model**—trust the code and the summary. (ii) `REFERENCE.md` calls the Approach-A optimizer “SGD with momentum”; the code and the 2026-03-31 session log show it is **LBFGS** (a proximal/MM pattern with the Tweedie target recomputed once per outer step), and `REFERENCE.md`’s constants are stale too. (iii) `motivation_*.md` says “ 30×30 , no implementation yet”; the real resolution is 64×64 and both approaches are implemented. Treat the *reasoning* in those notes as current, the *numbers and status* as historic.

Caveat. Both scripts are stale against the current GP engine and crash out-of-the-box. They pass `stop_window/stop_thresh` to the trainer, which the ELBO early-stopping refactor removed (now `patience/min_delta_rel/es_metric`) $\Rightarrow$ `KeyError`; the two kwargs must be deleted. Separately, `guided_reverse.py`’s `DEFAULT_MODEL_PATH` points at a nonexistent location, so the real finetuned model path must be passed explicitly. The fixes are trivial but the scripts do not run as shipped.

Caveat. Generation is fragile. In a representative run ($w = 50$, DDIM-50, 3 seeds) one seed of three collapsed to salt-and-pepper noise ($U = 0$, most pixels out of display range) because the firing-rate guard (107) fired on 49/50 steps and guidance never engaged; the best seed gave a textured-but-not-clearly-natural image. Best-of- K with $K \geq 10$ is required. Guidance-scale calibration is kernel-dependent (arc-cosine needs $w \sim 50$; RBF is effective near $w \sim 5$).

Caveat. Utility-accuracy ceiling at high firing. Both utilities share the Poisson-sum truncation at $r_{\max} = 100$ (Part III): past firing ≈ 50 the *standard* utility diverges to $+\infty$ (unbounded analytic noise-entropy term overflows) and the *distribution-aware* utility stays finite but is silently wrong (its marginal entropy collapses toward 0). The firing-rate guard (107) is what keeps generation in the firing ≈ 0.01 –1 zone where both match Monte-Carlo truth within $\sim 3\%$. An exact lookup table cures the standard path’s overflow but is wired only for `-utility standard` in the standalone package—**not** on the working branch and **not** for the distribution-aware path (a deliberate non-goal). Note also that the guided reverse path’s default distribution-aware utility conditions on a *single* sample, the target image itself ($n_{\text{mc}} = 1$), rather than averaging over $p(x)$.

Caveat. No neuroscience validation. Whether the naturalness constraint actually buys better experimental outcomes on real RGCs—and how the synthesized image evolves as the GP sharpens from $N = 50$ to 300—has not been tested; closed-loop and oracle validation are out of scope for the current package. The open method items include the norm-amplification fix for Approach A (the L_2 -Tweedie pull preserves structure, not amplitude), an SDEdit-style start-from-a-chosen-image variant (proposed, not implemented), and a systematic full-vs-approximate gradient sweep at 64×64 .

A Notation reference

One symbol denotes one concept throughout. Entries marked **[clash]** are symbols that collide across the source literature and are disambiguated here.

Symbol	Concept
<i>Model, kernel, structured prior</i>	
x, x^*	stimulus image (vectorized natural image); candidate image
ξ_i, ξ_0	pixel coordinate; RF center ($\varepsilon_{0x}, \varepsilon_{0y}$)
$\lambda(x)$	latent GP function (the tuning latent) [clash: not λ_0]
λ_0	firing-rate offset
$f(x) = e^{A\lambda + \lambda_0}$	firing rate [clash: f is the rate, never the latent]
$g = A\lambda + \lambda_0$	log-firing rate (so $f = e^g$)
A	Poisson gain [clash: not the projection a]
$y (\equiv r)$	spike count (synonyms r, n, target)
$K(x, x')$	arc-cosine prior kernel function
$\widetilde{K} = K(\widetilde{Z}, \widetilde{Z})$	inducing gram [clash: never a bare K for this]
$\mathcal{M} = \sqrt{v_x v_{x'}}$	kernel magnitude [clash: not M]
$\theta, J(\theta)$	kernel angle; angular term [J is never a Jacobian]
$v_x = x^\top Cx + \sigma_0^2$	kernel self-magnitude
C, C_{smooth}	structured spatial prior; RBF smoothing factor
α_i	locality mask / RF envelope [clash: not diffusion α_t]
β	RF half-width [clash: not diffusion β_t]
ρ	smoothness SD [clash: not correlation ρ_λ]
σ_0	bias std, $\sigma_0 = e^{\text{raw_sigma_0}}$
Amp	amplitude — kernel hyperparameter, Amp = softplus(raw_Amp), bounds (0, 1000], optimized by default
<i>Variational inference and eigenspace</i>	
$\widetilde{z}, \widetilde{Z}$	inducing point; inducing-point matrix
$\widetilde{\lambda} (\equiv u)$	inducing latent values, $q(\widetilde{\lambda}) = \mathcal{N}(m, V)$
M, N	number of inducing points; number of training images ($M=N$ in the loop)
$k(x), u(x) = \widetilde{K}^{-1}k(x)$	cross-covariance vector; projection vector
m, V	variational mean and covariance (inducing space)
$\mu(x), \sigma^2(x)$	posterior mean and variance of the latent
$L_{\widetilde{K}}$	Cholesky factor of $\widetilde{K}$ [clash: $\mathcal{L}$ is the ELBO]

Symbol	Concept
B, Λ, n_b	eigenbasis of $\widetilde{K}$; eigenvalues (diagonal); kept dimension [Λ not λ]
$m_b, V_b, \widetilde{K}_b$	eigenspace mean, dense covariance, diagonal inducing gram
$a = K\widetilde{K}^{-1}$	projection matrix (KKtilde_inv_b)
$\bar{f} = e^{A\mu + \frac{1}{2}A^2\sigma^2 + \lambda_0}$	expected firing rate (f_mean)
$\mathcal{L}$	evidence lower bound [clash: script $\mathcal{L}$, not the Cholesky factor]
<i>Training-step helpers</i>	
$g_E = A \sum_i k_i (y_i - \bar{f}_i)$	E-step gradient helper [clash: not the log-rate g]
$G_E = A^2 \sum_i \bar{f}_i k_i k_i^\top$	E-step Fisher curvature (PSD)
<i>Utility and information</i>	
$U_{\text{std}}, U_{\text{DA}}$	standard (production) and distribution-aware (research) utility
$H_{\text{marg}}, H_{\text{noise}}, H_{\text{cond}}$	marginal, aleatoric-noise, and conditional entropy [clash: never a bare H]
$p(x)$	natural-image distribution
$\mu_g = A\mu + \lambda_0, \sigma_g^2 = A^2\sigma^2$	log-rate mean and variance
$\rho_\lambda(x, x^*)$	posterior latent correlation [clash: not smoothness ρ; $\rho_\lambda^2 = \text{fractional variance reduction}$]
c	prior conditional covariance (the “correction” term)
κ	input scaling factor, $x^* = \kappa x_t$ [clash: not c]
σ_r^2, ζ	residual/aleatoric variance; subspace coordinate [clash: not an inducing point]
$W_0, \bar{g}_r, r_{\text{max}}$	Lambert- W (principal); Laplace mode; count truncation
σ_f^2	saturation-kernel amplitude
<i>Diffusion (Part IV only)</i>	
t	diffusion timestep [there is no temporal RF: spatial GP only]
$x_0, x_t, \hat{x}_0$	clean, noisy, and Tweedie-estimated image
β_t	noise-schedule variance [clash: not GP β]
$\alpha_t, \bar{\alpha}_t$	signal retention; cumulative retention [clash: not GP mask α]
$\epsilon_\theta, s_\theta$	U-Net noise predictor; score function
w, g_t	guidance scale; guidance gradient (= the GP utility gradient)

Symbol	Concept
γ_{reg}	approach-A regularization weight [clash: not the GP latent λ]

B Reconciled discrepancies and open caveats

The document is anchored to the current code. The following notes-vs-code and notes-vs-notes discrepancies were reconciled during consolidation (code wins), and the honest open caveats are collected for the reader.

B.1 Reconciled (code is authoritative)

E-step mean update. The “corrected” derivation gives $m \leftarrow m + \widetilde{K}(\widetilde{K} + G_E)^{-1}(g_E - m)$; the code uses the older $m \leftarrow \widetilde{K}(\widetilde{K} + G_E)^{-1}(G_E \widetilde{K}^{-1}m + g_E)$ ((39)). Both maps share the fixed point $m^* = g_E$, so the *converged* posterior is identical; only the transient differs (§6.2).

Constraint transforms. $\sigma_0 = \exp(\text{raw})$ and $\text{Amp} = \text{softplus}(\text{raw})$; the handoff note claiming “softplus for sigma_0” is wrong (§6.4).

Two gradient systems. The “compute dK once, single contraction” form is the kernel-level autograd backward, *not* the M-step, which recomputes C, dC, K, dK inside the LBFGS closure and runs the full moment/loss chain (§6.4).

Production utility. The shipped acquisition is U_{std} (`standard_utility`); the distribution-aware U_{DA} is the research variant derived in the write-ups (§7).

Deprecated helper. `lambda_moments` does not exist in the current utility code; the moments come from `get_gp_marginal_moments` and `get_gp_conditional_moments`.

Cross-covariance. The manual Σ_{x,x^*} is unreliable outside the inducing hull; the shipped predict path slices GPyTorch’s joint `covariance_matrix`, which already includes c (§4.7, Rem. 7.4).

Diffusion specs. The adopted guided-reverse method (Approach D) uses the large linear-schedule model, not the small cosine model; Approach A optimizes by LBFGS, not SGD; some prose describing sizes/status is stale (§9.4).

B.2 Open caveats (carried honestly)

Checkpoint basis drift. The checkpoint saves m_b, V_b but not the eigenbasis B , which is recomputed on load; a different torch version can yield a subtly different basis and $\sim 5\%$ drift (§5.9).

Utility accuracy ceiling. At high firing the standard utility can diverge and the marginal entropy collapses through the r_{max} truncation; a firing guard keeps generation in the accurate regime (§8.6, §9.4).

Subspace offset. In the C-eigenspace optimization the non-zero offset equal to the mean is not mathematically motivated; it compensates a limiter precision issue and remains to be investigated (§8.3).

Divergence under free ascent. Because the arc-cosine kernel is positively homogeneous, unconstrained gradient ascent on the image inflates the utility without bound (Thm. 8.9); this is why a subspace/norm constraint or a naturalness prior (Part IV) is required.

C Glossary of experimental terms

Retinal ganglion cell (RGC) the output neuron of the retina whose spikes are recorded; the GP models one RGC's tuning at a time.

Receptive field (RF) the region of the visual field (here, of the image) to which the neuron responds; encoded by the structured prior center ξ_0 and width β .

Multi-electrode array (MEA) the recording device that captures spikes from many RGCs simultaneously.

Spike count the number of spikes y a neuron fires in the response window to a displayed image; the observed data.

Firing rate the instantaneous rate $f = e^{A\lambda + \lambda_0}$; the Poisson intensity generating the spike count.

Tuning how the neuron's firing rate depends on the stimulus image; the latent function λ the GP infers.

Natural image a grayscale photograph-like stimulus (as opposed to synthetic gratings); the stimulus domain the prior is designed for.

Closed loop the online cycle: display an image, count spikes, update the GP, choose the next image.

Active learning choosing the next stimulus to maximize expected information gain about the neuron's tuning, rather than sampling at random.